

Deep Reinforcement Learning applied to Forex Market: Deep Deterministic Policy Gradient Trading System

Vicente Aser Marques Rebelo Castillo Lorenzo

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisor: Prof. Rui Fuentecilla Maia Ferreira Neves

Examination Committee

Chairperson: Prof. Name of the Chairperson
Supervisor: Prof. Rui Fuentecilla Maia Ferreira Neves
Member of the Committee: Prof. Name of First Committee Member

October 2026

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

Abstract

Reinforcement learning for currency trading is usually tested on one pair, on bar data and without real costs. This thesis trains one Deep Deterministic Policy Gradient (DDPG) agent, with one design and no tuning per pair, on each of the seven major currency pairs, over eleven years of tick data from 2015 to 2025. The agent is implemented as in the original DDPG paper: an off-policy actor-critic with actor and critic networks, slowly updated target copies of both, a replay buffer and Ornstein-Uhlenbeck exploration noise, with the published hyperparameters where they apply. The actor maps 38 dimensionless features from sixteen technical indicators to one continuous action in $[-1, 1]$, whose sign sets the direction of the position and whose magnitude sets its size within a volatility-based risk budget. The critic learns the value of that action, and the reward is the log return of account equity. Four documented changes adapt it to this problem: smaller networks with layer normalisation, a penalty against output saturation, a scaled reward and mirrored training episodes. The agent is trained and evaluated in a tick-level backtesting engine, built for this work to reproduce cTrader's backtester, that replays 1.6 billion quotes and charges the real costs of a raw-spread, swap-free retail account: the spread quoted at each fill, a commission of 3.5 US dollars per lot per side and a swap of zero. Each pair is trained with an anchored walk-forward, the standard validation method for trading systems: seven folds validate on 2018 to 2024, 2025 is held out as the test year, and a model is kept only if it passes eleven checks on return and behaviour. All seven models are profitable net of all costs. They returned between **[WF min]** and **[WF max]** over the walk-forward years, between **[2025 min]** and **[2025 max]** in the held-out year and between **[full min]** and **[full max]** over the full period, with Sharpe ratios of **[Sharpe min]** to **[Sharpe max]** and a yearly alpha of **[alpha min]** to **[alpha max]**, and each beats its own pair in all three windows. As an equal-weight portfolio they returned **[portfolio]** against +233.3% for the US 500 index, with a maximum drawdown of **[portfolio DD]** against 34.2% and a Sharpe ratio of **[portfolio Sharpe]** against 0.764, because the seven models make and lose money at different times.

Keywords

Forex Trading; Deep Reinforcement Learning; Deep Deterministic Policy Gradient; Backtesting; Transaction Costs; Walk-Forward Validation

Resumo

A aprendizagem por reforço aplicada aos mercados cambiais é normalmente testada num único par, com dados em *bars* e sem custos reais. Esta tese treina um agente Deep Deterministic Policy Gradient (DDPG), com um único desenho e sem ajustes por par, em cada um dos sete principais pares de moeda, com onze anos de dados ao *tick*, de 2015 a 2025. O agente é implementado como no artigo original do DDPG: um método ator-crítico *off-policy* com redes do ator e do crítico, cópias-alvo de ambas atualizadas lentamente, memória de repetição e ruído de exploração de Ornstein-Uhlenbeck, com os hiperparâmetros publicados onde se aplicam. O ator transforma 38 variáveis adimensionais, obtidas de dezasseis indicadores técnicos, numa única ação contínua em $[-1, 1]$, cujo sinal define a direção da posição e cuja magnitude define a sua dimensão, dentro de um orçamento de risco baseado na volatilidade. O crítico aprende o valor dessa ação, e a recompensa é o retorno logarítmico do capital. Quatro alterações documentadas adaptam-no a este problema: redes mais pequenas com normalização por camada, uma penalização contra a saturação da saída, uma recompensa escalada e episódios de treino espelhados. O agente é treinado e avaliado num motor de *backtesting* ao *tick*, construído para reproduzir o do cTrader, que percorre 1,6 mil milhões de cotações e cobra os custos reais de uma conta de retalho *raw-spread* e *swap-free*: o *spread* de cada execução, uma comissão de 3,5 dólares americanos por lote e por lado e um *swap* nulo. Cada par é treinado com uma validação *walk-forward* ancorada, o método padrão para sistemas de negociação: sete partições validam de 2018 a 2024, 2025 é reservado como ano de teste, e cada modelo tem de passar onze critérios de retorno e comportamento. Os sete modelos são lucrativos após todos os custos. Obtiveram entre **[WF min]** e **[WF max]** nos anos de validação, entre **[2025 min]** e **[2025 max]** no ano de teste e entre **[full min]** e **[full max]** no período completo, com rácios de Sharpe de **[Sharpe min]** a **[Sharpe max]** e um alfa anual de **[alpha min]** a **[alpha max]**, e cada um supera o seu par nos três períodos. Numa carteira de pesos iguais obtiveram **[portfolio]** contra +233,3% do índice US 500, com uma perda máxima de **[portfolio DD]** contra 34,2% e um rácio de Sharpe de **[portfolio Sharpe]** contra 0,764, porque os sete modelos ganham e perdem dinheiro em momentos diferentes.

Palavras Chave

Negociação Forex; Aprendizagem por Reforço Profunda; Deep Deterministic Policy Gradient; *Backtesting*; Custos de Transação; Validação *Walk-Forward*

Contents

1	Introduction	1
1.1	Motivation	2
1.2	Problem Statement	2
1.3	Objectives	3
1.4	Contributions	3
1.5	Structure	4
2	Background	5
2.1	Financial Markets and Trading	5
2.1.1	Forex Market: Characteristics and Significance	6
2.1.2	Forex Trading: Fundamental Concepts and Operation	6
2.1.3	Forex Costs: Spreads, Commissions and Overnight Financing	8
2.1.4	Financial Analysis: Main Types and Formulas	9
2.1.5	Performance Measurement: Returns, Risk and Ratios	10
2.1.6	Algorithmic and Quantitative Trading	12
2.2	Neural Networks	12
2.2.1	Fundamentals and History	12
2.2.2	Key Concepts and Feed-Forward	13
2.2.3	Learning using Backpropagation	14
2.2.4	Bias-Variance Trade-off and Model Generalisation	15
2.2.5	Architectures for Sequential Data	15
2.2.6	Universal Approximation Theorem	16
2.3	Reinforcement Learning	17
2.3.1	Fundamentals and History	17
2.3.2	Key Concepts and Agent-Environment Interaction	17
2.3.3	Value Functions and Q-Functions	19
2.3.4	Methodological Variants and Learning Methods	20
2.4	Deep Reinforcement Learning	21
2.4.1	Fundamentals and History	21
2.4.2	Deep Value-Based RL Algorithms: Deep Q-Networks	22
2.4.3	Deep Policy-Based RL Algorithms: Deep Policy Gradients	22

3	Related Work	25
3.1	Survey Research: Identifying the State-of-the-Art	25
3.2	Article Research: Summarising Different Implementations	26
3.2.1	Forex Market Implementations	26
3.2.2	Stock Market Implementations	26
3.2.3	Other Market Implementations	27
3.2.4	Comparative Findings	27
3.3	Research Gap: Positioning This Work	28
4	Implementation	31
4.1	Trading Framework: The Environment the Agent Trades In	31
4.1.1	System Architecture: Three Execution Contexts	32
4.1.2	Market Data: Collection, Storage and Resolution	33
4.1.3	Cost Model: Spread, Commission and Account Type	34
4.2	Agent Design: Observation, Action and Reward	36
4.2.1	Observation Space: Indicators, Encoding and Normalisation	36
4.2.2	Action Space: Position Sizing and Risk	38
4.2.3	Reward Function: Log Return, Scaling and Clipping	38
4.2.4	Learning Algorithm: DDPG and its Adaptations	39
4.3	Experimental Procedure: Training, Selection and Deployment	41
4.3.1	Data Splits: Anchored Walk-Forward	41
4.3.2	Training Procedure: Seeds, Episodes and Checkpoints	42
4.3.3	Model Selection: Gates and Composite Ranking	44
4.3.4	Deployment: Decision Frequency and Rebalance Threshold	46
4.3.5	Worked Example: One Position From Decision to Settlement	47
5	Experimental Results	49
5.1	Evaluation Protocol	50
5.2	Benchmarks and Performance Metrics	50
5.3	Selection: How Many Seeds Pass	51
5.4	Walk-Forward Validation: 2018 to 2024	53
5.5	The Test Year: 2025	53
5.6	The Full Window: 2015 to 2025	55
5.7	Trading Behaviour: How the Agents Actually Trade	57
5.7.1	Use of the Continuous Action	58
5.7.2	Direction, Size and Outcome Together	59
5.7.3	Trend Identification and Reversal	60

5.7.4	Holding Periods and Two-Sided Exposure	61
5.7.5	Where the Profit Comes From	61
5.7.6	What the Trading Costs	61
5.8	Alpha and Beta Decomposition	62
5.9	Robustness: Starting Balance and Risk Level	63
5.9.1	Starting Balance	63
5.9.2	Risk Level	65
5.10	Portfolio: The Seven Models Combined	65
5.11	Design Iterations: What Did Not Work	67
6	Conclusion	69
6.1	Summary	69
6.2	Limitations	70
6.3	Future Work	71
	Bibliography	72
A	Technical Indicators	79
B	Reviewed Articles	83

List of Figures

2.1	Candlesticks showing upward (bullish) and downward (bearish) movements respectively [1].	8
2.2	Human Neuron compared to an Artificial Neuron [2].	13
2.3	Interaction between the agent and the environment in Reinforcement Learning [3].	18
4.1	Framework architecture. The strategy receives updates and returns actions. It does not know whether the updates come from a live account, from the platform's own backtester or from the stored quote database.	32
4.2	Monthly mean spread in points for the seven major pairs, 2015 to 2026, computed from the full tick tape. EUR/USD is drawn in black. A model evaluated at a fixed spread is evaluated in market conditions that do not occur in any month of this sample.	35
4.3	The actor. Layer normalisation replaces the batch normalisation of the original, so a single observation is processed exactly as a training batch is. The penalty of Equation (4.7) acts on the pre-activation u , before the tangent. This is where saturation would otherwise remove the gradient.	41
4.4	Anchored walk-forward. Every fold trains from scratch on all the years since 2015 and validates on the next one. The model of the last fold is the delivered model, and it is the only one that trades the test year.	42
4.5	The stages of the procedure. The Calmar ratio chooses among the episodes of one fold. The gates and the composite rank of Section 4.3 choose among seeds.	43
5.1	Yearly return of each model minus the move of its own pair, in percentage points. The years from 2018 to 2024 are traded by the model of their own fold. The last column, after the vertical line, is the test year, traded by the delivered model.	54
5.2	AUD/USD, EUR/USD and GBP/USD against buying and holding the same pair and against the US 500 index, rebased to 100. The shaded bands mark the validation years and the test year.	56
5.3	NZD/USD, USD/CAD and USD/CHF against buying and holding the same pair and against the US 500 index, rebased to 100.	57
5.4	USD/JPY against buying and holding the pair and against the US 500 index, rebased to 100.	57

5.5	Distribution of the magnitude of the daily action over the full window. The dashed line marks full size. No decision reaches it, but most decisions are close to it.	58
5.6	GBP/USD from 2018 to 2025 with the delivered model. From top: the price; the action, positive for long and negative for short; and the net position in thousands of units. The shaded bands mark the validation years and the test year.	59
5.7	GBP/USD from March to June 2025. Background shading marks the direction of the net position, red for short and blue for long. The lower panel gives the size of that position. The marked position is the most profitable one of the delivered model that lasted at least five days.	60
5.8	Alpha against beta for the seven models. Points inside the shaded band have a beta between -0.5 and $+0.5$, so their return hardly depends on the direction of the market. Provisional models are drawn in grey.	64
5.9	The equal-weight portfolio against its seven components, all rebased to 100, on a logarithmic scale. The portfolio is smoother than any component because the components do not fall together.	66
5.10	Correlation between the daily returns of the seven models over the full window.	67

List of Tables

2.1	Summary of Financial Markets and their Economic Role [4].	6
2.2	Summary of the most common Activation Functions [5].	14
2.3	Summary of the most common Loss Functions [5].	14
2.4	Summary of the most common Gradient Descent variants [5].	15
3.1	How this work compares with the eighteen surveyed articles.	28
4.1	Tick data of the seven pairs from January 2014 to December 2025, as downloaded from the broker's servers. The year 2014 only warms up the indicators. Bars of any timeframe are built from these ticks when a run needs them.	33
4.2	Cost and account configuration applied in training and in every evaluation reported in this work.	34
4.3	The sixteen technical indicators that form the market context of the observation. Periods are in hourly bars. Each moving average gives two features, its distance from the price and its own differential, both in units of the average true range. Each momentum reading gives one feature, scaled by the movement expected over its lookback.	37
4.4	Hyperparameters compared with the original Deep Deterministic Policy Gradient [6]. Three values were changed and seven mechanisms were added. The rest are the published values.	40
4.5	The eleven gates a seed must pass to be admissible. The first six test money, the last five test behaviour. The behaviour gates are measured on the full window with the delivered model, except Alive, which looks at the last fold.	44
4.6	Parts of the composite selection score of Equation (4.11). The three groups have equal weight, so no single metric dominates. The test-year return is one of four money metrics and makes up one twelfth of the score. The ratios are those defined in Section 2.1.5. . . .	45
4.7	Effect of the two deployment decisions, measured on the delivered GBP/USD model over the full window with nothing retrained. All rows use the same weights with different execution settings, so the differences are attributable to the deployment layer alone. The spread is charged in every row. The model was trained with daily decisions and the threshold, so the hourly rows use it outside the setting it learned in.	46

5.1	Summary of the performance metrics used in this work.	51
5.2	How many trained seeds pass each gate. Validation, Test and Full each need a positive return and a return above the pair's own over that window; the other five are the behaviour gates of Section 4.3. All counts the seeds that pass every gate. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	52
5.3	Walk-forward validation. Each year is scored by the model of its own fold, trained on every year before it from 2015 and never on the year itself; Stitched chains the seven years and Pair chains the pair's own moves over them. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	53
5.4	The test year against the validation year and the full window. 2024 chose which episode's weights the delivered model kept; 2025 was seen by no training and chose between seeds only as a confirmation gate. The last two columns are the move of the pair itself and of the US 500 index in 2025. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	54
5.5	Result for each pair over 2015 to 2025 under the full costs, from the delivered model of the last fold, which trained on 2015 to 2023. Return columns are the mean, standard deviation and range across the five starting balances; ratios and drawdown are from the reference run at 10,000 US dollars. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	55
5.6	Each model against buying and holding its own pair and against the US 500 index from 2015 to 2025, under the same costs. The model is the delivered model of the last fold, which trained on 2015 to 2023. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	55
5.7	Trading behaviour over the evaluation window. Mean $ a $ is the average fraction of the reference position held while in the market. Trend is the lookback whose strength the signal tracks best; Side ρ is the rank correlation of the signal with that trend and Size ρ that of its magnitude with the trend's strength. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	58
5.8	What each model earned before costs and what it kept, in US dollars at 10,000. Cost share is the spread and commission together as a fraction of profit before costs. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	61

5.9	Yearly regression of each model's return on its pair's over 2015 to 2025. Alpha is the return attributable to the policy and the beta return the part the pair's own movement explains; the two sum to the mean yearly return. The reading is a rule: an absolute beta below 0.5 is a policy with no market exposure. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	63
5.10	Total return from each of the five starting balances, in US dollars. Discrete volumes round each balance's positions differently, so each is a different sequence of trades rather than a rescaled copy. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	64
5.11	The same weights at three risk levels. The exposure feature is normalised by the sizer, so the policy acts identically and only the position size scales; drawdown scales with it while the ratios stay close. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	65
5.12	The equal-weight portfolio of the models, each from 10,000, against the average of its components. Return is identical by construction, so the effect is on risk. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.	66
A.1	Summary of frequently used Trend Technical Indicators [7].	79
A.2	Summary of frequently used Momentum Technical Indicators [7].	80
A.3	Summary of frequently used Volume Technical Indicators [7].	80
A.4	Summary of frequently used Volatility Technical Indicators [7].	81
B.1	Basic features of the related work articles.	84
B.2	Advanced features of the related work articles.	85

List of Algorithms

2.1	Gradient of the Loss Function [5].	15
2.2	Actor-Critic [3].	21
2.3	Deep Deterministic Policy Gradient [6].	23

Acronyms

A2C	Advantage Actor-Critic
A3C	Asynchronous Advantage Actor-Critic
ATR	Average True Range
AUD	Australian Dollar
CAD	Canadian Dollar
CHF	Swiss Franc
DDPG	Deep Deterministic Policy Gradient
DDQN	Double Deep Q-Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
DRQN	Deep Recurrent Q-Network
EMA	Exponential Moving Average
ER	Efficiency Ratio
ETF	Exchange-Traded Fund
EUR	Euro
FNN	Feedforward Neural Network
GBP	British Pound
GDPG	Gated Deterministic Policy Gradient
GDQN	Gated Deep Q-Network
GRU	Gated Recurrent Unit
JPY	Japanese Yen

LSTM	Long Short-Term Memory
MACD	Moving Average Convergence Divergence
MDP	Markov Decision Process
MSE	Mean Squared Error
NN	Neural Network
NZD	New Zealand Dollar
OHLCV	Open, High, Low, Close and Volume
PPO	Proximal Policy Optimisation
ReLU	Rectified Linear Unit
RL	Reinforcement Learning
RNN	Recurrent Neural Network
ROC	Rate of Change
RSI	Relative Strength Index
RV	Realised Volatility
SARSA	State-Action-Reward-State-Action
SD	Standard Deviation
SMA	Simple Moving Average
TD	Temporal Difference
TD3	Twin Delayed Deep Deterministic Policy Gradient
USD	US Dollar
WMA	Weighted Moving Average

Glossary

alpha

The part of a strategy's mean return that its exposure to the market does not explain, taken from the regression of the strategy's returns on the market's returns

ask price

The price at which the market sells, paid to open a long position or to close a short one

backtest

A simulation of a trading strategy on historical prices

balance

The account value from closed positions only, before the result of any open position

base currency

The first currency of a pair, the one being bought or sold; the pair's price is the amount of quote currency one unit of it costs

beta

The sensitivity of a strategy's return to the market's return, the slope of the regression of one on the other

bid price

The price at which the market buys, received when closing a long position or opening a short one

Calmar ratio

The annualised return divided by the maximum drawdown

commission

The fee a broker charges for the volume traded, on opening and on closing a position

currency pair

The price of one currency expressed in another, such as EUR/USD

drawdown

The fall of equity from its previous peak, as a fraction of that peak

equity

The balance plus the unrealised result of the open positions

hedging account

An account that can hold several positions on one instrument, in either direction, at the same time

leverage

The ratio between the size of a position and the account capital that backs it

long position

A position that gains when the price rises

lot

A standard trade size of 100,000 units of the base currency

netting account

An account that holds at most one position per instrument, so an opposite order reduces, closes or reverses it

pip

The fourth decimal place of most currency pair prices, and the second for pairs quoted in Japanese yen

quote currency

The second currency of a pair, in which the price is expressed

Sharpe ratio

The mean excess return divided by the standard deviation of returns

short position

A position that gains when the price falls

Sortino ratio

The mean excess return divided by the standard deviation of the negative returns only

spread

The difference between the ask and the bid prices, paid on every round trip

swap

The interest charged or paid for holding a position past the daily rollover time

tick

A single change in the quoted ask or bid price

walk-forward analysis

An evaluation in which a model is trained on one period and scored on the following one, repeated as the periods move forward in time

1

Introduction

Contents

1.1 Motivation	2
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Contributions	3
1.5 Structure	4

The foreign exchange market is the largest financial market in the world [8]. It trades continuously from Sunday evening to Friday evening, and its price history is available down to single quotes. This makes it a good target for automated trading. However, returns over short horizons are close to unpredictable [9], and trading costs are high. Many strategies that look profitable in a simplified test lose money once these costs are charged.

This work applies Deep Reinforcement Learning (DRL) to this problem. One Deep Deterministic Policy Gradient (DDPG) [6] agent is trained for each of the seven major currency pairs. Each agent is evaluated over eleven years of tick data, with the spread quoted at the time of the trade and the commission of a retail account.

1.1 Motivation

Automated currency trading is usually built from fixed rules. Technical indicators are computed from recent prices, a signal is taken from them, and a separate layer decides the position size [4, 10]. These systems are cheap to run and easy to check. Their parameters are set once, so they do not adapt when the market changes. This is the first difficulty that this work deals with.

Currencies were chosen over equities for practical reasons [11]. A trader can buy or sell with no borrowing cost and no short-selling restrictions, so a policy that sells is not penalised by market mechanics. There are only seven major pairs, so a design can be tested on all of them.

Reinforcement learning describes the task without splitting it into a prediction and an action. The agent observes the state of the market, chooses a position and receives the profit or loss of that position as its reward [3]. This is also how a trading system is judged in practice.

DDPG is an actor-critic method for continuous action spaces [6]. The output of its actor can be used directly as a signed position: the sign gives the direction and the magnitude gives the exposure. Much of the published work on trading uses value-based methods such as the Deep Q-Network [12–14]. These methods choose from a small discrete set of actions, normally buy, hold and sell. They leave position sizing to rules that were never trained on the reward.

The second difficulty is cost. Results in this field are often reported on bar data, with the spread fixed or ignored and without the broker commission [13, 15]. A strategy that changes its position often pays the cost on every change, so it can make money without costs and lose money with them. A test only shows this if it charges what the account would pay [16]. In this work, the commission removed 121 percentage points of total return from a model that adjusted its position at every small move of its target, and a threshold that skips small changes more than recovered them (Section 4.3).

1.2 Problem Statement

Most published work on deep reinforcement learning for trading uses equity markets, daily bars and a single instrument over a few years [13, 14, 17]. The work that does use currencies normally uses a discrete action space [18–20]. Costs are reported differently across studies: some charge both explicit and implicit costs, and others leave the spread fixed or out [15]. Two questions are therefore open. The first is whether one design works on several instruments instead of being fitted to one. The second is whether its performance still holds under realistic costs.

The problem of this work is to find out whether a single deep reinforcement learning design produces a profitable and risk-controlled policy on all seven major currency pairs. The design is trained separately for each pair, is otherwise unchanged, and is evaluated with the costs of a swap-free raw-spread retail account.

A model adjusted until it does well on one pair may only describe the past prices of that pair, and its results alone cannot tell this apart from a strategy [21]. Such a result will not hold on all seven pairs, so the same design applied to every pair shows the difference.

Answering this question also needs an execution environment close to a real broker. None was available, and building one took a large part of the effort behind this work (Chapter 4).

1.3 Objectives

The work was organised around five objectives.

- Build a trading framework that backtests a strategy on tick data with the costs of the broker and runs the same strategy inside cTrader, so the system that is tested is the system that is deployed.
- Implement DDPG in this framework, with an observation built from past prices and technical indicators and a continuous action read as a signed position size.
- Train and evaluate one agent for each of the seven major currency pairs, over the same period and with the same cost model.
- Find out whether the performance comes from the policy or from exposure to the movement of the underlying currency pair.
- Combine the seven agents into a single equal-weight portfolio and measure how it behaves.

The first objective supports the other four. A result from a simplified simulator cannot be trusted or deployed, so the framework was built first.

1.4 Contributions

The contributions of this work are the following.

- **A trading framework where one strategy implementation runs in three places.** A Python library holds the strategy, the models and the market abstractions. A C# component inside cTrader [22] connects it to the library through shared memory. The same strategy code runs without change in an offline backtest, in the backtesting tab of cTrader and on a live account.
- **A market database built from quote-level data.** It holds about 1.6 billion ticks for the seven pairs from 2014 to 2025, downloaded from the broker's servers and kept current.

- **A backtesting engine that charges what the account charges.** The engine moves forward tick by tick, so stop and target levels are checked against the recorded price path inside the bar. The spread is read from the recorded quote at each trade, and the commission is that of a swap-free raw-spread retail account. The same costs are charged in training and in evaluation. To our knowledge, no published work on deep reinforcement learning for currency trading combines quote-level data, a measured spread and a real commission schedule (Table B.1 and Table B.2).
- **DDPG applied to the seven major currency pairs under one protocol.** Every pair uses the same architecture, observation design, reward and cost model, so the pairs can be compared. Each is trained on an anchored walk-forward with seven folds that validate on 2018 to 2024, and 2025 is held out as the test year. One model per pair is chosen among the seeds that pass a fixed set of gates.
- **An evaluation of every pair on risk-adjusted measures.** Each chosen model is evaluated from 2015 to 2025 by its return, Sharpe ratio [23], Sortino ratio [24], Calmar ratio [25] and maximum drawdown.
- **An attribution of the returns to the policy and to market exposure.** Each model's yearly return is regressed on the yearly movement of its own pair, which separates the return of the policy from the exposure to the market. A model that only follows the market is reported as such, with the mechanism that causes it.

1.5 Structure

Chapter 2 explains the foreign exchange market, technical analysis and reinforcement learning, ending with DDPG. Chapter 3 reviews the published work that applies these methods to trading and the questions it leaves open. Chapter 4 describes the framework, the backtesting engine and its cost model, and the design of the observation, the action and the reward. Chapter 5 reports the evaluation protocol, the results for each pair, their attribution and the seven agents run together as a portfolio. Chapter 6 summarises what was achieved, the limitations of the study and the work that follows from it.

This work uses two fields that developed separately. Trading gives the market, the costs and the way performance is measured. Reinforcement learning gives the learning algorithm and the way the problem is set up. The next chapter covers both, because the system cannot be described without them.

2

Background

Contents

2.1 Financial Markets and Trading	5
2.2 Neural Networks	12
2.3 Reinforcement Learning	17
2.4 Deep Reinforcement Learning	21

This chapter covers the background needed to follow the rest of this thesis. It starts with financial markets, and in particular with the Forex market, how trading works there and what it costs. It then covers neural networks and reinforcement learning, the two fields of machine learning this work uses. It ends with Deep Reinforcement Learning, which joins the two, and with the algorithm used in this work.

2.1 Financial Markets and Trading

Financial markets move capital between those who have it and those who need it. In these markets, participants trade assets, hedge risk and raise funds. Each type of market has a different purpose, and Table 2.1 summarises the main ones. The markets are connected. A movement in one often passes

on to the others, so none of them is studied on its own. The market participants themselves set the prices. They range from private investors to banks and investment funds. Regulators set the rules that these participants must follow. Electronic platforms and algorithmic execution have opened the markets to more participants and have made prices adjust faster [4].

Table 2.1: Summary of Financial Markets and their Economic Role [4].

Market	Economic Role	Asset Examples
Equity Market	Raising capital by trading company shares.	Stocks, Equities
Bond Market	Debt financing, where entities borrow from investors.	Government and Corporate Bonds
Commodities Market	Trading basic raw materials and agricultural goods.	Oil, Gold, Wheat
Forex Market	Currency exchange for international trade.	USD, EUR, JPY, GBP
Derivatives Market	Trading contracts for hedging or speculation.	Futures, Options, Swaps
Indices Market	Combining and tracking the performance of a market or sector.	S&P 500, NASDAQ
Cryptocurrency Market	Exchanging digital currencies.	Bitcoin, Ethereum

2.1.1 Forex Market: Characteristics and Significance

The Foreign Exchange market, or Forex market, is where currencies are exchanged. It is the largest financial market by turnover. In April 2025 it traded 9.6 trillion US dollars per day on average, 28% more than the 7.5 trillion recorded three years earlier. It trades continuously across time zones, because the banks and institutions that quote its prices are spread around the world. Spot transactions, the instrument traded in this work, make up 3 trillion of that daily total. The US Dollar is on one side of 89.2% of all trades, and the ten most traded currency pairs all include it [8]. Prices react to interest rate decisions, economic releases and political events. Participants normally trade with leverage. Leverage allows a position much larger than the capital deposited, and it scales profits and losses in the same proportion. Its market depth, its continuous operation, its sensitivity to news and the routine use of leverage make it suitable for automated trading. The same properties make it hard to model. Its behaviour is noisy and non-stationary, which means that its statistical properties change over time. This is why adaptive, data-driven strategies are of interest [11].

2.1.2 Forex Trading: Fundamental Concepts and Operation

Trading a currency pair means buying one currency and selling another at the same time. Buying the EUR/USD pair means buying Euros and selling US Dollars, expecting that the Euro will strengthen against the US Dollar. This work uses the seven major currency pairs: EUR/USD, USD/JPY, GBP/USD, AUD/USD, USD/CHF, NZD/USD and USD/CAD. They are built from the Euro (EUR), US Dollar (USD), Japanese Yen (JPY), British Pound (GBP), Australian Dollar (AUD), Swiss Franc (CHF) and New Zealand

Dollar (NZD). Every one of them has the US Dollar on one side. Deciding whether to buy or sell a pair comes down to predicting the strength of one currency relative to the other [11].

A position is the exposure a trader holds in a pair. It is the amount of the pair that the trader has bought or sold and not yet closed. A long position makes money when the pair rises and loses money when it falls. A short position does the opposite. A currency pair is a ratio between two currencies, so short selling is as natural as buying and needs no borrowing. In equity markets this is not the case. Positions are opened and closed with orders. A market order is executed at once, at the price quoted at that moment. A limit order is executed only at a given price or better. A stop order is executed once the price passes a given level. Two stop orders are often attached to an open position. A stop loss closes the position when the loss reaches a chosen level. A take profit closes it when a target is reached.

Forex trading takes place in a decentralised market that works worldwide. There is no single exchange where all trades happen. Instead, trades are handled by a network of market makers and brokers. The market makers are mainly banks, and the market between them is known as the interbank market. Interbank trades are large, and most of them are made by institutions such as multinational companies and large hedge funds. Retail traders take part through brokers, who have access to that network and act as intermediaries. Market makers and brokers always quote two prices. The bid price is the price at which they will buy the pair. The ask price is the price at which they will sell it. The difference between the two is the spread. Without a central trading venue, one might expect the market to be poorly regulated. However, competition between market makers keeps their quoted prices close to one another and keeps spreads narrow [11].

Traders look at prices on charts, and the candlestick chart is the standard type. A candlestick chart is made of candlesticks. Each one shows how the price moved over a period of time, which can be anything from minutes to years. A tick is a single update of the quoted price. It is published whenever a market maker changes its bid or its ask. The name comes from its shape, which looks like a candle with a wick at both ends. Each candlestick shows the opening price (O), the highest price (H), the lowest price (L) and the closing price (C) of its period (see Figure 2.1). The colour of the candlestick shows the direction of the move. A lighter shade means the price went up, and a darker shade means it went down. This lets traders see the direction of the market quickly, just by looking at the chart. Candlestick charts do not show volume (V), which is normally drawn separately below the price. Forex has no central venue, so there is no consolidated record of traded size. Brokers report tick volume instead, which is the number of quote updates in the period. This work uses tick volume throughout [26].

This work uses tick data as its main record, because it has the finest resolution the broker provides. Whenever a coarser view is needed, it computes from the ticks the opening, highest, lowest and closing prices and the tick volume, called OHLCV data for short [26]. Indicators computed from that data give the model more context, as described below.

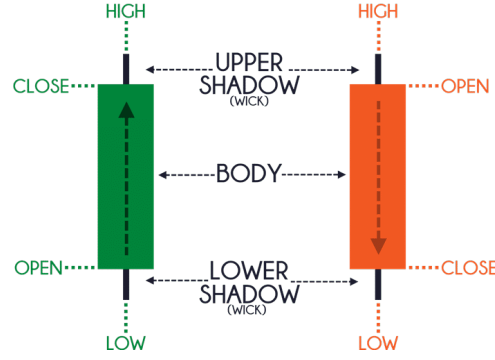


Figure 2.1: Candlesticks showing upward (bullish) and downward (bearish) movements respectively [1].

2.1.3 Forex Costs: Spreads, Commissions and Overnight Financing

Trading is not free, and its costs are large compared with the profit a single position can make. Three costs apply in Forex, and each is charged separately from the others. Prices are quoted in units that depend on the pair. A pip is the fourth decimal place of the quote for most pairs, or the second for pairs quoted against the Japanese Yen. A point is one tenth of a pip. It is the smallest price increment a broker quotes. The size of a position is given in units of the base currency, the first currency of the pair, and one standard lot is 100,000 units. For a position of volume q at price p , the notional value, which is the value of the position in the quote currency (the second currency of the pair), is

$$V = q \cdot p \quad (2.1)$$

The **spread** is the first cost. It is an implicit cost, paid through the price. A position is opened at the ask and closed at the bid, so a full round trip, opening and then closing, crosses the spread once. For a position of volume q , the cost is

$$C_{spread} = (p_{ask} - p_{bid}) \cdot q \quad (2.2)$$

The spread is not constant. It widens when liquidity falls, around economic releases and at the daily rollover. A strategy tested with a fixed spread is therefore tested on a market that does not exist in practice.

The **commission** is the second cost, and it is an explicit cost. Some accounts quote the raw inter-bank spread. On these accounts the broker makes its money through a commission charged on the traded volume, instead of through a wider spread. If the commission is given as a number of points c , and s_{point} is the size of one point, the cost of a position of volume q is

$$C_{commission} = c \cdot s_{point} \cdot q \quad (2.3)$$

The spread and the commission are separate costs, and a trader pays both. On a raw-spread account, treating the spread as if it already included the commission makes trading look cheaper than it is.

The **swap**, or overnight financing, is the third cost. It comes from the interest rate differential between the two currencies of the pair. It is charged when a position is still open at the daily rollover, the moment when the trading day ends. On one day of the week the charge is multiplied, to cover the weekend. Depending on the direction of the position and the sign of the differential, the swap can be a cost or a credit. Some account types remove it completely, so that clients can avoid interest for religious reasons. These are known as swap-free accounts. Finally, leverage decides how much capital a position needs. With leverage L , the margin, which is the capital set aside for a position of notional value V , is

$$M = \frac{V}{L} \quad (2.4)$$

Leverage does not change the profit or loss of a position. It only changes the capital set aside for it. So leverage affects the return of the account, and the return of the trade stays the same. These costs are not the same on every type of account. Choosing the account is therefore part of designing a strategy. A raw-spread account quotes the interbank spread with no markup and makes its money through the commission. This makes the total cost of a position lower. It also makes the cost explicit, which is more useful. The implicit part of the cost is reduced to the market spread, and the rest is a published rate. A swap-free account charges no overnight financing at all. An account with both features suits a strategy that holds positions for days. Such a strategy pays the spread and the commission only a few times per position, but it would otherwise pay financing for every night the position stays open. This work is evaluated on an account that has both features. The spread and the commission are therefore the only costs applied, and the financing cost is exactly zero, so it does not need to be estimated.

2.1.4 Financial Analysis: Main Types and Formulas

Representing the data is one half of the problem. The other half is analysing it to decide what to trade. Financial analysis is usually divided into three types [27]:

- **Fundamental Analysis:** studies the economic factors that affect currency prices. These include GDP growth, employment rates, interest rates and monetary policy, which help to predict currency trends from the economic health of a country. However, much of this data is qualitative. This makes it hard to use in computer models, which work best with structured numerical input.
- **Sentiment Analysis:** tries to measure the mood of the market by analysing news, market commentary and similar qualitative data. It aims to measure how the shared emotions of traders affect

market trends. It is complex and needs advanced natural language processing, so it is harder to include in a model.

- **Technical Analysis:** analyses past OHLCV data to predict future market movements. It assumes that past market behaviour will repeat itself, so patterns or measures found in current data can predict future movements. It is preferred because its results are numbers, which complex machine learning models can use.

Of the three, this work uses technical analysis. It produces numerical series directly from price data, so a model can use them without any further interpretation. Technical analysis is divided into indicators and candlestick patterns. Indicators are measures calculated from past market data, created mainly by mathematicians and economists. They give a numerical view of how the market moves. They are usually grouped into four main types [27]:

- **Trend Indicators:** Used to find the main direction of the market by filtering out market noise. See Table A.1 for details of frequently used indicators.
- **Momentum Indicators:** Used to measure the momentum of a trend, whether it is strong or weak, and whether a reversal is expected. See Table A.2 for details of frequently used indicators.
- **Volume Indicators:** Used to look at the trading volume, which gives clues about the strength of a price movement. See Table A.3 for details of frequently used indicators.
- **Volatility Indicators:** Used to measure the level of uncertainty or risk that comes with the volatility of the price. See Table A.4 for details of frequently used indicators.

The four tables list indicators in common use. They do not describe the contents of any one system. The framework built for this work implements the moving average family and the crossovers between its members, the Rate of Change, the Moving Average Convergence Divergence, the Average True Range, the Realised Volatility and the Efficiency Ratio. It implements no volume indicators. The tick volume a Forex broker reports counts quote updates, not traded size, and this limits what those indicators can measure. Ten instances of the implemented set form the observation the agent receives. They are listed in Chapter 4. Candlestick patterns work differently. They do not compute a value. They describe shapes, formed by one or more candlesticks, that traders link to a change in behaviour [26]. Such patterns can be encoded as numbers and given to a model in the same way as an indicator [7]. This work does not use them. The observation is built from indicators only.

2.1.5 Performance Measurement: Returns, Risk and Ratios

A trading strategy is judged by what it earns and by the risk it takes to earn it. A single return figure is therefore not enough to compare two strategies. The measures below are the ones reported in

Chapter 5. They are defined on an equity series, which is the value of the account over time. They therefore apply without change to a portfolio of several strategies. They are then computed on the combined equity of the portfolio, and not on each strategy separately. This matters because the risk of a portfolio depends on how its parts move together, and not only on the risk of each part.

The total return over a period compares the final and the initial equity E of the account:

$$R = \frac{E_{final} - E_{initial}}{E_{initial}} \quad (2.5)$$

Periods differ in length, so returns are annualised, that is, converted to a yearly rate, before they are compared. With a duration Δt in seconds and a year of 365 days,

$$R_{annual} = (1 + R)^{\frac{365 \cdot 86400}{\Delta t}} - 1 \quad (2.6)$$

Risk is measured in two ways. The first is volatility, which is the standard deviation of returns, annualised by the rate at which the strategy trades. The second is drawdown. At any moment, the drawdown is the distance between the current equity and the highest equity reached so far:

$$D(t) = \frac{E(t) - \max_{s \leq t} E(s)}{\max_{s \leq t} E(s)} \quad (2.7)$$

The maximum drawdown is the worst value of $D(t)$ over the period. It is the largest loss from a peak that an investor would have suffered. The mean drawdown is the average of $D(t)$. It shows how far below its peak the account is on a typical day, instead of at its worst moment. Four ratios combine return and risk. All of them divide an annualised excess return, which is the return above the risk-free rate, by a measure of risk. They differ only in the measure of risk they use. The absolute value of that measure is used, because a drawdown is negative under the definition above. With R_f as the risk-free rate,

$$\text{Ratio} = \frac{R_{annual} - R_f}{|\text{risk}|} \quad (2.8)$$

The **Sharpe ratio** [23] uses the annualised volatility, so it penalises upward and downward moves equally. The **Sortino ratio** [24] uses the downside volatility instead, computed from negative returns only. The argument is that an investor does not mind upside variability. The **Calmar ratio** uses the maximum drawdown, so it measures return against the worst loss suffered. The **Sterling ratio** uses the mean drawdown. This makes it less sensitive to a single bad period, and closer to what holding the strategy is like on a typical day. The Calmar and Sterling ratios both have several definitions in the literature, and the ones used here follow [25]. By construction, the mean drawdown is smaller than the maximum drawdown. So, for a strategy with a positive excess return, the Sterling ratio is the larger of the two ratios. When the excess return is negative, the order is reversed.

2.1.6 Algorithmic and Quantitative Trading

More and more of the trading in financial markets is done by algorithms. Algorithmic trading uses computer programs to execute trades faster and in larger volume than a human can. The programs analyse large datasets, read market conditions and act on set rules. Algorithmic trading is used for pre-trade analysis, signal generation and trade execution. It is what allows institutions to run complex strategies at a large scale. Quantitative trading is a subfield of algorithmic trading. It uses mathematical and statistical models to find opportunities in market data, through the analysis of past data and through machine learning. It follows systematic rules instead of the discretionary judgement of a trader, and it needs knowledge of both finance and mathematics [10]. Together, the two have changed how markets work. They brought practices such as high-frequency trading, and they shortened the time that prices take to react to new information. They have also made machine learning central to the field. Neural networks are used to find structure in market data, and reinforcement learning is used to decide what to do with it [7]. These two fields are the subject of the next sections.

2.2 Neural Networks

Machine learning covers algorithms that find patterns in data and act on them, without being programmed with the rules in advance. Tom Mitchell states the idea as follows [28]:

"A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E ."

The definition is useful here because it describes exactly what a trading agent does. The task is choosing a position. The performance measure is the money that position makes. The experience is the market data the agent has already seen. Neural networks, the subject of this section, are the class of models used to represent that behaviour.

2.2.1 Fundamentals and History

Neural Networks (NN) are algorithms designed for pattern recognition. They are named after the human brain, because their structure is similar to it. In the brain, a neuron receives signals through structures called dendrites and processes them in the nucleus. It then passes the result down its axon to the dendrites of nearby neurons. A neural network works in a similar way. Each node is an artificial neuron. It receives an input, processes it with a mathematical function and passes the output to the next layer of nodes. In both cases, the strength and the type of the connections between neurons decide how the network behaves [5]. Figure 2.2 illustrates these ideas.

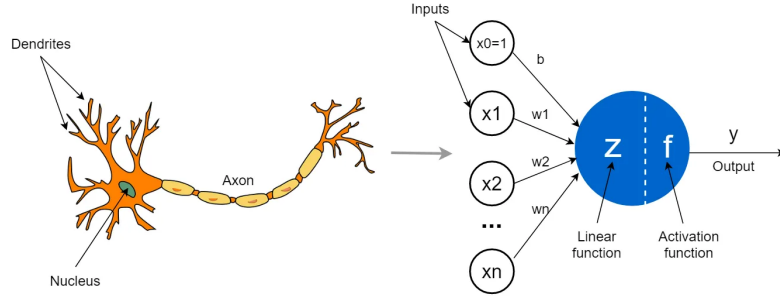


Figure 2.2: Human Neuron compared to an Artificial Neuron [2].

The idea of neural networks goes back to the 1950s and the perceptron, a simple model of a biological neuron developed by Frank Rosenblatt [29]. The perceptron was the start of NN research. However, it could only learn linearly separable datasets, in which a straight line or a flat plane can separate the classes. This limit led to some periods of low interest in the field, known as “AI winters”. A major advance came in the 1980s with the backpropagation algorithm. It allowed complex multilayer neural networks to learn patterns in datasets that are not linearly separable [30].

2.2.2 Key Concepts and Feed-Forward

In a neural network, the feed-forward computation in each layer l uses the output of the layer before it. For the first layer, this input x is the raw data itself. For the later layers, it is the output of the previous layer, $h^{(l-1)}(x)$. The computation in each layer can be written as [5]:

$$z^{(l)}(x) = W^{(l)}h^{(l-1)}(x) + b^{(l)} \quad (2.9)$$

Here, the weight matrix $W^{(l)} \in \mathbb{R}^{K_l \times K_{l-1}}$ and the bias vector $b^{(l)} \in \mathbb{R}^{K_l}$ are used in the computation of layer l . K_l is the number of neurons, or units, in layer l , and K_{l-1} is the number of neurons in the previous layer $l-1$. For the first layer, $W^{(1)} \in \mathbb{R}^{K_1 \times D}$, where D is the size of the input data. The activation function g is then applied to the pre-activation $z^{(l)}(x) \in \mathbb{R}^{K_l}$ to give the activated output $h^{(l)}(x) \in \mathbb{R}^{K_l}$ [5]:

$$h^{(l)}(x) = g(z^{(l)}(x)) \quad (2.10)$$

The choice of activation function matters in two places. In the hidden layers, the function must be non-linear. A stack of linear layers is equal to a single linear transformation, so the network would gain nothing from its depth. In the output layer, the task decides the function. A bounded function is used when the output must stay within a range, and no activation is used when it does not [5]. Table 2.2 summarises the most common activation functions.

Table 2.2: Summary of the most common Activation Functions [5].

Activation Function	Formula	Description
Linear	$a(x) = x$	Suitable for regression tasks.
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	Maps inputs to (0, 1), for binary classification.
Hyperbolic Tangent	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	Maps inputs to (-1, 1), with stronger gradients than sigmoid.
ReLU	$\text{ReLU}(x) = \max(0, x)$	Leads to sparse activation, helps gradient flow.
Softmax	$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}}$	Turns raw scores (logits) into probabilities, for multi-class classification.

2.2.3 Learning using Backpropagation

Neural networks learn by adjusting their weights and biases step by step, to reduce the error between the predicted outputs and the true target values. A loss function measures this error and guides the learning. The goal is to find the parameters that minimise the loss function. Different problems need different loss functions, such as mean squared error for regression tasks and cross-entropy for classification [5]. Table 2.3 gives their formulas.

Table 2.3: Summary of the most common Loss Functions [5].

Loss Function	Formula	Application
Mean Squared Error (MSE)	$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Regression
Negative Log-Likelihood (Cross-Entropy)	$-\sum_{i=1}^n y_i \log(\hat{y}_i)$	Classification

The main algorithm used to train neural networks is called backpropagation [30]. It computes the gradient of the loss function with respect to the weights and biases of the network. The gradient points in the direction of steepest increase of the loss. Algorithm 2.1 shows how these gradients are calculated. Gradient descent then moves the weights and biases in the opposite direction, which reduces the loss. Its variants, listed with their formulas in Table 2.4, suit different data and computing needs. Batch gradient descent uses the whole dataset for each update. It is stable but expensive on large datasets. Stochastic gradient descent updates the parameters after every data point. It is faster but noisier. Mini-batch gradient descent updates on subsets of the data. It is the compromise used in practice [5].

In Table 2.4, the learning rate η is a hyperparameter that controls the size of each update. A rate that is too large overshoots the minimum. A rate that is too small converges slowly and can get stuck in a local minimum. Some methods adapt η during training. They change the step size as the optimisation goes on, and this deals with both problems. Normalising or standardising the input data has the same aim, because features on very different scales produce gradients on very different scales [5].

Algorithm 2.1: Gradient of the Loss Function [5].

Require: f : network function, θ : network parameters, x : input data, y : true label, l : number of layers, L : loss function

Compute the output of the network $\hat{y} = f(x; \theta)$

Compute the gradient of the loss function with respect to the output layer before activation:

$$\nabla_{z^{(l+1)}} L(f(x; \theta), y) = \frac{\partial L(f(x; \theta), y)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(l+1)}}$$

for $\ell = l + 1$ **to** 1 **do**

 Compute gradients of hidden layer parameters:

$$\nabla_{W^{(\ell)}} L(f(x; \theta), y) = \nabla_{z^{(\ell)}} L(f(x; \theta), y) \cdot h^{(\ell-1)T}$$

$$\nabla_{b^{(\ell)}} L(f(x; \theta), y) = \nabla_{z^{(\ell)}} L(f(x; \theta), y)$$

 Compute gradient of previous hidden layer:

$$\nabla_{h^{(\ell-1)}} L(f(x; \theta), y) = W^{(\ell)T} \cdot \nabla_{z^{(\ell)}} L(f(x; \theta), y)$$

 Compute gradient of previous hidden layer (before activation):

$$\nabla_{z^{(\ell-1)}} L(f(x; \theta), y) = \nabla_{h^{(\ell-1)}} L(f(x; \theta), y) \odot g'(z^{(\ell-1)})$$

Table 2.4: Summary of the most common Gradient Descent variants [5].

Algorithm	Formula	Description
Batch Gradient Descent	$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; X, y)$	Updates the parameters using all the data
Mini-Batch Gradient Descent	$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; X^{(i:i+n)}, y^{(i:i+n)})$	Updates the parameters using subsets of the data
Stochastic Gradient Descent	$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}, y^{(i)})$	Updates the parameters after each data point

2.2.4 Bias-Variance Trade-off and Model Generalisation

How well a neural network generalises, that is, how well it works on data it has not seen, depends on a balance between bias and variance. Bias leads to underfitting. The model makes assumptions that are too simple and misses the real trends in the data. Variance leads to overfitting. The model is too complex and treats noise as if it were a signal. Both give poor results on new data. This matters in Forex trading, because the signal being sought is weak and the noise around it is large. Cross-validation is often used to measure how well a model generalises, on separate training, validation and testing datasets. Regularisation methods such as L1 and L2 limit the complexity of a model by adding penalties to the loss function. Dimensionality reduction methods such as principal component analysis reduce the number of inputs the model has to fit. Together, these methods make it more likely that the model performs well on out-of-sample data, which is data not used in training [5]. This work divides the period into training, validation and testing windows, described in Chapter 4. The performance on the validation window decides which model is kept. Complexity is not controlled through a penalty added to the loss.

2.2.5 Architectures for Sequential Data

The network described so far is feed-forward. The whole input is given at once, and the output depends only on that input. Financial data is a sequence, so an obvious question is whether a network that reads

a sequence directly would do better.

Two families of networks do that. A recurrent neural network keeps an internal state that is carried from one step to the next, so each observation is read together with the ones before it. The long short-term memory network is the variant used in practice. It adds gates that control what the state keeps and what it drops. This stops the gradient from vanishing, that is, from shrinking towards zero, as it is propagated back over many steps [5]. Attention-based networks reach the same goal in a different way. They let the network weigh every position in a window directly, instead of passing information along a chain.

Much of the work reviewed in Chapter 3 uses recurrent networks, and several of those studies report that they helped. This work does not use them, and this was a deliberate design decision. Recurrence is one way to give a network access to the past. Another way is to compute measures of the past explicitly and give them to the network as part of the input. The observation described in Chapter 4 uses the second way. It includes momentum and moving-average readings over horizons from one day to one year. The history that a recurrent state would have to learn to summarise is therefore measured directly and given to the network.

This choice has three consequences. First, a feed-forward network on engineered features has far fewer parameters than a recurrent network on raw prices. This matters when the signal is weak and the sample is one instrument over eleven years. Second, it is faster to train. Training speed decided how many independent runs were possible, and this turned out to matter more for the quality of the final result than any gain from a different architecture would have. Third, it keeps the input interpretable. Each feature is a named quantity with a known meaning, so when a policy behaves strangely, the observation can be inspected instead of guessed at.

This choice also has a cost. A recurrent network can, in principle, find a summary of the past that no fixed set of indicators contains. This work gives up that possibility. Whether it would have helped here has not been tested, and it is listed among the extensions in Section 6.3.

2.2.6 Universal Approximation Theorem

The universal approximation theorem, proved by Hornik, Stinchcombe and White in 1989, can be stated as follows [31]:

A neural network with one hidden layer and a linear output can approximate any continuous function arbitrarily well, given enough hidden units.

The theorem shows that neural networks are universal function approximators, and this is what justifies their use here. It means that a large enough network can represent the function that maps market observations to a trading decision. It does not say whether such a function can be found from data, or

whether it exists in a form stable enough to be useful. Those questions are the subject of the reinforcement learning methods covered next.

2.3 Reinforcement Learning

Reinforcement Learning (RL) is the branch of machine learning that studies an agent that interacts with an environment and learns which decisions to make. Unlike supervised learning, it is not given the correct answer for any situation. The agent acts, observes what happens and changes its behaviour according to the reward it receives. Learning therefore happens by trial and error, without labelled examples. This suits problems where the right action is not known in advance but its outcome can be measured. Robotics and finance are two fields where this is the case.

2.3.1 Fundamentals and History

Reinforcement Learning grew out of the combination of several other fields, such as psychology, computer science and control theory. It was inspired by the way humans and animals learn from their environment. In RL, an agent makes decisions and receives rewards or penalties for its actions, with the aim of maximising good outcomes over time. This process is based on operant conditioning, a type of learning studied by psychologists such as B. F. Skinner [32], in which behaviour is shaped by its consequences. Markov Decision Processes (MDPs) were formalised in the late 1950s. They model decision making where the outcomes are partly random and partly under the control of the decision maker. MDPs became the mathematical basis of RL, because they allowed the development of algorithms that learn optimal strategies in complex environments. One of the major advances in RL was Q-Learning, introduced in the late 1980s. It is an algorithm that learns the value of actions in different states, so an agent can make informed decisions without a model of the environment. Q-Learning and Temporal Difference Learning made it possible to apply RL to a wide range of problems, from playing video games better than humans to optimising financial trading strategies [3].

2.3.2 Key Concepts and Agent-Environment Interaction

Reinforcement Learning is about how an agent interacts with its environment. The interaction involves states, actions and rewards. The state S_t describes the situation of the agent at that moment. When the agent observes S_t , it chooses an action A_t . This action makes the environment transition to a new state S_{t+1} , and the environment gives feedback in the form of a reward R_{t+1} . Figure 2.3 shows this repeating cycle [3].

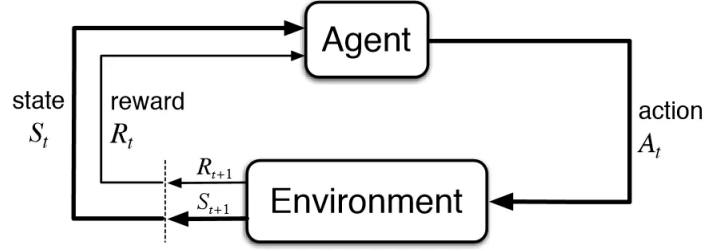


Figure 2.3: Interaction between the agent and the environment in Reinforcement Learning [3].

The goal of the agent is to maximise the return. The return is the discounted sum of the rewards the agent collects from time t onwards [3]:

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.11)$$

Here, γ is the discount factor. It sets how much future rewards count compared with immediate ones. A policy π decides the actions of the agent. It can be deterministic, giving one specific action for each state, or stochastic, giving a probability distribution over actions. A main difficulty in RL is the balance between exploration and exploitation. Exploration means trying new actions to find out their effects. Exploitation means choosing actions that are known to give a high reward. A common approach is the ϵ -greedy policy, which usually picks the best-known action but sometimes picks a random one [3]. This method needs a discrete set of actions, because it works by picking one of the alternatives at random. When the action is continuous, as in this work, exploration comes instead from adding noise to the action the policy returns. Section 2.4 describes this approach together with the algorithm that uses it.

Reinforcement Learning is built on the Markov Decision Process (MDP), defined by the tuple (S, A, P, R, γ) . Here, S is the set of all possible states of the environment and A is the set of actions the agent can take. P is the transition probability, the probability of moving from state s_t to state s_{t+1} after taking action a . R is the reward function, which gives the reward for that transition, and γ is the discount factor. MDPs rely on the Markov property. It requires the next state to depend only on the current state and action, and not on any earlier ones [3]:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1}|s_t, a_t) \quad (2.12)$$

The reward function matters because it is where the goal of the task is defined. It tells the agent what to achieve, but not how to achieve it. The agent will adopt any behaviour that raises the reward. In trading, the reward is normally computed from profit and loss. This drives the agent towards profitable positions without telling it which positions those are [3].

2.3.3 Value Functions and Q-Functions

The value function of a policy π , written $V^\pi(s)$, is the expected return when starting in state s and following policy π after that. It is defined as follows [3]:

$$V^\pi(s) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, \pi \right] \quad (2.13)$$

Here, γ is the discount factor, which sets how much future rewards count compared with immediate rewards. $\mathbb{E}$ is the expected value, the average of all possible outcomes weighted by their probabilities. The Q-function of a policy π , written $Q^\pi(s, a)$, is the expected return for taking action a in state s and following policy π after that. It is defined as follows [3]:

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a, \pi \right] \quad (2.14)$$

The Bellman equations are named after Richard Bellman, who first formulated them. They give a recursive decomposition of the total cumulative reward an agent can expect, starting from a given state or state-action pair. These equations are the basis of dynamic programming methods in RL. They express the value of a state, or of a state-action pair, in terms of the values of the states that follow it. This recursion is what allows the estimated values to be improved step by step, and it is the basis of convergence for most RL algorithms. The equations are as follows [3]:

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')] \quad (2.15)$$

$$Q^\pi(s, a) = \sum_{s' \in S} P(s'|s, a) \left[R(s, a, s') + \gamma \sum_{a' \in A} \pi(a'|s') Q^\pi(s', a') \right] \quad (2.16)$$

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) Q^\pi(s, a) \quad (2.17)$$

Equation (2.17) shows that the value of a state under policy π is the expected value of the Q-function over all possible actions, weighted by the probability the policy gives to each action. For optimal policies, the optimal value function $V^*(s)$ and the optimal Q-function $Q^*(s, a)$ give the highest return that can be achieved from any state or state-action pair, respectively. From the Bellman equation, it follows that [3]:

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')] \quad (2.18)$$

$$Q^*(s, a) = \sum_{s' \in S} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a' \in A} Q^*(s', a') \right] \quad (2.19)$$

$$V^*(s) = \max_{a \in A} Q^*(s, a) \quad (2.20)$$

By applying these equations again and again, RL algorithms gradually approximate the value function and the Q-function. This guides the agent towards the optimal policy, which is the policy that maximises the expected cumulative reward [3].

2.3.4 Methodological Variants and Learning Methods

Reinforcement learning methods can be divided into groups by the properties of the learning method: model-based or model-free, on-policy or off-policy, and value-based or policy-based. In model-based RL, the algorithm builds a model of the environment to simulate and predict the outcomes of actions. The agent can then plan by predicting the next states. This is possible when the environment is simple enough to be modelled. In model-free RL, the algorithm learns a policy or a value function directly from its interactions with the environment, without building a model of it. Model-free methods are often chosen when the environment is hard to model or when no accurate model is available. Within model-free RL, there are on-policy methods, which learn about the policy that is currently used to make decisions, and off-policy methods, which explore with a different policy while they try to learn an optimal one [3].

Another important distinction is between value-based methods, also called critic-only, and policy-based methods, also called actor-only. Value-based methods, such as Temporal Difference Learning (TD), Q-Learning and SARSA Learning, estimate the value of states or of state-action pairs. The policy is not learned directly. It follows from these values. Temporal Difference learning is the update rule that the other two are built on, and it works in both settings. SARSA is on-policy, because it evaluates the policy it is following. Q-Learning is off-policy, because it evaluates the greedy policy, the one that always takes the highest-valued action, whatever policy it actually follows. To take the highest-valued action, the method has to compare every action, so these methods are limited to discrete sets of actions. Policy-based methods, such as REINFORCE, optimise the policy directly. For this reason they extend to continuous actions, where the policy returns the action itself instead of a value for each possible action. Actor-Critic learning combines the two. It has a policy function (the actor) and a value function (the critic). The policy is improved directly, and the critic gives the value estimates that make those updates stable [3]. Algorithm 2.2 gives a detailed implementation.

For finite Markov Decision Processes, standard results guarantee that an optimal policy exists and that it can be taken to be deterministic and stationary. These guarantees do not extend to complex, non-stationary environments such as financial markets [3]. A raw price series does not have the Markov property. The information a trader uses is spread across the recent history, and is not all contained in the latest quote. RL applied to these markets therefore works with an approximation. Technical

Algorithm 2.2: Actor-Critic [3].

Require: $\pi(a | s; \theta)$: a differentiable policy, $Q(s, a; w)$: a differentiable Q -function, N : number of iterations and learning rates $\beta^\theta > 0$ and $\beta^w > 0$

Initialise policy parameter $\theta^{(0)}$ and Q -function parameter $w^{(0)}$

Initialise state s

for $n = 0, 1, \dots, N - 1$ **do**

 Sample $a \sim \pi(\cdot | s; \theta^{(n)})$

 Take action a , observe state s' and reward r

 Sample $a' \sim \pi(\cdot | s'; \theta^{(n)})$

 Compute the TD error: $\delta \leftarrow r + \gamma Q(s', a'; w^{(n)}) - Q(s, a; w^{(n)})$

 Update w : $w^{(n+1)} \leftarrow w^{(n)} + \beta^w \delta \nabla_w Q(s, a; w^{(n)})$

 Update θ : $\theta^{(n+1)} \leftarrow \theta^{(n)} + \beta^\theta \delta \nabla_\theta \ln \pi(a | s; \theta^{(n)})$

$s \leftarrow s'$

indicators and other engineered features put enough of that history into the state for the Markov property to hold approximately. Actor-Critic methods matter here because they are the point where reinforcement learning and neural networks meet. This is the subject of the next section.

2.4 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) combines the decision making of reinforcement learning with the function approximation of neural networks. The value functions and policies of Section 2.3 are defined over states. In a tabular setting, each state is stored separately, as one entry in a table. This is not possible when the state is a vector of continuous market variables, because there is no limit to the number of distinct states. Replacing the table with a neural network removes that limit. It also lets the agent generalise from states it has seen to states it has not seen.

2.4.1 Fundamentals and History

DRL appeared once it became clear that neural networks could serve as the function approximators that reinforcement learning needed to handle high-dimensional input. The result that started the field was the Deep Q-Network. It learned to play a range of Atari 2600 games directly from raw pixels and reached human-level scores on many of them [12, 33]. It showed that a single architecture could learn control policies from raw, high-dimensional input, without features designed for each task. This result led to interest in applying the same methods to financial markets. There, the input is also high-dimensional, and the mapping from observation to decision is also unknown. However, an Atari game is stationary and fully observable, and a market is neither. For this reason, results carry over to markets much less directly than the comparison with games suggests.

2.4.2 Deep Value-Based RL Algorithms: Deep Q-Networks

Deep value-based methods use a neural network to approximate the Q-function. This lets them work with state spaces too large to list one by one. Neural Fitted Q Iteration was an early method of this kind. It fitted a network to the Q-values of a stored set of transitions, and fitted it again as more transitions were collected [34]. The approach became established with the Deep Q-Network (DQN) algorithm [12], which added two mechanisms that made training stable enough to be practical. The first is a separate target network. This is a delayed copy of the value network, used to compute the learning targets, so that the target does not move as fast as the estimate. The second is experience replay. This is a buffer of past transitions from which mini-batches are drawn at random, which breaks the temporal correlation between consecutive samples. The algorithm used in this work uses both mechanisms.

After DQN, the Double Deep Q-Network (DDQN) dealt with the tendency of DQN to overestimate action values [35]. It separates action selection from action evaluation. The online network, the one being trained, chooses the action, and the target network scores it. Deep value-based methods are not used in this work. They select an action by comparing the values of all available actions, and this requires the set of actions to be discrete.

2.4.3 Deep Policy-Based RL Algorithms: Deep Policy Gradients

Deep policy-based algorithms optimise the policy directly, without deriving it from action values. This is what allows them to work with continuous actions. In trading this matters, because a position has a size as well as a direction. A method limited to a discrete set of actions has to decide that size outside the learned policy. The Deep Deterministic Policy Gradient (DDPG) algorithm is a model-free, off-policy, actor-critic method for continuous action spaces [6]. It keeps four networks. The actor $\mu(s|\theta^\mu)$ maps a state to a single action. The critic $Q(s, a|\theta^Q)$ estimates the value of a state-action pair. The other two are delayed copies of these, the target actor μ' and the target critic Q' . Transitions are stored in a replay buffer, and mini-batches of N transitions are sampled from it at random. The critic is trained in the same way as in DQN. It minimises the squared error between its estimate and a target built from the observed reward and the value of the next state [6]:

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}) | \theta^{Q'}) \quad (2.21)$$

$$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2 \quad (2.22)$$

The actor is updated with the deterministic policy gradient, which follows the direction in which the value estimated by the critic increases. Because the action is continuous, the gradient of the value with

respect to the action is defined, and the chain rule carries it back to the parameters of the actor [6]:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a | \theta^Q) \big|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) \big|_{s_i} \quad (2.23)$$

The target networks are not simply copied from the trained networks. After every update, they are moved a small fraction $\tau \ll 1$ towards them. This keeps the learning targets almost stationary, but it also slows down how fast they follow improvements [6]:

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta' \quad (2.24)$$

The policy is deterministic, so it does not explore by itself, and the exploration problem raised in Section 2.3 comes back here. DDPG solves it by adding a noise process $\mathcal{N}$ to the action during training, so the action actually executed is $\mu(s | \theta^\mu) + \mathcal{N}$. The original work uses an Ornstein-Uhlenbeck process. This process gives noise that is correlated in time, instead of independent at each step. The reason given is that a physical system with momentum explores better when a disturbance lasts for a while than when it changes sign at every step [6]. Algorithm 2.3 shows the complete procedure.

Algorithm 2.3: Deep Deterministic Policy Gradient [6].

Require: an actor $\mu(s | \theta^\mu)$, a critic $Q(s, a | \theta^Q)$, a discount factor γ , a soft update rate $\tau \ll 1$ and a minibatch size N

Randomly initialise the critic parameters θ^Q and the actor parameters θ^μ

Initialise the target parameters $\theta^{Q'} \leftarrow \theta^Q$ and $\theta^{\mu'} \leftarrow \theta^\mu$, and the replay buffer $\mathcal{B}$

for episode = 1, M **do**

Initialise a random process $\mathcal{N}$ for action exploration

Receive the initial observation state s_1

for $t = 1, T$ **do**

Select the action $a_t = \mu(s_t | \theta^\mu) + \mathcal{N}_t$ from the current policy and the exploration noise

Execute the action a_t and observe the reward r_t and the new state s_{t+1}

Store the transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{B}$

Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from $\mathcal{B}$

Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} | \theta^{\mu'}) | \theta^{Q'})$

Update the critic by minimising the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2$

Update the actor policy using the sampled policy gradient:

$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a | \theta^Q) \big|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) \big|_{s_i}$

Update the target networks:

$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$

$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$

DDPG is powerful, but it is not reliable. It has no guarantee of convergence except under restrictive conditions. It is also known to be sensitive to its hyperparameters and to the random seed, so two runs of the same configuration can end with different policies. A market makes this worse. It is non-stationary and only partially observable, which means the agent does not see the full state, and the guarantees

that apply to finite Markov Decision Processes do not hold. These limits belong to the method itself. They are not problems that can be fixed. Chapter 5 therefore reports how the results vary, instead of reporting a single run.

This work combines the four topics covered here. It needs a market to trade in, networks to approximate the functions involved, a reinforcement learning formulation of the trading decision, and an algorithm that solves it for continuous actions. Combining them is not new. Many published studies have already tried it.

Those studies differ in how they define the state, the action and the reward. They also differ in whether the agent pays any cost for trading. These choices decide whether a reported result would hold on a live account. This makes them the right basis for comparing the published studies and for placing this work among them.

3

Related Work

Contents

3.1 Survey Research: Identifying the State-of-the-Art	25
3.2 Article Research: Summarising Different Implementations	26
3.3 Research Gap: Positioning This Work	28

This chapter reviews the published work on deep reinforcement learning for financial trading. Surveys were read first, to find the open problems of the field and the papers worth reading in full. Then eighteen articles were read, and their design choices were recorded in one common format, so that studies on different markets and with different algorithms could be compared on the same points.

3.1 Survey Research: Identifying the State-of-the-Art

Hambly et al. [14] gave the theoretical basis. The survey gives the formulations and pseudocode of critic-only, actor-only and actor-critic algorithms and their uses in finance, and it was the main reference for Chapter 2.

Fischer [13] reviews implementations. This work took from it the dimensions used to compare studies: the dataset, the network under the agent, the state space, the action space, the reward function, the

methods used for the bias-variance and exploration-exploitation trade-offs, and the training procedure. These dimensions became the columns of Table B.1 and Table B.2.

Three more surveys [15, 17, 36] agree on the general picture. Most work in the field uses equity markets, daily data and discrete trading decisions. Reported results are hard to compare, because each paper builds a different trading environment.

3.2 Article Research: Summarising Different Implementations

Eighteen articles with working implementations were read in full. Table B.1 records the market, the data and the algorithm of each, and Table B.2 the design choices inside each agent. This section groups the articles by the market they trade.

3.2.1 Forex Market Implementations

Three articles trade currencies, and all three limit the agent to a discrete choice between buying, holding and selling. Carapuço et al. [18] ran a Deep Q-Network on raw tick data, with about one decision every two hours, and outperformed the benchmark by a wide margin. Huang [19] used a Deep Recurrent Q-Network on 15-minute bars and outperformed the benchmark with explicit and implicit costs charged. Rundo [20] used the same kind of model on one-minute bars and also reported outperformance, but with an accuracy high enough to suggest look-ahead bias and a large maximum drawdown. The two results suggest that recurrence alone does not make very short timeframes workable.

3.2.2 Stock Market Implementations

Most of the work reviewed trades equity markets. Among the critic-only studies, Sim and Kirk [37] and Théate and Ernst [38] trained Deep Q-Networks on daily bars. The first reported a policy that generalised to unseen assets, and the second consistent results in calm and volatile periods. Li et al. [39] found that Asynchronous Advantage Actor Critic converged better than a Deep Q-Network, and both beat the benchmark.

The actor-critic studies are closer to the design used in this work. Xiong et al. [40], Yousefi [41] and Gran et al. [42] applied DDPG to daily or monthly bars and outperformed their benchmarks. Yousefi found DDPG more stable than Advantage Actor Critic and attributes this to its replay buffer and its noise process. Azhikodan et al. [43] added news sentiment to the state and reported better results, but over only about five months. Majidi et al. [44] used the Twin Delayed variant of DDPG with a continuous action in $[-1, 1]$, and beat the cryptocurrency benchmark but not the equity benchmark. Only Sagiraju and

Mogalla [45] found DDPG worse than the alternatives. Of four algorithms that all beat the benchmark, DDPG and Advantage Actor Critic did worse than Proximal Policy Optimisation and the Deep Q-Network.

Two studies combine DDPG, Advantage Actor Critic and Proximal Policy Optimisation in an ensemble. Yang et al. [46] switched between the three with a turbulence measure and beat the benchmark. The ensemble of Kong and So [47] had unstable returns and underperformed. The two results indicate that combining agents does not by itself make the results stable. Wu et al. [48] found a gated recurrent policy-gradient agent better than a gated recurrent Q-network.

3.2.3 Other Market Implementations

Three articles trade other instruments. On exchange-traded funds, Chen and Gao [49] found a recurrent Deep Q-Network better than a fully connected one. On futures, Zhang et al. [50] applied Advantage Actor Critic over a long short-term memory network and outperformed the benchmark. Jia et al. [51] traded exchange-traded funds with DDPG over a long short-term memory network. A version that set the position size continuously did better than one with a fixed size, with costs charged. It is the closest study to this work.

3.2.4 Comparative Findings

The state space is usually price data plus technical indicators, and the studies with strong results almost always include indicators [18, 19, 46]. This agrees with Section 2.3: a raw price series is not Markovian, and indicators put enough of the recent history into the state for the property to hold approximately.

The action space is discrete in 15 of the 18 studies, usually buy, hold and sell [20, 40, 43]. Three studies use a continuous action [44, 50, 51], and only one uses it to size a position [51]. A discrete action set is easier to learn, but it leaves position sizing to a fixed rule that the agent cannot adapt. The reward is the total return in 14 studies, the Sharpe ratio in three [45, 47, 51] and the log return in one [44]. The studies with a risk-adjusted reward do not report a consistent gain from it.

The bias-variance trade-off is handled with feature engineering, normalisation, regularisation or changes to the network [38, 39]. Exploration uses the ϵ -greedy policy with a discrete action set, and additive Gaussian noise with a continuous one [44, 48].

Only five of the 18 studies state that their result holds with explicit and implicit costs [19, 42, 46, 50, 51]. The others do not say whether their returns are gross or net. Training usually uses a train-validation-test split, and one study keeps learning online as new data arrives [19].

The data covers long periods at a coarse resolution. Twelve of the 18 studies cover ten years or more, with a median of eleven years. Seventeen work on bars, 13 of them daily, and only one uses quote-level data [18].

3.3 Research Gap: Positioning This Work

Read against one another, the findings above also show what the field has not done yet. Table 3.1 sets out the comparison.

Table 3.1: How this work compares with the eighteen surveyed articles.

Dimension	Surveyed literature (18 articles)	This work
Market	Stocks 12, Forex 3, ETFs 2, Futures 1	Forex, the seven major pairs
Data resolution	Bars in 17 of 18, daily in 13; one study uses quotes	Quote-level tick data, hourly observations and daily decisions
Data span	Ten years or more in 12 of 18, median eleven years	Eleven years, 2015 to 2025
Action space	Discrete 15, continuous 3; one sizes the position	Continuous, sets direction and size together
Reward function	Total return 14, Sharpe ratio 3, log return 1	Log return; models selected by the Calmar ratio and a risk-adjusted composite score
Trading costs	Charged in 5 of 18	Quoted spread at the moment of the trade plus the commission of a swap-free raw-spread account
Instruments per design	One agent per instrument, or one agent on a basket	One design trained separately on seven pairs, then combined
Reported robustness	The candidate count and the sensitivity to starting conditions are not stated	Seven walk-forward folds, five starting balances and the number of seeds trained per pair

The first gap is in the action. The three studies that trade currencies [18–20] all use a discrete action, and the three that use a continuous action [44, 50, 51] all trade something else. So no study reviewed here has a currency agent that sets its own position size, and there is no existing result to measure this work against.

The second gap is trading costs. A strategy tested without costs is not tested against the market it claims to trade. In Forex this matters more, because the spread widens when liquidity falls and around releases of economic data, which are the moments when a model trained on an average spread would expect to act. The third gap is resolution. A long history is already normal in this literature, but working inside the bar is rare. A bar reports four prices and hides the order in which the price reached them, and that order decides which spread was paid and whether a stop was reached before a target.

The fourth gap is evidence. No study reviewed here states how many candidates its reported model was selected from, or how the figure changes when the evaluation is repeated from a different starting condition. Deep reinforcement learning is sensitive to initialisation, so a single figure cannot separate the method from the particular run. A model chosen as the best of many also tells as much about the sample it was chosen from as about the market it traded [16, 21]. The fifth gap is how results are combined. None of the studies trains one design separately on several instruments and then measures what those agents do when they run together, which is how a trader who holds several currency pairs would deploy them.

This work is placed in these gaps. One design is applied without change to the seven major cur-

rency pairs, with a continuous action that sets the direction and the size of the position together. The evaluation runs on quote-level data over eleven years. It charges the spread quoted at each trade plus the commission of a swap-free raw-spread account that a retail trader can open, in training as well as in evaluation. The model for each pair is chosen from waves of 20 seeds and kept only if it passes eleven gates, and the number of candidates is stated. The seven agents are then combined into an equal-weight portfolio, and Chapter 5 reports both the single results and the portfolio.

The reward is the log return, as in one of the surveyed studies [44]. The difference is that this work selects its model with measures other than the reward. The checkpoint kept from each walk-forward fold is the episode with the best validation Calmar ratio, and among the models that pass the gates a composite rank of return, risk-adjusted ratios, drawdown and policy behaviour chooses one. So a policy that earns its return by taking more risk does not win on return alone. Chapter 4 gives the detail.

The studies reviewed here show that a deep reinforcement learning agent can learn a profitable trading policy. They say much less about the conditions in which this holds, and they rarely describe their trading environment well enough to build it again. An agent trained on a simplified market learns to exploit the simplification, so the value of any result depends on how the environment charges for trading, moves time forward and fills orders. Chapter 4 describes the environment built for this work and the agent that runs in it.

4

Implementation

Contents

4.1 Trading Framework: The Environment the Agent Trades In	31
4.2 Agent Design: Observation, Action and Reward	36
4.3 Experimental Procedure: Training, Selection and Deployment	41

Chapter 3 showed what the reviewed literature leaves open. This chapter describes what was built to address it. It covers the environment the agent trades in, the agent itself, and the procedure used to train the agent and to choose one model for each currency pair.

The sections follow this order on purpose. The agent cannot be described without its environment, because the environment gives the agent its observations and charges it for its actions. No number the agent produces means anything unless that environment is accurate. So the framework comes first, then the agent, and then the experimental procedure.

4.1 Trading Framework: The Environment the Agent Trades In

No existing framework could both train an agent offline on quote-level history and run the same agent on a live account without rewriting it. Building one took a large part of the effort in this work. Its design

choices decide how far the results in Chapter 5 can be trusted. Three properties were required. First, the tested system and the deployed system had to be the same code, so that a result obtained offline says something about live behaviour. Second, the data had to have a fine enough resolution to rebuild the price path inside a bar, because that path decides what a position actually paid. Third, the costs had to be the costs of a real account, not simplified values chosen for convenience.

4.1.1 System Architecture: Three Execution Contexts

The framework has to be written in two languages. The strategy logic, the learning models and the market abstractions are written in Python, which has the numerical and machine learning libraries. Access to the broker is written in C#, because cTrader [22] runs algorithmic strategies as cBots on the .NET runtime and offers no other way to reach its order book. The two parts communicate through named memory-mapped files with paired auto-reset event handles, one pair for each direction. They work as a single-slot request and response exchange. The platform writes an update and signals. Python reads it, writes an action and signals back, and then the platform continues. Two kinds of message cross this boundary. Updates go from the platform to Python, and actions go the other way. Figure 4.1 shows the arrangement. The important result of this design is that the strategy object does not know where its updates come from. The same class, with the same weights, runs in three contexts.

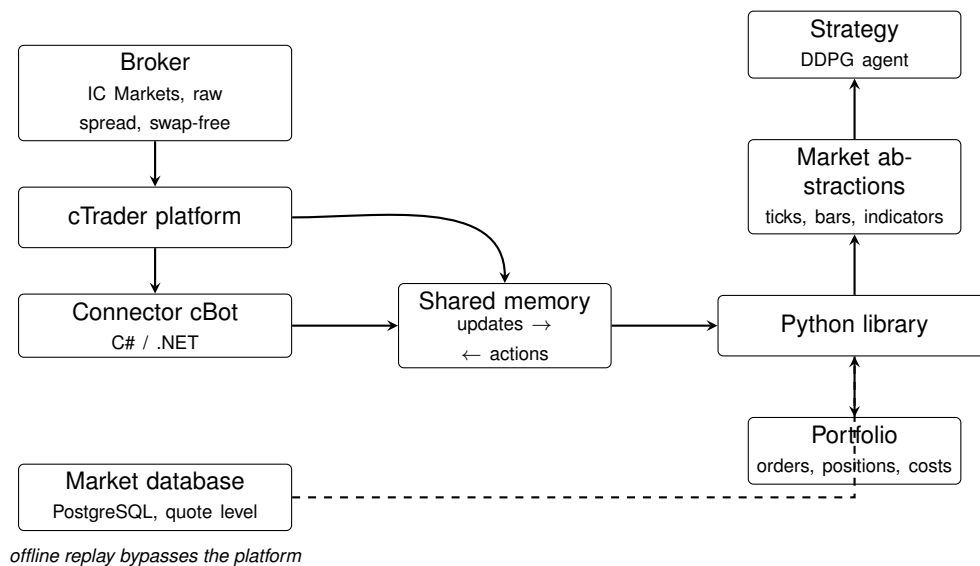


Figure 4.1: Framework architecture. The strategy receives updates and returns actions. It does not know whether the updates come from a live account, from the platform’s own backtester or from the stored quote database.

In the first context the platform is not used. Python replays stored quotes directly from the market database. This is the fastest of the three contexts, and it is the one used for training and for every result reported in this work. In the second context the strategy runs inside cTrader’s own backtesting tab on the

platform's historical data. This gives an independent check of the offline engine. In the third context the same strategy runs on a live account. It receives real quotes and sends real orders. The benefit of this design is that the three contexts differ only in where the updates come from. When a strategy behaves one way in a backtest and another way when deployed, the usual cause is two separate implementations that have diverged over time. Here there is only one implementation, so this problem does not occur.

4.1.2 Market Data: Collection, Storage and Resolution

The historical data comes from the broker's own servers, through the historical endpoint of the cTrader Open API. A service of the framework downloads every tick from January 2014 onwards and keeps the record current, about one second behind the market. It covers the 28 pairs that can be formed from the eight major currencies, so every conversion into a major account currency has a direct pair. The ticks are stored in one compressed table of a PostgreSQL database. A tick holds the bid and the ask quoted at one instant. Bars are not stored. They are built from the ticks when a run needs them, so a bar is a view of the tick tape and not a separate copy of the prices. Conversion rates are read the same way, from the tick tape of the conversion pair at the instant of each fill. Table 4.1 lists the data held for the seven pairs used here. The tick tape is the primary record. This is what allows the engine to rebuild the path a price took inside a bar, instead of interpolating between its end points.

Table 4.1: Tick data of the seven pairs from January 2014 to December 2025, as downloaded from the broker's servers. The year 2014 only warms up the indicators. Bars of any timeframe are built from these ticks when a run needs them.

Pair	Ticks	Trading days	Ticks per day
AUD/USD	218,427,975	3,732	58,528
EUR/USD	235,026,506	3,732	62,976
GBP/USD	269,366,880	3,732	72,178
NZD/USD	196,117,477	3,732	52,550
USD/CAD	220,059,726	3,732	58,966
USD/CHF	187,169,804	3,732	50,153
USD/JPY	275,370,555	3,732	73,786
Total	1,601,538,923	26,124	61,305

Two properties of this data must be stated, because otherwise they would look like defects.

Daily bars follow the broker's trading day, which starts at 17:00 New York time, so the daily bar of a date opens at 21:00 or 22:00 UTC of the day before, depending on daylight saving. Hourly bars open on the hour in UTC. The bars built from the tape were checked against the broker's own bars. Their timestamps agree on every bar from one minute to one month, and their open, high, low and close agree exactly from one minute to one day. The period from 11 to 25 January 2016 is missing from all seven pairs. The gap comes from the broker: its server returns no ticks for those days, so the gap cannot be repaired. The gap is the same for all seven pairs, so it does not bias any comparison between them. It lies in the training years of every fold, and no validation or test year contains it.

4.1.3 Cost Model: Spread, Commission and Account Type

The execution engine advances on ticks, not on bar closes. A position opened during a bar is filled at a quote that existed within that bar. A stop or target attached to it is tested against the quotes in the order they arrived. This matters for the reason given in Chapter 3. A bar reports four prices but hides the order in which the price reached them. That order decides whether a stop was reached before a target, and which spread was crossed.

The account modelled is a swap-free raw-spread retail account. Each of the three charges described in Section 2.1.5 is applied at the rate that such an account pays. A buy is filled at the ask and a sell at the bid, both read from the tick tape at the instant of the fill. The spread is therefore charged by construction, as part of every fill, and is not deducted afterwards. The commission is 3.5 US dollars per standard lot of 100,000 units per side, or 7 US dollars per round turn, which is the rate the broker charges on its raw-spread accounts. The swap is zero. On a swap-free account this is the rate the account really charges, so it is not an approximation. Table 4.2 lists the full configuration.

Table 4.2: Cost and account configuration applied in training and in every evaluation reported in this work.

Setting	Value	Meaning
Account type	Raw spread, swap-free	An account that a retail client can open, with the interbank spread and no charge for overnight financing
Spread	Accurate	The bid-ask difference recorded at the moment of the trade, read from the tick tape and not assumed
Commission	3.5 USD per lot per side	Charged on traded volume: 3.5 US dollars per standard lot of 100,000 units per side, or 7 per round turn, each fill rounded half up to the cent
Swap	0	The rate a swap-free account charges, so not an approximation
Account currency	USD	Profit and loss converted at the rate of the conversion pair recorded at the instant of the fill
Initial balance	10,000	Reported across five starting balances, as described in Chapter 5
Leverage	30	The retail limit for major currency pairs
Position mode	Netting	One net position per instrument, not separate hedged positions
Risk per position	1%	The reference risk ρ of Equation (4.5), the same for every pair

The choice to read the spread from the ticks, instead of assuming a value for it, should be supported with data. Assuming a value is common, and the error it causes is not small. Figure 4.2 plots the monthly mean spread of each pair across the evaluation window. For EUR/USD, the most liquid pair in the market, the monthly mean goes from 0.77 to 13.00 points. This is a factor of seventeen between the quietest month and the busiest one. USD/CHF reaches 37.82 points in January 2015, when the Swiss National Bank removed its floor against the Euro. For each of the seven pairs, the spread in the widest month is at least five times the spread in the narrowest month.

The pattern of this variation matters more than its size. Spreads widen when liquidity falls and around the events that move prices. These are the moments when a model trained on an average spread is

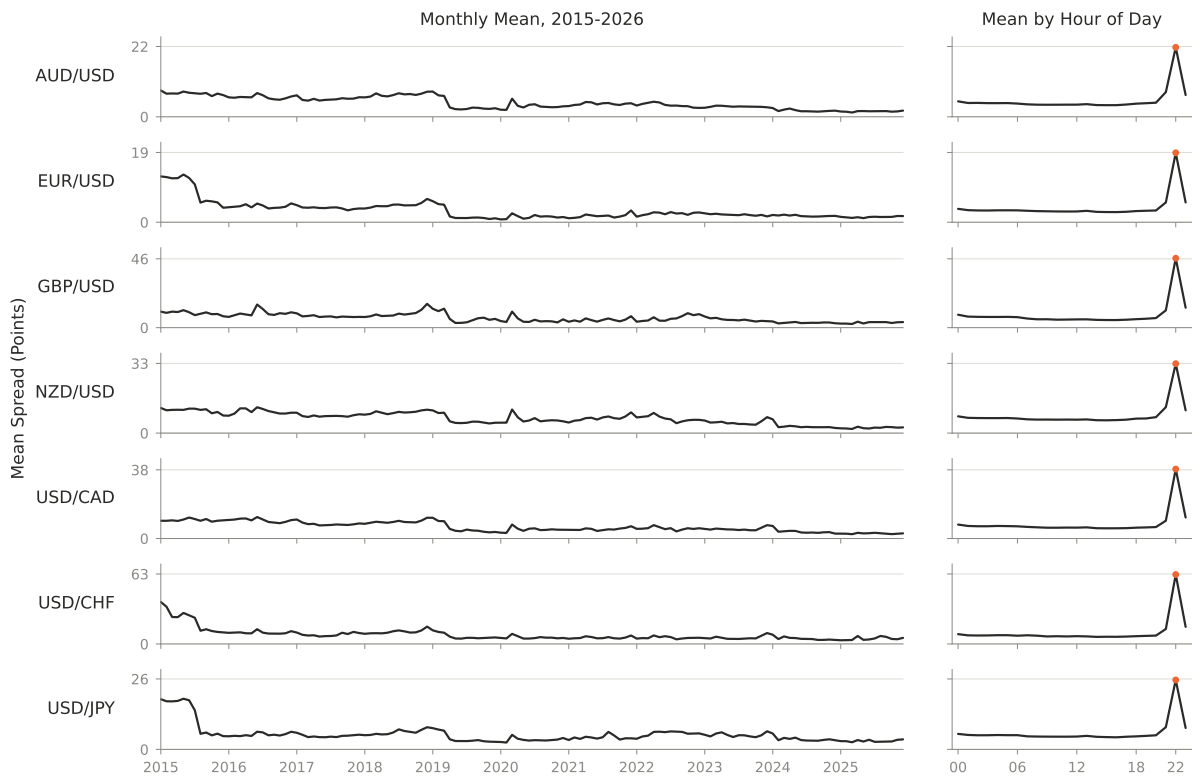


Figure 4.2: Monthly mean spread in points for the seven major pairs, 2015 to 2026, computed from the full tick tape. EUR/USD is drawn in black. A model evaluated at a fixed spread is evaluated in market conditions that do not occur in any month of this sample.

most likely to act. A strategy charged a constant spread is therefore charged the wrong amount, and the error is correlated with its own trading activity. The error also goes in the direction that makes the strategy look better.

The engine was validated against cTrader's own backtester, not against itself. Five reference backtests were run inside the platform, on three currency pairs and one index, in three account currencies, over windows of one to eleven years. They used a test strategy that exercises every position action the engine supports: opening, partial closing, increasing, reversing, and exits by stop loss and take profit. The same strategy was then run offline on the tick tape. Every position opens at the same time and on the same side in both engines, and between 93% and 99.6% of positions also close at the same time. The other closes differ by one tick or by one trailing step of a stop. On the four runs of currency pairs, the net profit and loss of the closed trades differs from the platform's by between 0.95 and 83.64 units of the account currency, on runs of about five hundred to a thousand positions. Part of that difference is deliberate: the engine rounds each commission half up to the cent, as the live broker does, where the platform's backtester truncates it. On the index the engine also charges the broker's minimum commission of 4 US dollars per side, which the platform's backtester leaves out. Live trades on demo accounts

were checked in the same way, and the engine reproduced every closing deal to the cent, on a netting account and on a hedging account. The engine therefore reproduces the platform it was built against, and it can also do more than that platform. The same strategy is evaluated offline directly on the tick tape. This runs between 6.7 and 35 times faster than the platform's own backtester, and over windows that the backtester will not cover. The cost model can be set to any account the broker offers, not only to the account the terminal happens to be connected to.

Two consequences follow. Both are stated here so that Chapter 5 can be read correctly. First, the returns reported in this work are net of the spread actually quoted and of a real commission schedule. They are therefore lower than the same policy would show on bar closes with a fixed spread or no spread, and they cannot be compared directly with results obtained in that way. Second, because the account is swap-free, no financing cost is estimated or left out. A strategy that holds positions over many nights is charged exactly what such an account would charge it.

4.2 Agent Design: Observation, Action and Reward

The environment of Section 4.1 supplies prices and charges for trades. Three more decisions are needed to turn it into a reinforcement learning problem: what the agent sees, what it is allowed to do, and what it is rewarded for. This section states these three decisions and then describes the algorithm that learns from them. Each decision is given with the reason for it. In every case the alternative was tried and worked worse, and those failures are part of the result.

4.2.1 Observation Space: Indicators, Encoding and Normalisation

The agent observes a vector of 38 values at each hourly bar. None of these values is a raw price. A price level carries no information that the agent can generalise from. A policy learned at a price of 1.05 has no reason to work at 1.20, and in eleven years of history the price moves by more than that. Every feature is therefore a ratio, and the whole observation is dimensionless. Sixteen technical indicators, listed in Table 4.3, give the market context. They are split into a fast group, which describes the last hours to days, and a slow group, which describes regimes that last months. This lets the agent tell a move within a trend apart from a change of trend.

The indicator values are encoded as follows. Here C is the bar close, σ_f and σ_s are the fast and slow realised volatilities, A is the average true range, and M_i is the i -th moving average, with period N_i :

$$\frac{A}{C}, \quad \sigma_f, \quad \ln\left(\frac{\sigma_f}{\sigma_s}\right), \quad \text{ER} \quad (4.1)$$

Table 4.3: The sixteen technical indicators that form the market context of the observation. Periods are in hourly bars. Each moving average gives two features, its distance from the price and its own differential, both in units of the average true range. Each momentum reading gives one feature, scaled by the movement expected over its lookback.

Indicator	Family	Period	Role in the observation
ATR	Volatility	14	Volatility unit; divides every trend feature and sets the stop distance
RV (fast)	Volatility	16	Short-horizon realised volatility; divides the momentum features
RV (slow)	Volatility	480	Long-horizon realised volatility; its ratio to the fast reading gives the volatility regime
ER	Trend	120	Efficiency ratio; how directional recent movement has been, bounded in $[0, 1]$
ROC Fast	Momentum	24	One day
ROC Medium	Momentum	120	One week
ROC Slow	Momentum	480	One month
ROC Regime	Momentum	1,440	One quarter
ROC Epoch	Momentum	2,880	Two quarters
SMA Fast	Trend	24	One day
SMA Medium	Trend	120	One week
SMA Slow	Trend	480	One month
SMA Regime	Trend	1,440	One quarter
SMA Epoch	Trend	2,880	Two quarters
SMA Cycle	Trend	4,320	Three quarters
SMA Era	Trend	5,040	Roughly one year

$$\frac{\text{ROC}(N_j)}{\sigma_f \sqrt{N_j}}, \quad \frac{C - M_i}{A}, \quad \frac{M_i - M_i^{-1}}{A} \quad (4.2)$$

Each formula removes a different dependence. Dividing the average true range by the close gives volatility as a fraction of the price. The logarithm of the ratio of fast to slow volatility shows the volatility regime. It is zero when the two agree, positive when volatility is rising and negative when it is falling. Dividing a momentum reading by $\sigma_f \sqrt{N_j}$ scales it by the movement expected over that lookback under a random walk. A value near one then means a move of normal size, whatever the period. The distance from a moving average, measured in units of the average true range, says how far the price is from its trend compared with the current volatility. The differential of the same quantity gives the direction in which the trend is moving. The other features are the position in the calendar, encoded as four sine and cosine pairs, the current signed exposure as a fraction of the reference volume of Equation (4.5), and six values that describe the last bar as log returns of its open, high, low and close against the previous close, divided by σ_f .

Features that are not bounded by construction are standardised with a causal exponentially weighted z-score, with $\alpha = 1/200$. Here x_t is a raw value, and μ_t and s_t^2 are its running mean and variance:

$$z_t = \frac{x_t - \mu_{t-1}}{\sqrt{s_{t-1}^2 + \epsilon}} \quad (4.3)$$

$$\mu_t = \mu_{t-1} + \alpha \delta_t, \quad s_t^2 = (1 - \alpha) (s_{t-1}^2 + \alpha \delta_t^2), \quad \delta_t = x_t - \mu_{t-1} \quad (4.4)$$

The subscript $t-1$ in Equation (4.3) is the most important detail of this design. A value is standardised with statistics that end at the previous step. Only after that is the value added to those statistics. A z-score computed over the whole sample would leak future information into every observation, and it would inflate every result derived from it. Features that are already bounded, such as the exposure fraction, the efficiency ratio and the calendar encodings, have a flag that skips this layer, so their natural range is kept.

4.2.2 Action Space: Position Sizing and Risk

The agent outputs a single continuous value $a \in [-1, 1]$. Its sign is the direction of the position, and its magnitude is the fraction of a reference size to hold. Direction and size are therefore set by one decision. The discrete action sets reviewed in Chapter 3 cannot express this. The reference size is fixed-fractional. Let B be the account balance, ρ the risk fraction per position, k the stop distance in multiples of the average true range, and x the rate that converts one unit of the quote currency into the account currency at the close of the decision bar. The reference volume V_{ref} is the volume that loses ρB if the price moves $k \cdot A$ against it. The target volume is this reference scaled by the action:

$$V_{target} = a \cdot V_{ref}, \quad V_{ref} = \frac{\rho B}{k A x} \quad (4.5)$$

with $\rho = 1\%$ and $k = 1.5$ throughout this work. The agent therefore never chooses an absolute number of units. It chooses a fraction of a size that the risk rule has already set, and this size becomes smaller on its own as volatility rises.

The conversion x makes the risk the same on every pair, whatever currency the pair is quoted in. Without it, a pair quoted near 150 Japanese yen would take a risk about a hundred and fifty times smaller than intended. The target volume is rounded to the broker's volume step, and a volume below the broker's minimum is raised to the minimum. Measured over the 28 pairs of the eight major currencies and eight account currencies, a stop of fifty pips risks between 0.968% and 0.994% of the balance. The shortfall comes from the volume step.

4.2.3 Reward Function: Log Return, Scaling and Clipping

The reward at each hourly step is the log return of account equity, scaled and then clipped:

$$r_t = \text{clip}\left(\lambda \ln \frac{E_t}{E_{t-1}}, -c, +c\right), \quad \lambda = 1000, \quad c = 1 \quad (4.6)$$

The agent acts once per day, so one transition of the Markov decision process covers one day, and its reward is the sum of the hourly rewards of that day. The log return is a natural choice for a trading

objective because it is additive over time. The per-step rewards telescope to $\ln(E_T/E_0)$, so maximising their sum maximises the final compounded wealth. The log return also does not depend on the account size. The scale factor is needed, and the reason for it comes from a measured result. Moody and Saffell describe differential risk-adjusted rewards, which accumulate an online estimate of a Sharpe or Sortino ratio. These rewards are scale-invariant: doubling every position leaves the ratio the same. An agent that optimises such a reward gets no gradient that pushes its position size up. In this environment the policy collapses to positions of about 3% of the size available to it. The resulting policy is directionally sensible, but it has no financial value. A scale-sensitive reward is therefore needed. The factor λ raises an hourly equity log return, which is usually of order 10^{-4} , to a magnitude that the critic can distinguish from its own approximation error.

The clip is a numerical safeguard of the kind introduced with the Deep Q-Network [12]. It is applied after the encoding. Its effect must be stated clearly here, because λ and c are not independent. Together they clip the raw log return at $\pm c/\lambda = \pm 0.1\%$. On the four delivered models that pass every gate, the bound is reached on 68% to 72% of the 68,150 hourly steps of the full window, and the median absolute scaled reward is at the bound. So for most steps the agent is told the direction of the equity move but not its magnitude. This is a real property of the objective. It is reported here so that the learned behaviour in Chapter 5 is read against the reward the agent actually received.

4.2.4 Learning Algorithm: DDPG and its Adaptations

The algorithm is the Deep Deterministic Policy Gradient of Section 2.4. The actor, the critic and their delayed targets are as described there, and they are trained from a replay buffer with Ornstein-Uhlenbeck exploration noise. The equations of that section are not repeated here. The rest of this section gives the six ways in which the implementation differs from the original, each with its reason. Table 4.4 compares every value with the published one.

The first difference is the observation and the reward, described above.

The second is the size of the networks. The original uses hidden layers of 400 and 300 units for low-dimensional control tasks. Here they have 64 and 32 units. A 38-dimensional observation of a noisy, non-stationary series supports much less capacity than a physics simulator with a clean reward. Larger networks fitted the training window but did not generalise beyond it. The third is the discount factor, which was raised from 0.99 to 0.9995. The value was chosen when the agent still acted every hour, where it gives a horizon $1/(1 - \gamma)$ of about two thousand hours, or about four months. The agent now acts once per day, so the same value gives a horizon of about two thousand trading days, or about eight years. This is longer than any training window, so the objective is close to the undiscounted sum of the rewards over a window.

The fourth is the normalisation inside the networks. The original applies batch normalisation to the

Table 4.4: Hyperparameters compared with the original Deep Deterministic Policy Gradient [6]. Three values were changed and seven mechanisms were added. The rest are the published values.

Parameter	Original	This work	Status	Reason for the difference
Actor hidden layers	400×300	64×32	Changed	Noisy, non-stationary series supports less capacity
Discount factor γ	0.99	0.9995	Changed	Horizon of about two thousand daily decisions, close to the whole training window
Hidden normalisation	Batch	Layer	Changed	Inference on one observation must match training on batches
Actor learning rate	10^{-4}	10^{-4}	Unchanged	
Critic learning rate	10^{-3}	10^{-3}	Unchanged	
Soft update rate τ	0.001	0.001	Unchanged	
Batch size N	64	64	Unchanged	
Replay capacity	10^6	10^6	Unchanged	
Critic weight decay	10^{-2}	10^{-2}	Unchanged	
Noise θ, σ	0.15, 0.2	0.15, 0.2	Unchanged	
Final layer init	$U[-3 \times 10^{-3}, 3 \times 10^{-3}]$	same	Unchanged	
Gradient clip	—	1.0	Added	Bounds the size of one update caused by a rare large error
Warmup transitions	—	3,000	Added	First updates drawn from a varied buffer
Actor penalty β	—	0.001	Added	Prevents saturation of the output tangent and collapse to a constant action
Reward scale λ	—	1,000	Added	Raises an hourly log return above the critic's approximation error
Reward clip c	—	1	Added	Numerical safeguard; limits the raw log return to $\pm 0.1\%$
Decision step	—	one day	Added	One transition per trading day; its reward is the sum of the hourly rewards
Mirrored episodes	—	every second	Added	Data augmentation: as many falling markets as rising ones

hidden layers. This implementation applies layer normalisation instead. The reason is deployment. Batch normalisation behaves differently on a single sample than on a training batch, because it depends on running statistics collected during training. On a live account, the agent must act on exactly one observation at a time. Layer normalisation normalises across the features of one sample. The computation done during training is therefore the same as the computation done in deployment. The target networks also do not need a second set of running statistics that must be kept in sync.

The fifth is gradient clipping. The original uses none. Here the gradient norm of both the actor and the critic is clipped to 1 before each optimiser step. This limits the size of a single update. It stops a rare large temporal-difference error from moving the parameters far enough to make the critic unstable. It is only a stability measure. In particular, it does not solve the collapse discussed next, which is caused by a lack of gradient, not by too much of it.

The sixth is a penalty on the actor. This is the only change that alters the objective itself, and not just the way it is optimised. It was added because the published algorithm does not train on this problem. The actor's final layer applies a hyperbolic tangent to a pre-activation $u(s)$ to keep the action bounded. The derivative of this function vanishes as $|u|$ grows. If the policy is pushed into that region, it stops responding. The action saturates at ± 1 , no gradient flows back, and the agent holds one constant

position for the rest of training. The fix is to penalise the magnitude of the pre-activation directly. This keeps the policy in the region where the tangent does not saturate. Figure 4.3 shows where in the network this penalty acts:

$$L_{actor} = -\frac{1}{N} \sum_i Q(s_i, \tanh u(s_i) | \theta^Q) + \frac{\beta}{N} \sum_i u(s_i)^2 \quad (4.7)$$

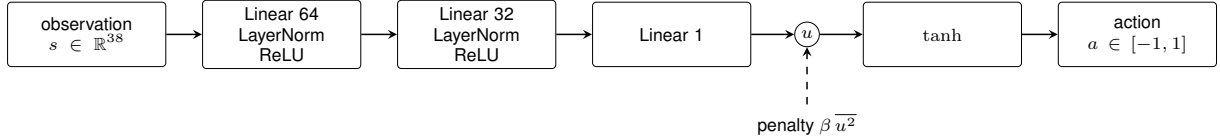


Figure 4.3: The actor. Layer normalisation replaces the batch normalisation of the original, so a single observation is processed exactly as a training batch is. The penalty of Equation (4.7) acts on the pre-activation u , before the tanh. This is where saturation would otherwise remove the gradient.

with $\beta = 0.001$. Setting $\beta = 0$ recovers the original actor loss exactly, so the change extends the original loss and does not replace it. Together with the scale-sensitive reward, this penalty prevents policy collapse. The two mechanisms deal with different parts of the same failure. The reward supplies a gradient that rewards larger positions, and the penalty keeps the actor in the region where that gradient can act. Everything else follows the published algorithm. The learning rates, the soft update rate, the batch size, the replay capacity, the exploration noise parameters, the critic weight decay and the layer initialisation are the values given in the original work. A warmup period of 3000 transitions is used before learning begins. This way the first updates are drawn from a buffer with some variety in it, and not from the first few correlated steps of an episode.

4.3 Experimental Procedure: Training, Selection and Deployment

Training is stochastic, so the same configuration run twice gives two different policies. A procedure is needed to decide which of them is reported, and why. This section describes that procedure: how the data is split, how one model is trained, how one model is chosen for each pair, and the two deployment decisions that sit between the learned policy and the orders that are actually sent.

4.3.1 Data Splits: Anchored Walk-Forward

The data used runs from 1 January 2015 to 31 December 2025. It is split with an anchored walk-forward scheme [16]. There are seven folds, and every fold starts its training on 1 January 2015. Fold k trains on every year from 2015 up to the year before its validation year, and it validates on that year:

$$\text{fold } k : \underbrace{[2015, 2017 + k)}_{\text{training}} \underbrace{[2017 + k, 2018 + k)}_{\text{validation}}, \quad k = 1, \dots, 7 \quad (4.8)$$

The first fold trains on three years and validates on 2018. The last fold trains on nine years and validates on 2024. The year 2025 is the test year. No fold trains on it. Figure 4.4 shows the scheme.

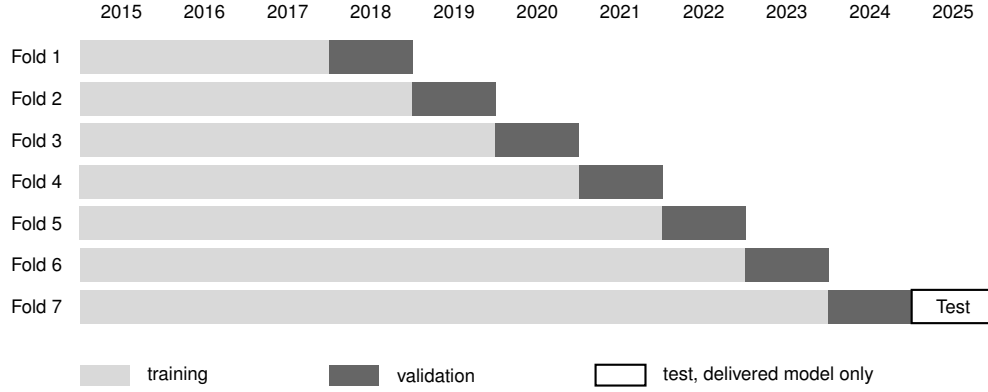


Figure 4.4: Anchored walk-forward. Every fold trains from scratch on all the years since 2015 and validates on the next one. The model of the last fold is the delivered model, and it is the only one that trades the test year.

A single split gives one validation year, so one good or one bad year decides the result. Here every year from 2018 to 2024 is traded by a model that never trained on it, and the seven years together form a walk-forward record of seven years. The folds are anchored, which means that the training window always starts in 2015 and grows by one year per fold. The alternative is a rolling window of fixed length, where each fold may also continue from the weights of the fold before it. An earlier campaign used rolling windows of three years with continued weights, and its policies collapsed to holding one side of the market. The anchored folds, each trained from scratch, did not show this collapse.

4.3.2 Training Procedure: Seeds, Episodes and Checkpoints

Within one fold, training works as follows. A seed fixes the network initialisation, the stream of exploration noise and the order in which the replay buffer is sampled, so it determines the whole run. The network starts from new random weights in every fold. The agent then runs 10 episodes. An episode is one pass over the training window. Weights are not reset between episodes, so the network at the start of episode $n + 1$ is the network at the end of episode n .

Every second episode runs on a mirrored copy of the training window. The mirror reflects the log price around its first value, so that each price p_t becomes p_0^2/p_t , and it swaps the bid with the ask and the high with the low. A rise becomes a fall of the same size in log terms, while the spread, the timing of every quote and the volatility stay as they were. Over a run the agent therefore sees as many falling

markets as rising ones. This is a form of data augmentation. Its purpose is to stop the agent from learning the direction of its training years instead of a trading rule.

The costs of Section 4.1 are charged during training, and the rebalance threshold described below is applied during training as well. The agent therefore learns under the same costs that it is judged on. The campaign also trained a second arm without costs and without the threshold, which applied both only at evaluation. Chapter 5 compares the two arms.

After each episode the agent is evaluated once over the validation year, with learning turned off. This evaluation is reduced to a single number, the Calmar ratio, which is the annualised return divided by the maximum drawdown. Let f_n be this value and let g_n be an eligibility test defined below. The state kept after episode n is

$$\theta^* \leftarrow \theta_n \quad \text{if} \quad (g_n \wedge \neg g^*) \vee (g_n = g^* \wedge f_n > f^*) \quad (4.9)$$

which prefers an eligible episode to an ineligible one, whatever the scores. Otherwise it keeps the higher score within the same eligibility class. The saved state is a copy. Training continues from θ_n and not from θ^* , so a later episode never restarts from an earlier checkpoint. The eligibility test g_n rejects policies that are degenerate. It does not reject a policy only because it performs poorly. An episode passes only if it opened at least ten positions, spent at least three hundred bars on each side of the market, and held its weaker side for at least 30% of the time it held its stronger one. A policy that never trades, or that is long for the whole year, can reach a good Calmar ratio in a trending year without having learned anything about when to hold a position. The test removes such a policy before the scores are compared.

The checkpoint kept in fold k is the model of that fold. The models of folds 1 to 6 are used only to trade their own validation years. The model of fold 7, trained on 2015 to 2023 and checkpointed on 2024, is the delivered model of the seed. Seeds were trained in waves of up to twenty per pair, and the runs of a wave differ only in the seed. Figure 4.5 shows how the stages fit together.

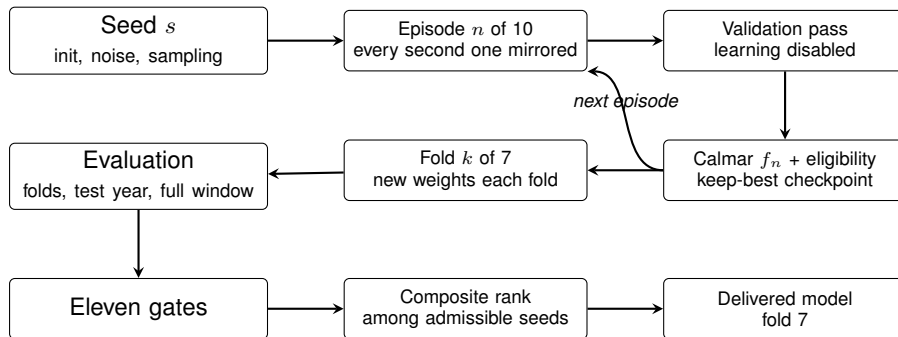


Figure 4.5: The stages of the procedure. The Calmar ratio chooses among the episodes of one fold. The gates and the composite rank of Section 4.3 choose among seeds.

4.3.3 Model Selection: Gates and Composite Ranking

Once a wave is trained, every seed is evaluated with the cost model of Table 4.2 on three kinds of window. Each validation year from 2018 to 2024 is traded by the model of its own fold. The test year 2025 and the full window from 2015 to 2025 are traded by the delivered model. The full window contains the training years of the delivered model, so it is not an out-of-sample result. It is used to measure how the model trades over the longest record available.

A seed is admissible only if it passes the eleven gates listed in Table 4.5. The first six are about money. The model must have a positive return, and a return above that of the pair itself, over the validation years, over the test year and over the full window. The last five are about behaviour. They exist because a model can make money for the wrong reason. A model that is long for the whole window of a rising pair makes money and has learned nothing, and the same is true of a model that trades only one side, holds positions for a few hours, or stops trading in its last fold.

Table 4.5: The eleven gates a seed must pass to be admissible. The first six test money, the last five test behaviour. The behaviour gates are measured on the full window with the delivered model, except Alive, which looks at the last fold.

Gate	Window	Condition
Validation	2018 to 2024, fold models	The mean of the seven yearly returns is positive
Validation pair	2018 to 2024, fold models	The seven yearly returns chained together beat the pair's own moves chained over the same years
Test	2025, delivered model	The return is positive
Test pair	2025, delivered model	The return is above the pair's own move
Full	2015 to 2025, delivered model	The return is positive
Full pair	2015 to 2025, delivered model	The return is above the pair's own move
Sides	2015 to 2025, delivered model	The weaker side holds at least 20% of the time spent in the market
Regime	2015 to 2025, delivered model	The regime score is above the median of 2,000 time rotations of the same positions, with $p < 0.10$
Holds	2015 to 2025, delivered model	The mean holding period is at least one day, with at least twenty reversals
Sizing	2015 to 2025, delivered model	The magnitude of the action has a positive rank correlation with the trend strength
Alive	2024, fold 7 model	At least ten trades, and in the market at least 10% of the time

The regime gate needs some explanation, because a fixed threshold was tried first and does not work. The regime score is the fraction of the market's movement during which the policy held the correct side. A policy that holds long positions for long periods in a trending pair scores well on it by construction. The score is therefore compared with a null distribution for the same policy. The sequence of positions is rotated in time by a random offset, 2,000 times. A rotated policy holds the same positions for the same durations, but at the wrong moments. Its score measures what the holding pattern of the policy would earn by luck alone. The gate requires the real score to be above the median of the rotated scores, the regime edge, and to be exceeded by fewer than 10% of them. A fixed threshold would be below chance for one pair and too strict for another, because the null differs a lot from pair to pair.

The sizing gate checks the claim that a continuous action allows: that the model takes larger positions

when the trend is stronger. For each of five lookbacks, from one week to one year of trading days, the trend strength is the move of the pair over the lookback divided by its volatility over the same period. The lookback kept is the one where the sign of the action follows the direction of the trend most closely. The gate requires a positive rank correlation between the magnitude of the action and the absolute trend strength at that lookback.

The test gates must be stated clearly, because they decide what the test year can claim. The year 2025 is never used to train a model or to choose its checkpoint. It is used, however, to reject seeds. A seed that loses money in 2025, or does worse than its pair in that year, is not admissible. The validation years are used in the same way: they choose the checkpoint of each fold and they reject seeds. No window in this procedure is therefore free from selection. What the procedure does guarantee is narrower: no model ever trades a year it was trained on, except in the full window, which is reported as a record of behaviour. The 2025 figures of Chapter 5 are the figures of models that passed a filter on that same year, and Table 5.2 reports how many seeds the filter rejected.

When more than one seed of a pair is admissible, one is chosen by a composite of percentile ranks within the pool. For a metric m and a pool of K candidates, the rank of candidate i is

$$R_m(i) = 100 \cdot \frac{|\{j : v_m(j) < v_m(i)\}|}{K - 1} \quad (4.10)$$

where v_m is negated for metrics that are better when smaller, such as drawdown. The metrics are split into three groups, listed in Table 4.6. The score is the unweighted mean of the three group means:

$$S(i) = \frac{1}{3} \sum_{G \in \{\text{money, safety, behaviour}\}} \frac{1}{|G|} \sum_{m \in G} R_m(i) \quad (4.11)$$

Table 4.6: Parts of the composite selection score of Equation (4.11). The three groups have equal weight, so no single metric dominates. The test-year return is one of four money metrics and makes up one twelfth of the score. The ratios are those defined in Section 2.1.5.

Group	Metrics, each used as a percentile rank within the pool of admissible seeds
Money	Return in the test year, annualised return over the validation years, annualised return over the full window, Calmar ratio over the full window
Safety	Sharpe ratio, Sortino ratio, Sterling ratio and inverse maximum drawdown, all over the full window
Behaviour	Two-sidedness (the weaker side's share of the time in the market), regime edge over the rotation null, mean holding period, sizing correlation, inverse absolute beta

Two design decisions are built into Equation (4.11). First, ranks are relative to the pool and not absolute, so no threshold can exclude a candidate that is simply better than all the others. Second, the three groups have equal weight, so a policy cannot win on return alone. While no seed of a pair passes every gate, the pair is represented by a provisional model: the seed that passes the most gates, with ties broken by the composite score. Every pair trades at the same reference risk, $\rho = 1\%$. Chapter 5

reports the same models at lower risk levels as a check, and not as a choice.

4.3.4 Deployment: Decision Frequency and Rebalance Threshold

Two decisions sit between the learned policy and the orders that are actually sent. The first is the decision frequency, which is how often the agent may choose a new action. The agent observes every hourly bar, but it chooses a new action only once per calendar day. A key is built from the timestamp of each bar, and a new action is taken only when that key changes:

$$\kappa(t) = (\text{year}(t), \text{month}(t), \text{day}(t)) \quad (4.12)$$

The key is built from the calendar and not from a count of bars. The gaps and short sessions described in Section 4.1 therefore do not affect it, and the decisions stay aligned with the trading days. Between two decisions the action stays fixed, but the target volume $a \cdot V_{ref}$ of Equation (4.5) is computed again at every hourly bar. The reference volume falls when volatility rises, so a position is reduced within the day when the market becomes more volatile, and increased when it becomes calmer. The action decides the direction and the fraction of risk. The risk rule keeps that fraction constant as the market moves.

The second decision is a rebalance threshold. A target that changes a little at every bar would send a stream of small orders, and each of them would pay the spread and a commission. An order is therefore sent only when the change in the target position is large enough to be worth its cost:

$$|V_{target} - V_{current}| \geq \max(V_{min}, \eta |V_{ref}|), \quad \eta = 0.20 \quad (4.13)$$

where V_{min} is the broker's minimum volume. The threshold is a fraction of the reference volume, not of the position currently held, so it does not shrink to zero as a position is reduced. Table 4.7 isolates the effect of each decision on one delivered model, with nothing retrained. The rows use the same weights with different execution settings, so the differences are attributable to the deployment layer alone.

Table 4.7: Effect of the two deployment decisions, measured on the delivered GBP/USD model over the full window with nothing retrained. All rows use the same weights with different execution settings, so the differences are attributable to the deployment layer alone. The spread is charged in every row. The model was trained with daily decisions and the threshold, so the hourly rows use it outside the setting it learned in.

Frequency	Threshold	Commission	Return	Sharpe	Max drawdown	Trades
Daily	0.20	3.5 USD	+506.6%	0.591	49.2%	2,364
Daily	—	3.5 USD	+264.1%	0.490	50.6%	19,898
Daily	—	none	+384.8%	0.551	48.1%	20,461
Hourly	—	none	+27.8%	0.269	66.9%	19,621
Hourly	—	3.5 USD	−8.7%	0.198	68.8%	18,944
Hourly	0.20	3.5 USD	+66.5%	0.332	64.8%	3,361

Two things follow from that table. First, the decision frequency matters more than any other setting. The same weights, allowed to act every hour instead of once a day, return +27.8% instead of +384.8% before commission, and lose 8.7% once the commission is charged. An hourly decision changes the position on hourly noise, and every change pays the spread and the commission. Second, the rebalance threshold is worth more than the commission costs. Under costs it raises the return from +264.1% to +506.6% and cuts the number of trades from 19,898 to 2,364, with a slightly lower drawdown. The return with the threshold and the commission is even higher than the return without either, because the threshold also saves most of the spread: without it the model paid 19,435 US dollars of spread over the window, and with it 8,898.

4.3.5 Worked Example: One Position From Decision to Settlement

The earlier parts of this chapter define each component separately. This section follows one position through all of them, using a trade of the delivered GBP/USD model. The pair is quoted in US dollars, the account currency, so no conversion is needed and the arithmetic can be checked by hand.

At 01:00 UTC on 15 June 2021 the daily decision key of Equation (4.12) changed, and the agent was asked for an action for the first time that day. It received the 38-value observation of Section 4.2, built from the bar that had just closed, and it returned an action of -0.130 . The model was long 133,000 units at that moment. The action scaled the reference volume of Equation (4.5), and the change from the current position was rounded to the broker's volume step of 1,000 units. The change was a sell of 152,000 units, far above the threshold of Equation (4.13), so one order was sent and the model was left 19,000 units short. In netting mode this single order closed the long position and opened the short one. The sell was filled at the bid quoted at that instant, 1.41116, with the ask five points higher. Twenty-four hours later the key changed again and the action became $+0.962$. A buy order of 150,000 units closed the short position at the ask, 1.40791, and opened a long one of 131,000 units. The profit of the short position follows from the two prices and the volume:

$$19,000 \times (1.41116 - 1.40791) = 61.75 \text{ USD} \quad (4.14)$$

The spread is already included in that figure. The position was sold at the bid and bought back at the ask, and the five points crossed at entry are worth 0.95 USD. The commission is 3.5 USD per standard lot per side. On 0.19 lots this is 0.665 USD, which the broker rounds half up to 0.67 USD per side:

$$2 \times 0.67 = 1.34 \text{ USD} \quad (4.15)$$

The position was held overnight, but the swap is zero because the account is swap-free. The position therefore settled at $61.75 - 1.34 = 60.41$ USD, which is the figure in the exported trade record. The same

day shows the risk rule of the previous section at work. The long position of 131,000 units was still open when the United States Federal Reserve published its rate decision at 18:00 UTC. The hourly bar that held the announcement had a range of 89 pips, several times the range of the bars before it. The average true range rose, the reference volume fell, and at the close of that bar the engine sold 48,000 units. No new action was taken. The position was reduced to the same fraction of a smaller reference size.

One trade says little on its own. Over the full window the delivered GBP/USD model made 2,365 trades in 191 positions, the last one settled at the end of the window. Across all of them the same accounting gives a gross profit of 55,147.61 USD, which already includes 8,899.90 USD of spread, a commission charge of 4,485.87 USD and a swap charge of exactly zero, for a net of 50,661.74 USD. Spread and commission together took 20.9% of the profit before costs, the figure given for this pair in Table 5.8. A result reported without these charges would describe a different strategy.

Everything described here was fixed before any result was seen. There is one architecture, one observation, one reward and one selection rule, and they were applied unchanged to seven instruments. This is what makes the results comparable across pairs.

It also makes the results falsifiable, because they can show the design to be wrong. A design that is tuned on one instrument until it works can always be made to work on that instrument. Its results then cannot tell a real strategy apart from a description of that instrument's past prices. When the same design is applied to seven instruments, a failure on any of them shows in the results. The results in Chapter 5 include a pair on which the design does not work.

5

Experimental Results

Contents

5.1	Evaluation Protocol	50
5.2	Benchmarks and Performance Metrics	50
5.3	Selection: How Many Seeds Pass	51
5.4	Walk-Forward Validation: 2018 to 2024	53
5.5	The Test Year: 2025	53
5.6	The Full Window: 2015 to 2025	55
5.7	Trading Behaviour: How the Agents Actually Trade	57
5.8	Alpha and Beta Decomposition	62
5.9	Robustness: Starting Balance and Risk Level	63
5.10	Portfolio: The Seven Models Combined	65
5.11	Design Iterations: What Did Not Work	67

This chapter reports the results of the procedure of Section 4.3, applied to the seven major pairs. Every figure below comes from a stored run that can be opened again, and every table is generated from those runs by the same script. At the time of writing a model that passes all eleven gates exists for

four of the seven pairs. The other pairs are represented by their provisional model, marked † in every table and figure.

The order of the chapter follows the order in which the evidence should be read. The selection comes first, because it decides how much every later number can claim. Then come the walk-forward years, the test year and the full window, and after them the checks on how the models trade, where their return comes from and how stable it is. The chapter ends with the seven models combined and with the designs that did not work.

5.1 Evaluation Protocol

The account, the cost model and the execution engine are described in Section 4.1 and are not repeated here. Four points are specific to the evaluation.

First, three kinds of window are reported, as defined in Section 4.3. The walk-forward years, 2018 to 2024, are each traded by the model of their own fold. The test year, 2025, is traded by the delivered model. The full window, 2015 to 2025, is also traded by the delivered model, and it contains that model's training years. Only the first two are out-of-sample with respect to training, and neither is out-of-sample with respect to selection.

Second, every window starts with the indicators already warmed up. The slowest indicator in the observation is a moving average over five thousand hourly bars, so the data before each window, back to January 2014, is used only to compute it.

Third, the full window is also measured from five starting balances: 9,900, 10,000, 10,050, 10,100 and 10,200 US dollars. A strategy whose result depends on its starting point is not a real strategy. Small changes to the opening balance change the rounding of every position size, and so they change the whole sequence of trades. The tables give the reference run at 10,000 and, where stated, the mean across the five balances and the dispersion around it.

Fourth, every pair trades at the same reference risk of Equation (4.5), $\rho = 1\%$. The same weights are also run at 0.25% and 0.5% in Section 5.9. Position size scales every trade, but it does not change the policy that chooses the trades. The ratios, the alpha and beta decomposition and the correlation structure are therefore almost unaffected by it.

5.2 Benchmarks and Performance Metrics

The main benchmark is to buy the pair at the start of a window and hold it to the end. This is the fair comparison for a directional strategy on a single instrument. Anyone can do it, and it costs one spread. A strategy that cannot beat it does not justify its extra complexity. The US 500 index is reported beside

it as a second reference, because it is what an investor would hold instead of a currency strategy. The thesis proposal named a random-signal baseline and a moving-average crossover. Both were dropped, because a trader would not hold either of them as a real alternative.

The reported metrics are those defined in Section 2.1.5: total and annualised return, the Sharpe, Sortino, Calmar and Sterling ratios, maximum drawdown and the number of trades. Table 5.1 states what each one measures. All ratios use a risk-free rate of zero, so they can be compared across pairs without any assumption about interest rates.

Table 5.1: Summary of the performance metrics used in this work.

Metric	Description
Net Profit	Total profit or loss
Profit Factor	Ratio of gross profit to gross loss
Commission and Swap	Costs that reduce net profit
Max Balance and Equity Drawdown	Largest losses observed
Total Trades and Winning Trades	How much the strategy trades and how often it wins
Max Consecutive Winning and Losing Trades	How consistent the results are
Largest Winning and Losing Trades	How large single gains or losses can be
Average Trade	Average result per trade, a measure of trading efficiency
Sharpe Ratio and Sortino Ratio	Risk-adjusted return

Return says how much was made, but nothing about the risk taken to make it. The Sharpe ratio divides return by total volatility, so it penalises a strategy for moving up as much as for moving down. The Sortino ratio counts only downside movement. Calmar and Sterling divide by drawdown instead of volatility. Calmar uses the worst loss and Sterling the average one, so they measure how far the account fell, which is the question an investor asks. A strategy can look strong on one of these and weak on another, and where that happens the text points it out.

Every number in this chapter is net of the spread quoted at the moment of each trade and of the commission in Table 4.2. The results therefore cannot be compared with results computed on bar closes with a fixed spread or with no spread, and they are lower than the same policies would show under those conditions.

5.3 Selection: How Many Seeds Pass

A delivered model is one seed out of many, and every number reported for it is conditioned on having been chosen. The first result of this chapter is therefore the selection itself. 240 seeds were trained and evaluated across the seven pairs. Table 5.2 shows how many of them pass each gate of Table 4.5.

The gates remove different seeds, and no single gate explains the result. Most seeds pass the validation gates, the holding gate and the gate that checks the last fold still trades. Fewer pass the

Table 5.2: How many trained seeds pass each gate. Validation, Test and Full each need a positive return and a return above the pair's own over that window; the other five are the behaviour gates of Section 4.3. All counts the seeds that pass every gate. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.

Pair	Seeds	Validation	Test	Full	Sides	Regime	Holds	Sizing	Alive	All
AUD/USD [†]	40	29	6	23	26	11	29	25	29	0
EUR/USD [†]	60	34	4	29	40	24	43	44	43	0
GBP/USD	30	24	4	7	18	14	22	17	22	1
NZD/USD [†]	40	25	12	26	16	8	24	26	21	0
USD/CAD	30	16	7	18	20	14	22	20	20	1
USD/CHF	30	19	4	6	11	4	14	16	13	1
USD/JPY	10	8	5	5	8	7	9	8	9	2
Total	240	155	42	114	139	82	163	156	157	5

regime gate, which asks whether the positions are timed better than the same positions rotated in time. The tightest filter by far is the test year. Only 23.8% of all seeds made money in 2025, and the mean return of all seeds in that year was -13.0% .

This number decides how the test year should be read. 23 seeds pass all nine gates that do not look at 2025. If the test year were an independent check of those seeds, their 2025 results would show how the design does on new data. Only 6 of them made money in 2025, and their mean return in that year was -14.6% . A seed that is good on every other criterion is therefore more likely to lose money in 2025 than to make it. The delivered models are profitable in 2025 because the test gates kept only the seeds that were. Their 2025 figures show that such seeds exist. They do not show that the design earns money in 2025 on average, and the evidence above says it does not.

The same selection also shows why return alone is a poor way to choose a model. On five of the seven pairs, the seed with the highest return over the full window fails at least one gate. On AUD/USD, USD/CAD and USD/CHF it fails the regime gate, with permutation p -values between 0.67 and 0.97. Its positions are timed no better than the same positions shifted in time, and its return comes from following the trend of the pair. A campaign that ranked seeds by return would have delivered these models.

The two training arms of Section 4.3 give similar results. 130 seeds were trained without costs and 2 of them pass every gate. 110 were trained with costs and the rebalance threshold, and 3 of them pass every gate. The mean number of gates passed is 5.7 and 5.9. Charging the costs in training does not make the agent worse, and it may make it slightly better, but the difference is too small to separate from chance at these sample sizes. The later waves of the campaign trained with costs only.

These counts also limit the statistical claims of the chapter. Each delivered model was chosen from between ten and sixty seeds of its pair. With that many draws, a permutation p -value near 0.05 is close to what chance alone gives to the best of them, and the regime p -values of the models that pass every gate lie between 0.02 and 0.05. The permutation test is therefore used here as a screen that rejects bad models. It is not used as a claim that the delivered models are statistically significant.

5.4 Walk-Forward Validation: 2018 to 2024

The walk-forward years are the longest record in this work that no model was trained on. Each year from 2018 to 2024 is traded by the model of its own fold, which was trained on every earlier year since 2015 and never on the year itself. Table 5.3 gives the return of each year, their mean, the seven years chained together, and the pair's own move chained over the same years.

Table 5.3: Walk-forward validation. Each year is scored by the model of its own fold, trained on every year before it from 2015 and never on the year itself; Stitched chains the seven years and Pair chains the pair's own moves over them. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.

Pair	2018	2019	2020	2021	2022	2023	2024	Mean	Stitched	Pair
AUD/USD [†]	−6.3%	+9.8%	+46.5%	+0.7%	−11.7%	+0.4%	+51.2%	+13.0%	+103.6%	−20.9%
EUR/USD [†]	−4.9%	−14.6%	+45.7%	+70.6%	+18.9%	+0.0%	−17.6%	+14.0%	+97.7%	−14.1%
GBP/USD	−1.9%	+0.0%	+3.2%	+18.8%	+6.7%	−13.2%	+69.3%	+11.8%	+88.6%	−7.5%
NZD/USD [†]	−5.6%	+11.7%	−0.6%	+4.9%	+60.0%	+11.7%	+34.3%	+16.6%	+164.1%	−21.4%
USD/CAD	+44.8%	+1.3%	+7.1%	+21.7%	+34.2%	+27.5%	−39.3%	+13.9%	+98.7%	+14.6%
USD/CHF	−36.2%	+39.7%	+66.0%	+3.3%	+43.7%	+0.7%	−38.0%	+11.3%	+37.2%	−6.6%
USD/JPY	−7.1%	−32.0%	+54.9%	−1.7%	+116.6%	+10.5%	+15.5%	+22.4%	+166.0%	+39.3%

The validation years are not free from selection. They chose the checkpoint of each fold, and the validation gates rejected seeds whose mean was negative or whose chained return was below the pair's. The table is therefore a record of models that passed those gates. What it adds is the shape of the result: how the return is spread over the years, and whether it comes from one year or from several.

The return is spread unevenly, and every model has losing years. Across the four models that pass every gate, between four and six of the seven years are positive. Each of them also has at least one year with a large loss: USD/CAD and USD/CHF both lose close to forty percent in 2024, and USD/CHF also lost 36.2% in 2018. A single large year carries much of the chained result of GBP/USD, +69.3% in 2024, and of USD/JPY, +116.6% in 2022. This is the pattern of a trend-following system, which earns most of its return in a few years with strong moves and gives some of it back in the years between them.

Figure 5.1 shows each year against the pair itself. A positive cell is a year in which the model did better than holding the pair. The last column is the test year, traded by the delivered model.

Most cells are positive, and the negative ones do not cluster in one year across pairs. The models that lose against their pair do so in different years. This matters for Section 5.10, because a set of models that lose in different years is safer together than any of them alone.

5.5 The Test Year: 2025

The year 2025 was not used to train any model or to choose any checkpoint. It was used, as Section 5.3 showed, to reject seeds that lost money in it or did worse than their pair. Table 5.4 gives the delivered

AUD/USD †	+3.5	+10.3	+36.7	+6.6	-5.5	+0.6	+60.3	+53.3
EUR/USD †	-0.3	-12.4	+36.7	+77.8	+24.8	-3.1	-11.4	-6.6
GBP/USD	+3.6	-3.9	+0.1	+19.9	+17.3	-18.4	+71.0	+5.7
NZD/USD †	-0.2	+11.4	-7.3	+10.2	+67.1	+12.3	+45.7	-19.7
USD/CAD	+36.3	+6.0	+9.1	+22.3	+27.2	+29.5	-47.8	+4.9
USD/CHF	-36.8	+41.2	+74.4	+0.1	+42.3	+9.6	-45.8	+16.0
USD/JPY	-4.2	-31.0	+59.8	-13.2	+102.7	+2.8	+4.0	+27.2
	2018	2019	2020	2021	2022	2023	2024	2025

Figure 5.1: Yearly return of each model minus the move of its own pair, in percentage points. The years from 2018 to 2024 are traded by the model of their own fold. The last column, after the vertical line, is the test year, traded by the delivered model.

models in 2025, next to 2024, the validation year of their last fold, and next to the full window.

Table 5.4: The test year against the validation year and the full window. 2024 chose which episode's weights the delivered model kept; 2025 was seen by no training and chose between seeds only as a confirmation gate. The last two columns are the move of the pair itself and of the US 500 index in 2025. Pairs marked † are provisional: the best seed so far, which does not yet pass every gate.

Pair	2025 (Test)	2024 (Validation)	Full Window	2025 Market	2025 US500
AUD/USD †	+61.1%	+51.2%	+409.1%	+7.7%	+16.7%
EUR/USD †	+6.8%	-17.6%	+28.0%	+13.5%	+16.7%
GBP/USD	+13.3%	+69.3%	+506.6%	+7.6%	+16.7%
NZD/USD †	-16.9%	+34.3%	+10.0%	+2.8%	+16.7%
USD/CAD	+0.4%	-39.3%	+255.6%	-4.5%	+16.7%
USD/CHF	+3.3%	-38.0%	+270.3%	-12.7%	+16.7%
USD/JPY	+26.6%	+15.5%	+812.9%	-0.6%	+16.7%
Portfolio	+13.5%	—	+327.5%	—	—

The models that pass every gate are all profitable in 2025 and all beat their pair, which follows from the gates. The size of the result differs a lot between pairs. USD/JPY returned +26.6% while its pair was flat, and GBP/USD +13.3% against +7.6% for the pair. USD/CHF returned +3.3% while its pair fell 12.7%, and USD/CAD returned +0.4% while its pair fell 4.5%. The US 500 index rose 16.7% in the same year, so only USD/JPY did better than an investor holding equities.

The comparison with 2024 is informative. For USD/CAD and USD/CHF, 2024 was the worst validation year and 2025 is a small gain. For GBP/USD, 2024 was the best year and 2025 is a moderate gain. A strong validation year is therefore no guide to the next year, and a weak one is no sign that the model has stopped working. This agrees with the pattern of Section 5.4: the return of these models comes in

uneven years.

The test year must be read with the evidence of Section 5.3. Across all trained seeds, the mean return in 2025 was -13.0% , and the seeds that passed every other gate did no better. The figures in Table 5.4 are the result of models that were kept partly because of that year. They show that the design can produce a model that trades 2025 at a profit. They are not an estimate of what a new model from the same design would earn in 2025.

5.6 The Full Window: 2015 to 2025

The full window is traded by the delivered model of each pair, the model of its last fold. That model was trained on 2015 to 2023, so nine of the eleven years are its own training data. The full window is therefore not an estimate of future performance. It is the longest record of how each model trades, and it is the window on which the behaviour gates, the costs and the alpha and beta decomposition are measured. Table 5.5 gives the result for each pair and Table 5.6 compares it with the pair itself and with the US 500 index.

Table 5.5: Result for each pair over 2015 to 2025 under the full costs, from the delivered model of the last fold, which trained on 2015 to 2023. Return columns are the mean, standard deviation and range across the five starting balances; ratios and drawdown are from the reference run at 10,000 US dollars. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.

Pair	Mean	SD	Min	Max	Sharpe	Sortino	Calmar	Max DD	Trades
AUD/USD [†]	409.1%	—	—	—	0.591	0.825	0.272	58.6%	572
EUR/USD [†]	28.0%	—	—	—	0.263	0.374	0.038	59.8%	337
GBP/USD	537.2%	70.6	413.5%	604.8%	0.591	0.843	0.362	49.2%	191
NZD/USD [†]	10.0%	—	—	—	0.215	0.314	0.013	64.5%	431
USD/CAD	251.7%	4.0	246.5%	257.0%	0.486	0.708	0.173	70.4%	449
USD/CHF	252.0%	18.8	219.2%	270.3%	0.497	0.750	0.219	57.8%	423
USD/JPY	757.4%	37.3	700.7%	812.9%	0.694	0.984	0.342	65.0%	703

Table 5.6: Each model against buying and holding its own pair and against the US 500 index from 2015 to 2025, under the same costs. The model is the delivered model of the last fold, which trained on 2015 to 2023. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.

Pair	Market	Model	Excess	US500	Positive Years
AUD/USD [†]	-18.3%	+409.1%	+427.5 pp	+233.3%	8 of 11
EUR/USD [†]	-2.9%	+28.0%	+31.0 pp	+233.3%	5 of 11
GBP/USD	-13.5%	+506.6%	+520.1 pp	+233.3%	9 of 11
NZD/USD [†]	-26.2%	+10.0%	+36.2 pp	+233.3%	6 of 11
USD/CAD	+18.2%	+255.6%	+237.4 pp	+233.3%	8 of 11
USD/CHF	-20.3%	+270.3%	+290.6 pp	+233.3%	7 of 11
USD/JPY	+30.8%	+812.9%	+782.2 pp	+233.3%	9 of 11

Every model finishes the window with a profit, and every model finishes above its own pair. For the four models that pass every gate, the return from the reference balance goes from $+255.6\%$ on

USD/CAD to +812.9% on USD/JPY, and between seven and nine of the eleven years are positive. The pairs themselves did much less. GBP/USD fell 13.5% and USD/CHF 20.3% over the window, while USD/CAD rose 18.2% and USD/JPY 30.8%. Part of each model's return therefore came from a falling market and part from a rising one, and Section 5.8 separates the two.

The returns come with large drawdowns. At the reference risk of 1% per position, the maximum drawdown of the four models is between 49.2% and 70.4%. An account that falls by half needs to double to recover, so these figures matter more than the total return. They are a consequence of the risk level and not of the policy. The same weights at a quarter of the risk keep a similar Sharpe ratio, a little lower, with drawdowns between 12.8% and 24.7%, as Section 5.9 shows. The Sharpe ratios, between 0.49 and 0.69, are the better summary of the policies, because they do not depend on the risk level.

Figures 5.2 to 5.4 plot each model against its pair and against the US 500 index. The shaded bands mark the validation years of the folds, and the last band is the test year.

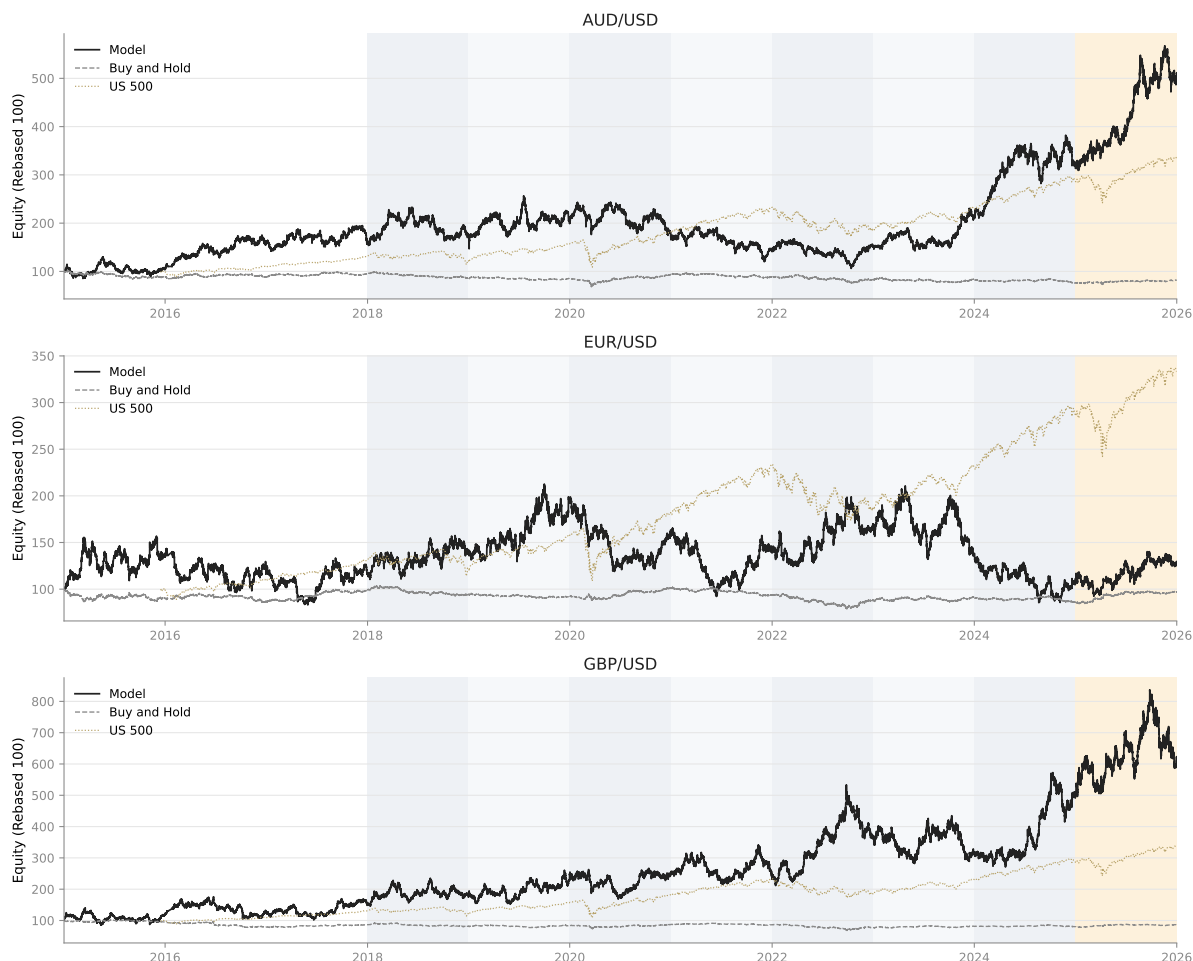


Figure 5.2: AUD/USD, EUR/USD and GBP/USD against buying and holding the same pair and against the US 500 index, rebased to 100. The shaded bands mark the validation years and the test year.

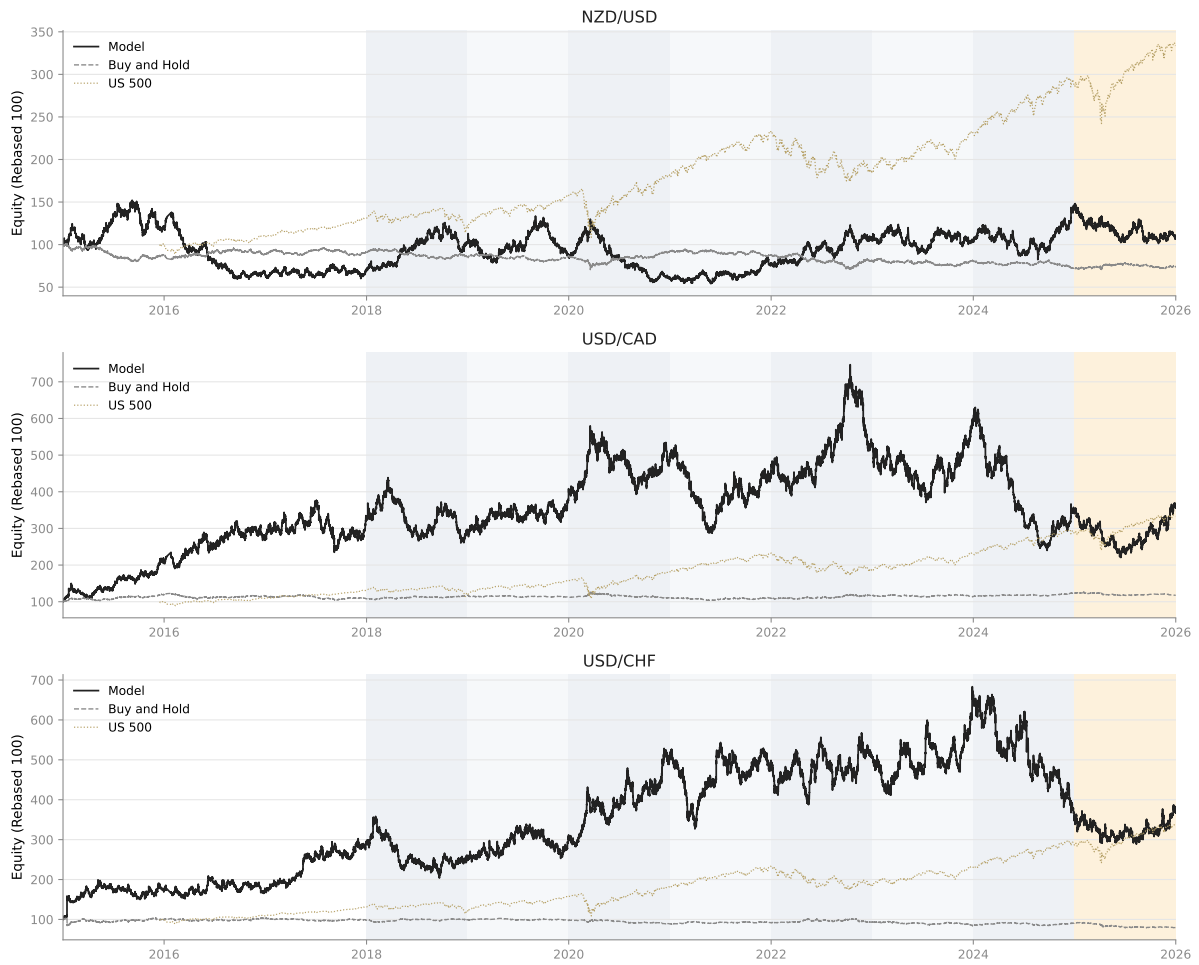


Figure 5.3: NZD/USD, USD/CAD and USD/CHF against buying and holding the same pair and against the US 500 index, rebased to 100.



Figure 5.4: USD/JPY against buying and holding the pair and against the US 500 index, rebased to 100.

5.7 Trading Behaviour: How the Agents Actually Trade

A return figure says whether an agent made money. It says nothing about how, and here that matters for a specific reason. The main claim of the design is that a continuous action lets one decision set both

the direction and the size of a position. This section examines what the agents did with that action, how long they held their positions, and what the trading cost.

Table 5.7: Trading behaviour over the evaluation window. Mean $|a|$ is the average fraction of the reference position held while in the market. Trend is the lookback whose strength the signal tracks best; Side ρ is the rank correlation of the signal with that trend and Size ρ that of its magnitude with the trend's strength. Pairs marked $\dagger$ are provisional: the best seed so far, which does not yet pass every gate.

Pair	Positions	Long	Median Hold	Longest Hold	Win Rate	Mean $ a $	Trend	Side ρ	Size ρ
AUD/USD $\dagger$	572	54.1%	2.0 d	70 d	64.3%	0.76	250 d	-0.36	+0.08
EUR/USD $\dagger$	337	34.1%	1.1 d	180 d	54.3%	0.83	120 d	+0.72	+0.34
GBP/USD	191	52.3%	8.9 d	70 d	58.1%	0.93	60 d	+0.47	+0.10
NZD/USD $\dagger$	431	27.3%	2.9 d	168 d	37.6%	0.90	5 d	+0.60	+0.17
USD/CAD	449	63.9%	2.0 d	105 d	43.9%	0.87	20 d	+0.29	+0.14
USD/CHF	423	27.3%	2.1 d	83 d	37.1%	0.81	5 d	+0.52	+0.14
USD/JPY	703	56.7%	2.1 d	63 d	45.8%	0.81	120 d	+0.50	+0.11

5.7.1 Use of the Continuous Action

Figure 5.5 shows the distribution of the magnitude of the action over the full window for each pair. The action never reaches the ends of its range: on every pair, no daily decision is above $|a| = 0.98$, so the penalty of Equation (4.7) does its job and the output never saturates. But most of the mass lies close to the top of the range. The median magnitude is between 0.87 and 0.96 on every pair, and fewer than one decision in six is below 0.5.

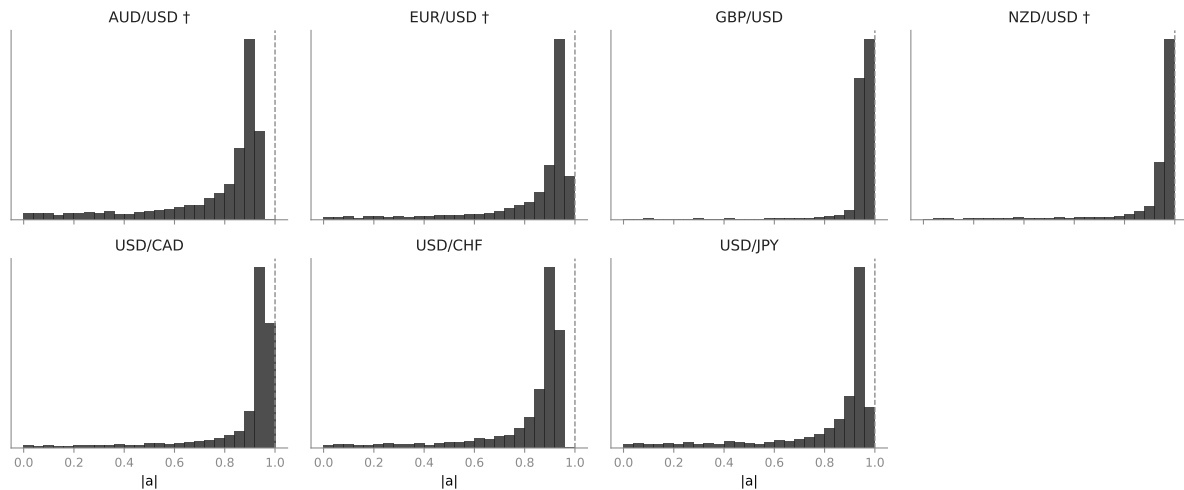


Figure 5.5: Distribution of the magnitude of the daily action over the full window. The dashed line marks full size. No decision reaches it, but most decisions are close to it.

The agents therefore use the action mostly to choose a direction, and they hold that direction at close to full size. The size of the position still varies a lot, but most of that variation comes from the risk rule and not from the action. The reference volume of Equation (4.5) falls when volatility rises and grows with

the balance, and the engine follows it every hour, as the worked example of Section 4.3 showed. The action adds a smaller modulation on top. The sizing gate measures it: the magnitude of the action has a positive rank correlation with the strength of the trend on every delivered model, but the correlation is small, between +0.08 and +0.34.

One cause of this is visible in the reward. As Section 4.2 reported, the clip of Equation (4.6) is reached on about seven hourly steps in ten. On those steps the agent is told the direction of the equity move but not its size, so the signal it learns from is mostly about direction. Learning a finer use of size would need a reward that keeps more of the magnitude, which is discussed in Section 6.3.

5.7.2 Direction, Size and Outcome Together

Figure 5.6 follows the delivered GBP/USD model from 2018 to 2025. It shows the price, the action and the net position that resulted.

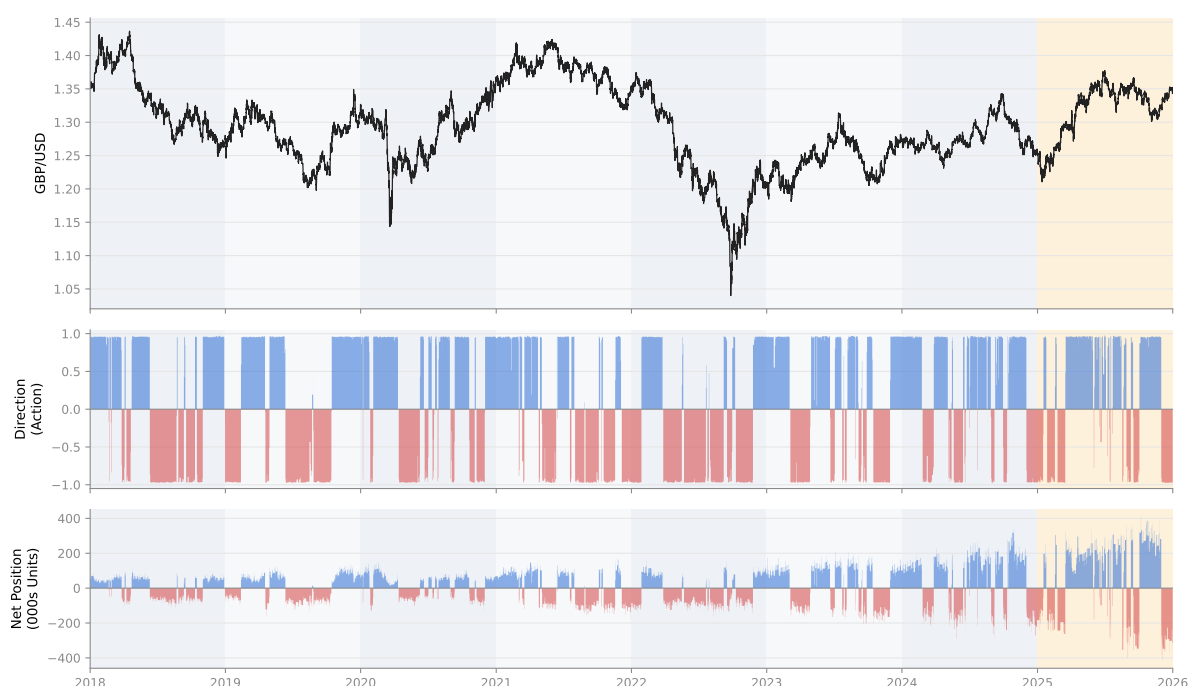


Figure 5.6: GBP/USD from 2018 to 2025 with the delivered model. From top: the price; the action, positive for long and negative for short; and the net position in thousands of units. The shaded bands mark the validation years and the test year.

Three things can be seen. First, the action changes sign in blocks that last weeks to months, and it stays close to one of its two ends within a block. Second, the direction follows the trend of the pair: the model is short through the falls of 2018, 2022 and late 2024, and long through the rises of mid-2020, early 2021 and 2025. The side correlation in Table 5.7 measures this for every pair. On the four models that pass every gate, the sign of the action has a rank correlation of +0.29 to +0.52 with the trend of the

pair, at lookbacks from one week to six months. These are trend-following models. Third, the position grows over the window because the balance grows, and within each block it moves up and down with volatility.

5.7.3 Trend Identification and Reversal

Figure 5.7 shows the most profitable position of the delivered GBP/USD model that lasted at least five days. It falls in the test year. The model had been short while sterling rose from 1.26 to 1.29 in early March 2025. It reversed on 17 March, opened long at 1.293, and held the long position for seventy-seven days while the pair rose to 1.345. The position closed with a net profit of 8,299 US dollars.



Figure 5.7: GBP/USD from March to June 2025. Background shading marks the direction of the net position, red for short and blue for long. The lower panel gives the size of that position. The marked position is the most profitable one of the delivered model that lasted at least five days.

Two details show how the risk rule and the policy work together. In early April the pair fell sharply for a few days, after the United States announced new import tariffs. The action stayed long, but the position was cut from about two hundred thousand to about one hundred thousand units as volatility rose, which limited the loss. As the market calmed and the rise continued, the position was built back up. The model did not change its mind on the direction, and it did not have to: the risk rule reduced the position while the market was most volatile.

5.7.4 Holding Periods and Two-Sided Exposure

Median holding periods are between one and nine days, and the longest single positions last from two to six months. These agents are swing traders, which hold positions for days, and not scalpers. This is what the daily decision frequency of Section 4.3 was meant to produce.

The share of time spent long goes from 27% to 64%. No delivered model trades only one side. This follows from the design: the eligibility test and the sides gate reject any seed whose weaker side is below a threshold, however profitable it is.

5.7.5 Where the Profit Comes From

The win rate is the fraction of positions closed at a profit. On three of the four models that pass every gate it is below one half: 43.9% on USD/CAD, 37.1% on USD/CHF and 45.8% on USD/JPY. GBP/USD wins 58.1%. The profit factor, gross profit divided by gross loss, is between 1.11 and 1.39. So most of these models lose on more positions than they win. They make money because the average winning position earns more than the average losing one loses: between 1.4 and 1.9 times as much on the three models with a win rate below one half. This is the usual profile of trend following, and it matches the side correlations above.

5.7.6 What the Trading Costs

The engine charges the spread quoted at each fill and the commission of Table 4.2, and it records both for every trade. The difference between what the policies earned and what the account kept is therefore measured and not estimated. Table 5.8 gives it for each pair.

Table 5.8: What each model earned before costs and what it kept, in US dollars at 10,000. Cost share is the spread and commission together as a fraction of profit before costs. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.

Pair	Before Costs	Spread	Commission	Net	Cost Share
AUD/USD [†]	52,059	−5,387	−5,758	40,915	21.4%
EUR/USD [†]	8,207	−2,312	−3,090	2,805	65.8%
GBP/USD	64,048	−8,900	−4,486	50,662	20.9%
NZD/USD [†]	7,577	−4,360	−2,217	1,000	86.8%
USD/CAD	48,685	−12,343	−10,780	25,562	47.5%
USD/CHF	51,027	−15,726	−8,271	27,030	47.0%
USD/JPY	108,014	−14,865	−11,857	81,293	24.7%
Total	339,617	−63,893	−46,457	229,267	32.5%

Across the seven models, spread and commission together took about a third of the profit before costs. The share differs a lot between pairs, from about a fifth on GBP/USD and USD/JPY to close to a half on USD/CAD and USD/CHF. The spread is the larger of the two charges on most pairs. This

follows from the way these models trade: they adjust their position often as the reference volume moves, and each adjustment crosses the spread. The rebalance threshold of Equation (4.13) limits this, and Table 4.7 measures by how much.

Table 5.8 has no swap column because the swap is zero on every pair. A swap-free account charges no overnight financing, so zero is the exact figure and not an approximation. This matters at these holding periods, since positions held for days to months would otherwise pay financing on every night.

Every return quoted in this thesis is net of both charges. A study that tested the same policies on bar closes with no spread and no commission would have reported the before-costs column of Table 5.8, and would have described a strategy that no account could have traded.

5.8 Alpha and Beta Decomposition

A currency pair moves on its own. An agent that is long a pair that happens to rise will show a profit that has nothing to do with the policy it learned, and the same is true for an agent that is short a pair that happens to fall. Total return cannot separate this profit from the profit of the policy. The results are therefore decomposed into the part explained by the market and the part that is not.

For each pair, the yearly return of the model is regressed on the yearly move of the pair itself, over the eleven years from 2015 to 2025. With r_i the model return in year i and m_i the market move in the same year, ordinary least squares gives

$$\beta = \frac{\sum_i (m_i - \bar{m})(r_i - \bar{r})}{\sum_i (m_i - \bar{m})^2}, \quad \alpha = \bar{r} - \beta \bar{m} \quad (5.1)$$

β measures the model's exposure to the market's movement. α is the average yearly return that is left once that exposure is removed. The mean return then splits into an active part and a passive part:

$$\bar{r} = \underbrace{\alpha}_{\text{policy}} + \underbrace{\beta \bar{m}}_{\text{market exposure}} \quad (5.2)$$

Yearly returns are used instead of daily returns for a specific reason. The agents hold positions for days to months and act once a day. A daily regression would measure how the model tracks the market within a holding period. It would not measure whether the model chose the right direction over the horizon it trades. Eleven yearly observations are a small sample, and each regression coefficient has a large uncertainty. The full window also contains the training years of the delivered models. The decomposition is therefore good enough to sort the models into groups, but it does not give a precise value for any one of them.

Table 5.9 gives both terms and Figure 5.8 plots them. The four models that pass every gate all have positive alpha, between +13.0% and +23.6% per year. Their market exposure is large. GBP/USD has a

Table 5.9: Yearly regression of each model's return on its pair's over 2015 to 2025. Alpha is the return attributable to the policy and the beta return the part the pair's own movement explains; the two sum to the mean yearly return. The reading is a rule: an absolute beta below 0.5 is a policy with no market exposure. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.

Pair	Beta	Alpha (%/yr)	Beta Return (%/yr)	Mean (%/yr)	Reading
AUD/USD [†]	+0.147	+19.26	−0.24	+19.02	Policy, no market exposure
EUR/USD [†]	−0.646	+4.41	−0.01	+4.39	Policy plus inverse market exposure
GBP/USD	−0.528	+18.80	+0.53	+19.33	Policy plus inverse market exposure
NZD/USD [†]	−3.958	−4.59	+10.11	+5.52	No policy; the model is the market
USD/CAD	+1.359	+14.67	+2.50	+17.16	Policy plus leveraged market exposure
USD/CHF	−3.131	+12.96	+5.56	+18.52	Policy plus leveraged inverse market exposure
USD/JPY	+1.084	+23.55	+2.92	+26.47	Policy plus leveraged market exposure

beta of -0.53 , and the other three have betas above one in magnitude: $+1.36$ on USD/CAD, $+1.08$ on USD/JPY and -3.13 on USD/CHF. A beta of that size means that the yearly return of the model moves more than the pair itself does. It is what a trend-following model at close to full size looks like: in a year when the pair trends, the model rides the trend with leverage, in whichever direction the trend goes.

The sign of the beta shows the direction the model held on average, and the beta return shows how much the average market move added. Here the two parts are unequal. The beta return is between 0.5 and 5.6 percentage points per year on the four models, against a mean yearly return between 17.2% and 26.5%. The market explains at most about a third of the mean return, on USD/CHF, and much less on the other three. Most of the return is therefore attributable to the policy. On average it would survive a market that moved the other way over the decade, although the large betas mean that individual years would look very different.

The decomposition also catches a model with no policy. The provisional NZD/USD model has negative alpha and a beta of about -4 : its whole return is explained by holding the pair short while it fell. It fails only the two test-year gates, and even its regime test is passed. The decomposition shows that it would not deserve to be delivered even if it had passed them.

5.9 Robustness: Starting Balance and Risk Level

Two checks ask whether the full-window results depend on choices that should not matter. The first changes the starting balance. The second changes the risk level.

5.9.1 Starting Balance

Each model that passes every gate was run from the five starting balances of Section 5.1. Table 5.10 gives the individual figures.

It is not obvious why this test is needed, so the reason is given here. A change of one hundred dollars in the opening balance looks like a change of scale. If position sizes were continuous, it would be exactly

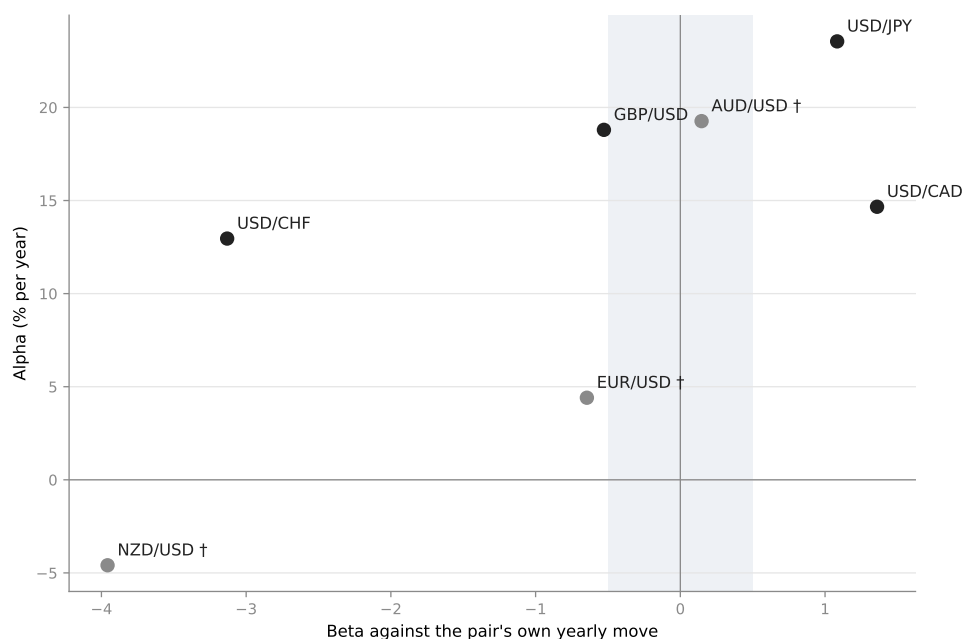


Figure 5.8: Alpha against beta for the seven models. Points inside the shaded band have a beta between -0.5 and $+0.5$, so their return hardly depends on the direction of the market. Provisional models are drawn in grey.

Table 5.10: Total return from each of the five starting balances, in US dollars. Discrete volumes round each balance's positions differently, so each is a different sequence of trades rather than a rescaled copy. Pairs marked $\dagger$ are provisional: the best seed so far, which does not yet pass every gate.

Pair	9,900	10,000	10,050	10,100	10,200	Mean	SD	Relative SD
GBP/USD	604.8%	506.6%	567.3%	413.5%	593.9%	537.2%	70.63	13.1%
USD/CAD	257.0%	255.6%	250.3%	249.3%	246.5%	251.7%	3.95	1.6%
USD/CHF	243.4%	270.3%	262.4%	264.9%	219.2%	252.0%	18.77	7.4%
USD/JPY	778.8%	812.9%	746.4%	700.7%	748.3%	757.4%	37.32	4.9%

that: every trade would be one percent larger, and the percentage return would be the same. Sizes are not continuous. A broker accepts volume only in steps, so the target volume is rounded before an order is sent. A slightly larger balance can round a position up where a smaller one rounds it down. This changes the equity available for the next decision, which changes the next position, and so on. After a few dozen trades, two runs that differ only in their opening balance hold different positions at different times.

Every model stays profitable on all five paths, and its returns are close together. The relative standard deviation, the standard deviation divided by the mean, is between 1.6% on USD/CAD and 13.1% on GBP/USD. No path turns a profit into a loss: the worst path of each model still returns more than +219%. The results therefore do not depend on one particular sequence of roundings.

This test has limits. It changes the starting path, not the training run. Each delivered model is one seed out of many, and the variation across seeds, discussed in Section 5.3, is a different and much

larger source of uncertainty. A result can be fully stable across starting paths and still come from a lucky seed.

5.9.2 Risk Level

The reference risk ρ of Equation (4.5) scales every position. The exposure feature of the observation is measured as a fraction of the reference volume, so the policy sees the same inputs at any risk level, and only the size of its positions changes. Table 5.11 runs the same weights at a quarter, a half and the full reference risk.

Table 5.11: The same weights at three risk levels. The exposure feature is normalised by the sizer, so the policy acts identically and only the position size scales; drawdown scales with it while the ratios stay close. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.

Pair	0.25% Return	Max DD	Sharpe	0.5% Return	Max DD	Sharpe	1% Return	Max DD	Sharpe
AUD/USD [†]	—	—	—	—	—	—	—	—	—
EUR/USD [†]	—	—	—	—	—	—	—	—	—
GBP/USD	+69.7%	12.8%	0.505	+170.1%	27.3%	0.519	+506.6%	49.2%	0.591
NZD/USD [†]	—	—	—	—	—	—	—	—	—
USD/CAD	+52.3%	24.7%	0.433	+129.9%	42.8%	0.473	+255.6%	70.4%	0.486
USD/CHF	+48.0%	18.3%	0.418	+102.2%	33.6%	0.427	+270.3%	57.8%	0.497
USD/JPY	+96.7%	20.9%	0.655	+246.3%	39.6%	0.652	+812.9%	65.0%	0.694

Drawdown scales roughly in proportion to the risk. At a quarter of the risk, the maximum drawdown of the four models is between 12.8% and 24.7%, against 49.2% to 70.4% at the reference risk. Return grows faster than in proportion, because gains compound on a larger balance. The Sharpe ratio changes little, and is slightly higher at the full risk. The risk level is therefore a choice of how much drawdown an account accepts, and not a property of the learned policy. The full reference risk was used in training and selection, and it gives drawdowns that most investors would not accept. A quarter or a half of it gives the same policies at an acceptable level of risk.

5.10 Portfolio: The Seven Models Combined

A trader would not run just one of these agents. They would run all seven. For a portfolio, what matters is not only what each component returns, but also whether the components lose money at the same time.

The seven models are combined by giving each one a seventh of the capital at the start of the window, with no rebalancing afterwards. With $E_p(t)$ the equity of pair p at time t , the portfolio is

$$V(t) = \frac{1}{7} \sum_p \frac{E_p(t)}{E_p(0)} \quad (5.3)$$

By construction, the total return of such a portfolio is the mean of its components. The interesting part is therefore the risk, and not the return. Table 5.12 compares the portfolio with the average of the seven models taken separately.

Table 5.12: The equal-weight portfolio of the models, each from 10,000, against the average of its components. Return is identical by construction, so the effect is on risk. Pairs marked [†] are provisional: the best seed so far, which does not yet pass every gate.

Measure	Portfolio	Mean of the 7	Effect
Total return	+326.9%	+326.9%	Equal by construction Compounding of the mean
Annualised return	+14.10%	+11.98%	
Maximum drawdown	26.3%	58.9%	Diversification
Sharpe ratio	0.964	0.496	
Sortino ratio	1.396	0.725	
Calmar ratio	0.535	0.212	
Sterling ratio	2.034	0.590	
Mean correlation	−0.002 across 21 pairings		Daily returns

Figure 5.9 plots the portfolio against its components. The maximum drawdown falls from a mean of **[mean DD]** for the seven models to **[portfolio DD]** for the portfolio, and every risk-adjusted ratio in Table 5.12 is about twice as high or more. The portfolio gives the same return with less than half of the worst loss.

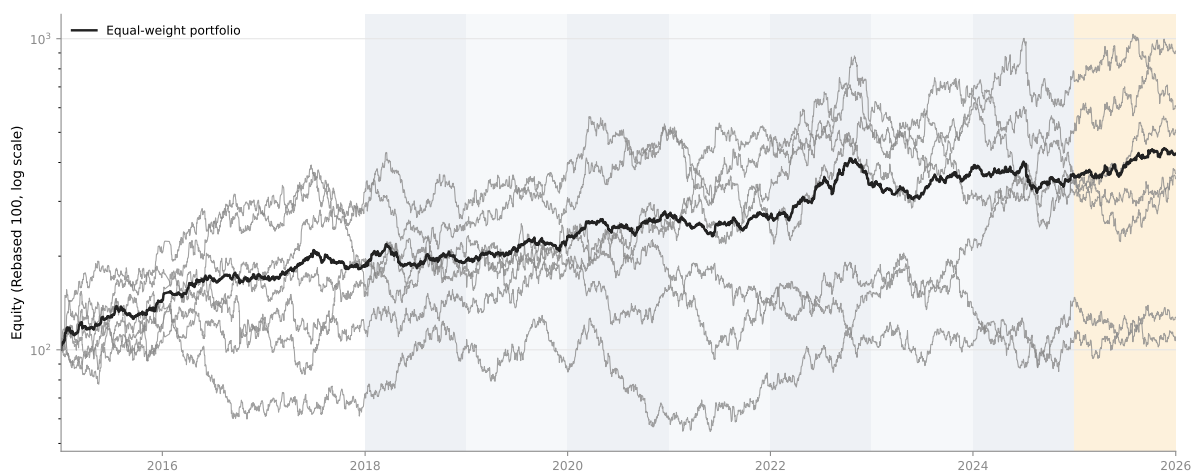


Figure 5.9: The equal-weight portfolio against its seven components, all rebased to 100, on a logarithmic scale. The portfolio is smoother than any component because the components do not fall together.

Figure 5.10 shows the reason. The mean correlation between the daily returns of two models is **[correlation]**, so on average the models are uncorrelated. This is not an obvious result. The seven models were trained by the same procedure on seven instruments that are themselves correlated, since all of them have the US dollar on one side. Even so, their profits and losses come at different times. Each model learned the timing of its own market, so the models share a method but not a position.

Two limits apply to this result. The portfolio is measured on the full window, which contains the

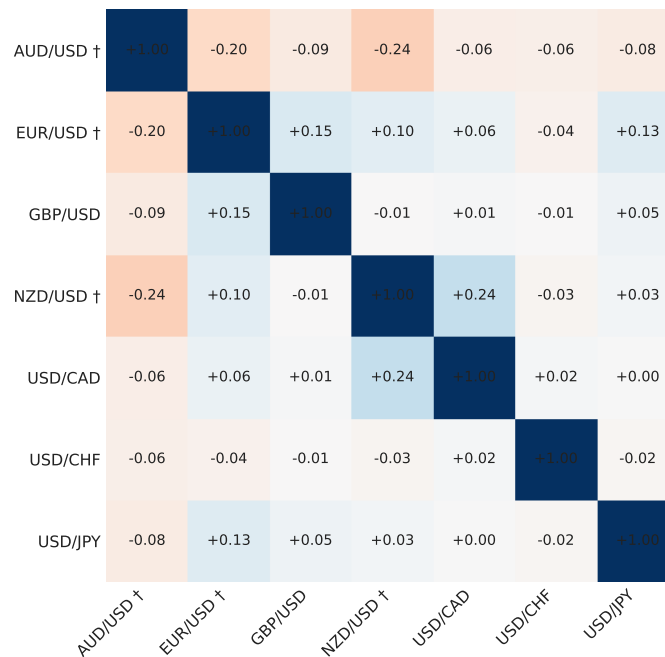


Figure 5.10: Correlation between the daily returns of the seven models over the full window.

training years of every delivered model, so it shares the in-sample character of Section 5.6. And it inherits the selection of its components: in the test year its return is the mean of theirs, **[2025 mean]**, which the test gates of Section 5.3 helped to make positive. The main finding is less affected by these limits. The correlation between the models depends on when they hold their positions, not on how much they earn, and choosing seeds by their gates does not make them uncorrelated. Models trained by one procedure on the seven majors lose money at different times, and that alone makes their combination much safer than any of them.

5.11 Design Iterations: What Did Not Work

The procedure of Section 4.3 is the last of several. The earlier ones failed in ways that explain the final design, so they are reported here.

The first design used a single split: nine years of training, one year of validation and one year of testing, with the seed chosen on its result over the whole window. Its held-out year was negative on five of the seven pairs. The seed choice looked at the same eleven years that the results were then reported on, so the reported results carried the effect of that choice, and the one year that nothing had looked at showed it. This led to the walk-forward validation of the final design, where every year from 2018 to 2024 is traded by a model that never trained on it.

The second design used rolling folds of three years, where each fold continued from the weights of

the fold before it. Its policies collapsed to one side of the market. On GBP/USD, 39 of 40 seeds held a long share between 20% and 80% in the validation year of their first fold, and only 2 of 40 still did so in their last fold. By then the policy was long, or short, for 90% to 100% of every year. Each fold started from a policy already chosen on the previous years, so a bias towards one side was carried forward and grew. The anchored folds of the final design train every fold from scratch, and the collapse did not return: on EUR/USD, nine of the first ten anchored seeds held their weaker side for between 20% and 49% of the time.

The third question was whether the agent should learn with costs or without them. Learning without costs gives a cleaner reward, while learning with them teaches the agent what its trades cost. The two arms of Section 5.3 gave similar results overall. On GBP/USD, the arm with costs produced a seed that passes every gate, where twenty seeds without costs passed at most eight gates, and the same wave produced the models of USD/CAD and USD/CHF. The model of USD/JPY comes from the arm without costs. The later waves trained with costs only.

Two further designs are described in Chapter 4 and are not repeated here: a scale-invariant reward, which made the agent shrink its positions to about three percent of the size available, and hourly decisions, which traded on noise and lost money under costs.

The results support two claims and limit a third. The design produces models that pass every gate on several pairs, and seven models trained by one procedure lose money at different times, so their combination is much safer than any one of them. The test year, however, cannot be read as an unbiased estimate, because it was used to reject seeds. Chapter 6 sets out what these results allow this work to claim and what they do not.

6

Conclusion

Contents

6.1 Summary	69
6.2 Limitations	70
6.3 Future Work	71

The aim of this work was to find out whether one deep reinforcement learning design gives a profitable policy with controlled risk on all seven major currency pairs. The design was trained separately for each pair and was otherwise not changed, and it was charged the same costs that a real account pays.

The answer is partly yes. A model that passes every check was found for four of the seven pairs, and the seven models together form a portfolio with much lower risk than any of them. The conditions attached to this answer are as much a part of the result as the answer itself.

6.1 Summary

This work built three things and answered one question.

The first is a trading framework. In it, one strategy implementation runs in three places: an offline backtest over stored ticks, the cTrader backtesting tab, and a live account. Its execution engine steps

through every tick instead of only the bar closes, and it charges the spread recorded at the moment of each trade. Against cTrader's own backtester it opens every position at the same time and on the same side, and it closes between 93% and 99.6% of them at the same time. The framework reads from a database of about 1.6 billion ticks for the seven pairs, downloaded from the broker's servers, and every bar used in this work is built from those ticks.

The second is an agent. It observes thirty-eight dimensionless values, built from sixteen technical indicators, and outputs one continuous action that sets both the direction and the size of its position. The published algorithm needed several changes to train on this problem, and two of them matter most. The first is a scale-sensitive reward. A scale-invariant reward gives no gradient towards holding a position large enough to matter. The second is a penalty on the actor's pre-activation. Without it, the policy saturates at +1 or -1 and collapses to a constant action.

The third is an evaluation procedure. Each pair is trained with an anchored walk-forward, so every year from 2018 to 2024 is traded by a model that never trained on it, and 2025 is held out from training. A seed is kept only if it passes eleven gates on return and on behaviour, and among the seeds that pass, a composite rank chooses one. The procedure was built because ranking on return does not work, and Section 5.3 shows this with measured results.

The answer to the question is as follows. A model that passes every gate was found for four of the seven pairs. These models are profitable in the walk-forward years taken together, in the test year and over the full window, and they do better than their own pair in each. Their alpha is positive, and most of their return is attributable to the policy, although their exposure to the market is large. An equal-weight portfolio of the seven lowers the maximum drawdown from a mean of **[mean DD]** to **[portfolio DD]**, because the models are close to uncorrelated with one another.

Three findings lie behind these numbers, and they are more likely than the numbers themselves to carry over to other problems. First, the agents use the continuous action mostly to choose a direction, and they hold that direction at close to full size. The size of their positions still varies a lot, but most of that variation comes from the volatility-based risk rule, not from the action. Second, most of the models lose on more positions than they win, and they make money because their winning positions are larger than their losing ones. This is the profile of trend following. Third, the seed with the highest return fails at least one gate on five of the seven pairs, often because its positions are timed no better than chance. Choosing a model by its return alone picks a model that just follows the market.

6.2 Limitations

The most important limitation comes first, because it puts a limit on everything else. No window in this work is free from selection. The validation years chose the checkpoint of each fold and rejected seeds.

The test year was never used for training or for checkpoints, but it was used to reject seeds that lost money in it. Section 5.3 measured what this means. Across all trained seeds, the mean return in 2025 was -13.0% , and the seeds that passed every other gate did no better, at -14.6% . The 2025 figures of the delivered models show that the design can produce a model that trades that year at a profit. They are not an estimate of what a new model from the same design would earn in it.

The second limitation is related. Each delivered model was chosen from between ten and sixty seeds of its pair, and the selection is not corrected for multiple testing. With that many draws, a permutation p -value near 0.05 is close to what chance alone produces. The permutation test is therefore reported as a filter that rejects bad models, and not as a claim that the models that passed are statistically significant.

The third concerns the full window. It contains nine training years of every delivered model, so its returns, ratios and alpha are in-sample. They describe how the models trade over the longest record available. They do not describe how they would trade in the future.

Four smaller limitations should also be recorded. First, at the reference risk of 1% per position the drawdowns are large, between about a half and two thirds of the account. A lower risk level gives the same policies at an acceptable drawdown, but the models were trained and selected at the reference level. Second, the reward clip is reached on about seven hourly steps in ten, so the agent learns mostly the direction of each equity move and little about its size. This limits what the continuous action can learn. Third, training cannot be reproduced across software environments. It is bit-exact within one installation on a single thread, but a different version of a numerical library changes the outcome. The delivered models are therefore archived artifacts whose replay has been verified, not a recipe that produces the same models again. Fourth, two weeks of January 2016 are missing from all seven pairs, for reasons on the broker's side. The gap is the same for all pairs and lies in the training years only, so it does not bias any comparison in this work.

Finally, the tests cover a narrower scope than the method itself. They use seven instruments from one asset class, one eleven-year period, one decision frequency and one cost model. Nothing in this work shows that the design carries over to other markets, other periods or other holding horizons.

6.3 Future Work

The limitations above set the order of the future work. The most useful next step is the one that would most change the value of the results.

The first step is a true held-out evaluation. The delivered models would be fixed, and then tradeded on a period that no part of the procedure has seen, neither training nor checkpoints nor gates. The figures of that period would be the first out-of-sample estimate of the design in the strict sense. The framework already supports this, since it keeps the market data current and runs the same models on new data

without any change.

The second step is to choose seeds without the test year. The gates would use only the walk-forward years and the behaviour checks, and the test year would then be reported for every seed that passes them, not only for the ones that were kept. This costs nothing in the framework and would turn the test year into an unbiased estimate, at the price of reporting a lower number.

The third step is a reward that keeps more of the size of each equity move. The clip used here is reached on most hourly steps, so the agent learns mostly direction. A smaller scale factor, a wider clip, or a reward computed over the whole day before clipping would give the critic a signal about magnitude. This is the most direct way to test whether the continuous action can learn position sizing on its own, beyond what the risk rule already does.

Apart from the evaluation and the reward, what the agents did suggests three further directions. The portfolio in Section 5.10 gives the seven models equal weights. Weighting them by risk, or by their correlation structure, should do better, since the models are close to independent of each other. The observation holds thirty-eight values from a single point in time, and a recurrent or attention-based encoder that reads a window of past data directly is the natural next step. This was part of the original plan for this work. Last, the framework already runs the same strategy on a live account. A live deployment over a long period would test whether the execution assumptions hold when the orders are real, which no backtest can do.

Some reported returns suggest that deep reinforcement learning for currency trading is a solved problem. Its out-of-sample record suggests that it leads nowhere. Neither view is correct. This work shows that the main difficulty is not in the learning algorithm. It is in the environment the agent is trained in, in the costs it is charged, and most of all in deciding which of many trained models to trust. The test year made the size of that last problem visible: the seeds that passed every other check lost money on average in 2025, and only a filter on that same year kept the ones that did not.

Bibliography

- [1] R. Teo, "Candlestick Pattern OHLC." [Online]. Available: <https://www.tradingwithrayner.com/wp-content/uploads/2019/04/candlestick-pattern-ohlc.png>
- [2] R. Pramoditha, "Human Neuron vs Artificial Neuron." [Online]. Available: https://miro.medium.com/v2/resize:fit:2902/format:webp/1*hkYITODpjJgo32DoCOWN5w.png
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, Nov. 2018, google-Books-ID: sWV0DwAAQBAJ.
- [4] P. Brandimarte, *An Introduction to Financial Markets: A Quantitative Approach*. John Wiley & Sons, Feb. 2018, google-Books-ID: _2tODwAAQBAJ.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, Nov. 2016, google-Books-ID: omivDQAAQBAJ.
- [6] T. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *CoRR*, Sep. 2015. [Online]. Available: <https://www.semanticscholar.org/paper/Continuous-control-with-deep-reinforcement-learning-Lillicrap-Hunt/024006d4c2a89f7acacc6e4438d156525b60a98f>
- [7] S. Jansen, *Machine Learning for Algorithmic Trading*. Packt Publishing Ltd, Jul. 2020, google-Books-ID: 4f30DwAAQBAJ.
- [8] BIS, "OTC foreign exchange turnover in April 2025," Sep. 2025. [Online]. Available: <https://www.bis.org/publications/202509-commentary-otc-derivatives>
- [9] E. F. Fama and B. G. Malkiel, "EFFICIENT CAPITAL MARKETS: A REVIEW OF THEORY AND EMPIRICAL WORK*," *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, May 1970. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1111/j.1540-6261.1970.tb00518.x>
- [10] E. P. Chan, *Quantitative Trading: How to Build Your Own Algorithmic Trading Business*. John Wiley & Sons, Jun. 2021, google-Books-ID: pWE0EAAAQBAJ.
- [11] B. Donnelly, *The Art of Currency Trading: A Professional's Guide to the Foreign Exchange Market*. John Wiley & Sons, Jun. 2019, google-Books-ID: r8ybDwAAQBAJ.

- [12] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. [Online]. Available: <https://www.nature.com/articles/nature14236>
- [13] T. G. Fischer, "Reinforcement learning in financial markets - a survey," *Friedrich-Alexander-Universität Erlangen-Nürnberg, Institute for Economics*, 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Reinforcement-learning-in-financial-markets-a-Fischer/922864ede84bc49be4ac676951278a9b568b6383>
- [14] B. Hambly, R. Xu, and H. Yang, "Recent advances in reinforcement learning in finance," *Mathematical Finance*, vol. 33, no. 3, pp. 437–503, Jul. 2023. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1111/mafi.12382>
- [15] T.-V. Pricope, "Deep Reinforcement Learning in Quantitative Algorithmic Trading: A Review," *ArXiv*, May 2021. [Online]. Available: <https://www.semanticscholar.org/paper/Deep-Reinforcement-Learning-in-Quantitative-A-Pricope/a6a7e7be402fc5f7703e5c74a7b2afa84613a429>
- [16] M. M. López de Prado, *Advances in financial machine learning*. Hoboken, New Jersey: Wiley, 2018.
- [17] A. Alameer, H. Saleh, and K. Alshehri, "Reinforcement Learning in Quantitative Trading: A Survey," *TechRxiv*, Mar. 2022. [Online]. Available: <https://www.techrxiv.org/doi/full/10.36227/techrxiv.19303853.v1>
- [18] J. Carapuço, R. Neves, and N. Horta, "Reinforcement learning applied to Forex trading," *Applied Soft Computing*, vol. 73, pp. 783–794, Dec. 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1568494618305349>
- [19] C.-Y. Huang, "Financial Trading as a Game: A Deep Reinforcement Learning Approach," *ArXiv*, Jul. 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Financial-Trading-as-a-Game%3A-A-Deep-Reinforcement-Huang/a13a070940d48bce111f31ba9c0233884a17ae62>
- [20] F. Rundo, "Deep LSTM with Reinforcement Learning Layer for Financial Trend Prediction in FX High Frequency Trading Systems," *Applied Sciences*, vol. 9, no. 20, p. 4460, Oct. 2019. [Online]. Available: <https://www.mdpi.com/2076-3417/9/20/4460>
- [21] D. H. Bailey, J. M. Borwein, M. López De Prado, and Q. J. Zhu, "Pseudo-Mathematics and Financial Charlatanism: The Effects of Backtest Overfitting on Out-of-Sample Performance,"

- Notices of the American Mathematical Society*, vol. 61, no. 5, p. 458, May 2014. [Online]. Available: <https://www.ams.org/jourcgi/jour-getitem?pii=noti1105>
- [22] “Forex & Algorithmic Trading Community | cTrader Community.” [Online]. Available: <https://ctrader.com/>
- [23] W. F. Sharpe, “The Sharpe Ratio,” *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, Oct. 1994. [Online]. Available: <http://pm-research.com/lookup/doi/10.3905/jpm.1994.409501>
- [24] F. A. Sortino and L. N. Price, “Performance Measurement in a Downside Risk Framework,” *The Journal of Investing*, vol. 3, no. 3, pp. 59–64, Aug. 1994. [Online]. Available: <http://pm-research.com/lookup/doi/10.3905/joi.3.3.59>
- [25] C. R. Bacon, *Practical portfolio performance measurement and attribution*, third edition ed., ser. Wiley finance. Hoboken, NJ: Wiley, 2023.
- [26] R. Rhoads, *Candlestick Charting For Dummies*. John Wiley & Sons, Jun. 2022, google-Books-ID: 92l6EAAAQBAJ.
- [27] M. A. Lim, *The Handbook of Technical Analysis + Test Bank: The Practitioner’s Comprehensive Guide to Technical Analysis*. John Wiley & Sons, Dec. 2015, google-Books-ID: N01_BwAAQBAJ.
- [28] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997, google-Books-ID: EoYBngEACAAJ.
- [29] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain.” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. [Online]. Available: <https://doi.apa.org/doi/10.1037/h0042519>
- [30] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986. [Online]. Available: <https://www.nature.com/articles/323533a0>
- [31] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, vol. 2, no. 5, pp. 359–366, Jan. 1989. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/0893608089900208>
- [32] B. F. Skinner, “Operant behavior,” *American Psychologist*, vol. 18, no. 8, pp. 503–515, 1963, place: US Publisher: American Psychological Association.
- [33] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, “Playing Atari with Deep Reinforcement Learning,” *ArXiv*, Dec. 2013. [Online]. Available: <https://www>

[semanticscholar.org/paper/Playing-Atari-with-Deep-Reinforcement-Learning-Mnih-Kavukcuoglu/2319a491378867c7049b3da055c5df60e1671158](https://www.semanticscholar.org/paper/Playing-Atari-with-Deep-Reinforcement-Learning-Mnih-Kavukcuoglu/2319a491378867c7049b3da055c5df60e1671158)

- [34] M. Riedmiller, “Neural Fitted Q Iteration – First Experiences with a Data Efficient Neural Reinforcement Learning Method,” *Machine Learning: ECML 2005*, vol. 3720, pp. 317–328, 2005, series Title: Lecture Notes in Computer Science. [Online]. Available: http://link.springer.com/10.1007/11564096_32
- [35] H. Van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-Learning,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, Mar. 2016, iSSN: 2374-3468, 2159-5399 Issue: 1 Journal Abbreviation: AAAI. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/10295>
- [36] T. L. Meng and M. Khushi, “Reinforcement Learning in Financial Markets,” *Data*, vol. 4, no. 3, p. 110, Jul. 2019. [Online]. Available: <https://www.mdpi.com/2306-5729/4/3/110>
- [37] J. Sim and B. Kirk, “Generalized Deep Reinforcement Learning for Trading,” *Journal of Student Research*, vol. 12, no. 1, Feb. 2023. [Online]. Available: <https://www.jsr.org/hs/index.php/path/article/view/4316>
- [38] T. Théate and D. Ernst, “An application of deep reinforcement learning to algorithmic trading,” *Expert Systems with Applications*, vol. 173, p. 114632, Jul. 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417421000737>
- [39] Y. Li, W. Zheng, and Z. Zheng, “Deep Robust Reinforcement Learning for Practical Algorithmic Trading,” *IEEE Access*, vol. 7, pp. 108 014–108 022, 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8786132/>
- [40] Z. Xiong, X.-Y. Liu, S. Zhong, H. Yang, and A. Elwalid, “Practical Deep Reinforcement Learning Approach for Stock Trading,” *ArXiv*, Nov. 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Practical-Deep-Reinforcement-Learning-Approach-for-Xiong-Liu/412fbb7ffa29aa6b2e172c1f545223baf18f1b6e>
- [41] N. Yousefi, “Deep Reinforcement Learning for Tehran Stock Trading,” *Journal of Novel Engineering Science and Technology*, vol. 1, no. 02, pp. 37–42, Nov. 2022. [Online]. Available: <https://journal.iistr.org/index.php/JNEST/article/view/171>
- [42] P. K. Gran, A. J. K. Holm, and S. Søgård, “A Deep Reinforcement Learning Approach to Stock Trading,” *NTNU*, 2019. [Online]. Available: <https://www.semanticscholar.org/paper/A-Deep-Reinforcement-Learning-Approach-to-Stock-Gran-Holm/ef9f709e845d282cfb9371708d05d61580d1e10c>

- [43] A. R. Azhikodan, A. G. K. Bhat, and M. V. Jadhav, "Stock Trading Bot Using Deep Reinforcement Learning," *Springer*, vol. 32, pp. 41–49, 2019, book Title: Innovations in Computer Science and Engineering Series Title: Lecture Notes in Networks and Systems. [Online]. Available: http://link.springer.com/10.1007/978-981-10-8201-6_5
- [44] N. Majidi, M. Shamsi, and F. Marvasti, "Algorithmic Trading Using Continuous Action Space Deep Reinforcement Learning," *Expert Systems with Applications*, 2022, publisher: arXiv Version Number: 1. [Online]. Available: <https://arxiv.org/abs/2210.03469>
- [45] H. K. Sagiraju and S. Mogalla, "Application of multilayer perceptron to deep reinforcement learning for stock market trading and analysis," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 24, no. 3, p. 1759, Dec. 2021. [Online]. Available: <http://ijeecs.iaescore.com/index.php/IJECS/article/view/24816>
- [46] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, "Deep reinforcement learning for automated stock trading: an ensemble strategy," *Proceedings of the First ACM International Conference on AI in Finance*, pp. 1–8, Oct. 2020, conference Name: ICAIF '20: ACM International Conference on AI in Finance ISBN: 9781450375849 Place: New York New York Publisher: ACM. [Online]. Available: <https://dl.acm.org/doi/10.1145/3383455.3422540>
- [47] M. Kong and J. So, "Empirical Analysis of Automated Stock Trading Using Deep Reinforcement Learning," *Applied Sciences*, vol. 13, no. 1, p. 633, Jan. 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/1/633>
- [48] X. Wu, H. Chen, J. Wang, L. Troiano, V. Loia, and H. Fujita, "Adaptive stock trading strategies with deep reinforcement learning methods," *Information Sciences*, vol. 538, pp. 142–158, Oct. 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025520304692>
- [49] L. Chen and Q. Gao, "Application of Deep Reinforcement Learning on Automated Stock Trading," *2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS)*, pp. 29–33, Oct. 2019, conference Name: 2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS) ISBN: 9781728109459 Place: Beijing, China Publisher: IEEE. [Online]. Available: <https://ieeexplore.ieee.org/document/9040728/>
- [50] Z. Zhang, S. Zohren, and S. Roberts, "Deep Reinforcement Learning for Trading," *The Journal of Financial Data Science*, vol. 2, pp. 25–40, Apr. 2020, iSSN: 2640-3943 Issue: 2 Journal Abbreviation: JFDS. [Online]. Available: <http://pm-research.com/lookup/doi/10.3905/jfds.2020.1.030>
- [51] Z. Jia, Q. Gao, and X. Peng, "LSTM-DDPG for Trading with Variable Positions," *Sensors*, vol. 21, no. 19, p. 6571, Sep. 2021. [Online]. Available: <https://www.mdpi.com/1424-8220/21/19/6571>



Technical Indicators

Table A.1: Summary of frequently used Trend Technical Indicators [7].

Indicator Name	Formula	Range
Simple Moving Average	$SMA(N)_t = \frac{1}{N} \sum_{i=1}^N Price_{t-N+i}$	-
Exponential Moving Average	$EMA(N)_t = \frac{2}{N+1} \times Price_t + (1 - \frac{2}{N+1}) \times EMA(N)_{t-1}$	-
Weighted Moving Average	$WMA(N)_t = \frac{2}{N \times (N+1)} \sum_{i=1}^N i \times Price_{t-N+i}$	-
Double Exponential Moving Average	$DEMA(N)_t = 2 \times EMA(N)_t - EMA_2(N)_t$	-
Triple Exponential Moving Average	$TEMA(N)_t = 3 \times [EMA(N)_t - EMA_2(N)_t] + EMA_3(N)_t$	-
Hull Moving Average	$HMA(N)_t = WMA\left(\sqrt{N}\right) \left[2 \times WMA\left(\frac{N}{2}\right)_t - WMA(N)_t \right]$	-
Triangular Moving Average	$TRIMA(N)_t = \frac{1}{N} \sum_{i=1}^N SMA(N)_{t-N+i}$	-
Efficiency Ratio	$ER(N)_t = \frac{ Price_t - Price_{t-N} }{\sum_{i=1}^N Price_{t-i+1} - Price_{t-i} }$	[0, 1]
Kaufman Adaptive Moving Average	$KAMA(N)_t = KAMA(N)_{t-1} + SC \times (Price_t - KAMA(N)_{t-1})$ $SC = [ER(N)_t \times (\phi_{short} - \phi_{long}) + \phi_{long}]^2, \phi_{short} = \frac{2}{N_{short}+1}, \phi_{long} = \frac{2}{N_{long}+1}$ where typically $N_{short} = 2$ and $N_{long} = 30$	-

Table A.2: Summary of frequently used Momentum Technical Indicators [7].

Indicator Name	Formula	Range
Price Momentum	$PM_t = \frac{Price_t - Price_{t-1}}{Price_{t-1}}$	-
Rate of Change	$ROC(N)_t = \ln \left(\frac{Price_t}{Price_{t-N}} \right)$	-
Price Acceleration	$PA_t = \frac{\text{Short Linear Regression Coefficient}}{\text{Long Linear Regression Coefficient}} = \frac{\beta_{\text{short}}}{\beta_{\text{long}}}$	-
Relative Strength Index	$RSI(N)_t = 100 - \frac{100}{1 + \frac{\text{Average Gain}_t}{\text{Average Loss}_t}}$ $\text{Average Gain}_t = \frac{1}{N} \sum_{i=1}^N \max(Price_i - Price_{i-1}, 0)$ $\text{Average Loss}_t = \frac{1}{N} \sum_{i=1}^N \max(Price_{i-1} - Price_i, 0)$	[0, 100]
Aroon Oscillator	$Aroon(N)_t = Aroon\ Up(N)_t - Aroon\ Down(N)_t$ $Aroon\ Up(N)_t = \left(\frac{N - \text{Periods Since Highest High in } N}{N} \right) \times 100$ $Aroon\ Down(N)_t = \left(\frac{N - \text{Periods Since Lowest Low in } N}{N} \right) \times 100$	[-100, 100]
Balance of Power	$BOP_t = \frac{Close_t - Open_t}{High_t - Low_t}$	[-1, 1]
Commodity Channel Index	$CCI(N)_t = \frac{TP_t - SMA_{TP}(N)_t}{0.015 \times \text{Mean Deviation}(N)_t}$ $TP_t = \frac{High_t + Low_t + Close_t}{3}$ $\text{Mean Deviation}(N)_t = \frac{1}{N} \sum_{i=1}^N TP_i - SMA_{TP}(N)_t $	-
Moving Average Convergence Divergence	$MACD_t = EMA(N_{\text{short}})_t - EMA(N_{\text{long}})_t$ where typically $N_{\text{short}} = 12$ and $N_{\text{long}} = 26$	-
Stochastic RSI	$StochRSI(N)_t = \frac{RSI(N)_t - \min(RSI(N))}{\max(RSI(N)) - \min(RSI(N))}$	[0, 1]
Stochastic Oscillator	$Stoch\ \%K(N)_t = \frac{Close_t - LL(N)_t}{HH(N)_t - LL(N)_t} \times 100$ $Stoch\ \%D(M)_t = SMA_{Stoch\ \%K(N)}(M)_t$ $HH(N)_t = \max_{i=t-N+1}^t (High_i)$, $LL(N)_t = \min_{i=t-N+1}^t (Low_i)$	[0, 100]
Ultimate Oscillator	$UO_t = \frac{4 \times \text{Avg}_t(N_1) + 2 \times \text{Avg}_t(N_2) + \text{Avg}_t(N_3)}{4 + 2 + 1} \times 100$ $\text{Avg}_t(N) = \frac{\sum_{i=0}^{N-1} BP_{t-i}}{\sum_{i=0}^{N-1} TR_{t-i}}$ $BP_t = Close_t - \min(Close_{t-1}, Low_t)$ $TR_t = \max(Close_{t-1}, High_t) - \min(Close_{t-1}, Low_t)$ where typically $N_1 = 7$, $N_2 = 14$, and $N_3 = 28$	[0, 100]
Williams %R	$Williams\ \%R(N)_t = \frac{HH(N)_t - Close_t}{HH(N)_t - LL(N)_t} \times -100$	[-100, 0]

Table A.3: Summary of frequently used Volume Technical Indicators [7].

Indicator Name	Formula	Range
Chaikin Accumulation/Distribution Line	$AD_t = AD_{t-1} + MFV_t$ $MFV_t = \frac{(Close_t - Low_t) - (High_t - Close_t)}{High_t - Low_t} \times Volume_t$	-
Chaikin Accumulation/Distribution Oscillator	$ADOSC_t = EMA_{AD}(N_{\text{short}})_t - EMA_{AD}(N_{\text{long}})_t$ where typically $N_{\text{short}} = 3$ and $N_{\text{long}} = 10$	-
On-Balance Volume	$OBV_t = \begin{cases} OBV_{t-1} + Volume_t & \text{if } Price_t > Price_{t-1} \\ OBV_{t-1} - Volume_t & \text{if } Price_t < Price_{t-1} \\ OBV_{t-1} & \text{otherwise} \end{cases}$	-

Table A.4: Summary of frequently used Volatility Technical Indicators [7].

Indicator Name	Formula	Range
Standard Deviation	$SD(N)_t = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (\text{Price}_i - \overline{\text{Price}})^2}$	-
Realised Volatility	$RV(N)_t = \sqrt{\sigma_t^2}$, where $\sigma_t^2 = \alpha \times r_t^2 + (1 - \alpha) \times \sigma_{t-1}^2$, $r_t = \ln\left(\frac{\text{Price}_t}{\text{Price}_{t-1}}\right)$ and $\alpha = \frac{1}{N}$	-
Bollinger Bands	Upper $BB(N, k)_t = SMA(N)_t + k \times SD(N)_t$ Lower $BB(N, k)_t = SMA(N)_t - k \times SD(N)_t$ where typically $k = 2$	-
Average True Range	$ATR(N)_t = \frac{(N-1) \times ATR(N)_{t-1} + TRANGE_t}{N}$ $TRANGE_t = \max(\text{High}_t - \text{Low}_t, \text{High}_t - \text{Close}_{t-1} , \text{Low}_t - \text{Close}_{t-1})$	-
Normalised Average True Range	$NATR(N)_t = \frac{ATR(N)_t}{\text{Close}_t} \times 100$	-
Chaikin Volatility Index	$CVI(N)_t = \frac{EMA_{HL}(N)_t - EMA_{HL}(N)_{t-N}}{EMA_{HL}(N)_{t-N}} \times 100$ $EMA_{HL}(N)_t = EMA(N)$ applied to $(\text{High}_t - \text{Low}_t)$	-

B

Reviewed Articles

Table B.1: Basic features of the related work articles.

Reference	Market	Dataset	Timeframe	Period	NN	RL	DRL	Key Findings
[18]	Forex	Raw Ticks	≈ 2 Hours	2010 - 2017	FNN	Critic-Only	DQN	Significantly outperformed the benchmark.
[19]	Forex	OHLCV	15 Minutes	2012 - 2017	RNN (LSTM)	Critic-Only	DRQN	Significantly outperformed the benchmark even with explicit and implicit costs.
[20]	Forex	OHLCV	Minute	2004 - 2018	RNN (LSTM)	Critic-Only	DRQN	Outperformed the benchmark, but with unusually high accuracy and maximum drawdown.
[43]	Stocks	OHLCV, News	Daily	≈ 5 Months	RNN (LSTM)	Actor-Critic	DDPG	Outperformed the benchmark. News features were important.
[42]	Stocks	OHLCV	Monthly	2005 - 2019	FNN	Actor-Critic	DDPG	Outperformed the benchmark even with explicit and implicit costs.
[47]	Stocks	OHLCV	Daily	2016 - 2020	FNN	Actor-Critic	DDPG, A2C, PPO	The ensemble underperformed the benchmark and had unstable returns.
[39]	Stocks, Futures	OHLCV	Minute	2010 - 2020	FNN (SDAE), RNN (LSTM)	Critic-Only, Actor-Critic	DQN, A3C	Both significantly outperformed the benchmark. A3C had better results and convergence.
[44]	Stocks, Cryptos	OHLCV	Daily	2010 - 2021	FNN	Actor-Critic	DDPG (TD3)	Outperformed the crypto benchmark but underperformed the stock benchmark.
[45]	Stocks	OHLCV	Daily	1992 - 2020	FNN	Critic-Only, Actor-Critic	DQN, DDPG, PPO, A2C	All outperformed the benchmark. PPO and DQN had better and more consistent results. DDPG and A2C did worse than expected.
[37]	Stocks	OHLCV	Daily	2006 - 2022	FNN	Critic-Only	DQN	Outperformed the benchmark and generalised well to out-of-sample assets.
[38]	Stocks	OHLCV	Daily	2012 - 2019	FNN	Critic-Only	DQN	Outperformed the benchmark with consistent results in stable and volatile markets.
[48]	Stocks	OHLCV	Daily	2008 - 2018	RNN (GRU)	Critic-Only, Actor-Critic	GDQN, GDPG	Both outperformed the benchmark with consistent results in stable and volatile markets. GDPG had better results.
[40]	Stocks	OHLCV	Daily	2009 - 2018	FNN	Actor-Critic	DDPG	Outperformed the benchmark.
[46]	Stocks	OHLCV	Daily	2009 - 2020	FNN	Actor-Critic	DDPG, A2C, PPO	The ensemble significantly outperformed the benchmark with explicit costs. After it, PPO had the best results, then A2C, then DDPG.
[41]	Stocks	OHLCV	Daily	2009 - 2021	FNN	Actor-Critic	DDPG, A2C	Both outperformed the benchmark. DDPG had better results, convergence and stability, due to experience replay and noise.
[49]	ETFs	OHLCV	Daily	2000 - 2018	FNN, RNN (LSTM)	Critic-Only	DQN, DRQN	Both outperformed the benchmark. DRQN had better results.
[51]	ETFs	OHLCV	Daily	2002 - 2020	RNN (LSTM)	Actor-Critic	DDPG	Outperformed the benchmark with variable position sizing, with explicit and implicit costs charged.
[50]	Futures	OHLCV	Daily	2005 - 2019	RNN (LSTM)	Actor-Critic	A2C	Outperformed the benchmark with explicit and implicit costs.

Table B.2: Advanced features of the related work articles.

Reference	State Space	Action Space	Reward Function	Bias/Variance	Exploration/Exploitation	Training Techniques
[18]	Ask, Bid, Volume, Technical Analysis	Discrete (1, 0, -1)	Total Return	Adjust Network, Early Stopping	Policy ϵ -greedy	Cross-Validation
[19]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Total Return	Adjust Replay Memory, Adjust RNN Sequence	Policy ϵ -greedy	Online Learning
[20]	OHLCV	Discrete (1, 0, -1)	Total Return	Adjust Network	-	Train-Validate-Test Split
[43]	OHLCV, Sentiment and Technical Analysis	Discrete (1, 0, -1)	Total Return	Adjust Network	Policy ϵ -greedy, Shuffling	Cross-Validation
[42]	Log Returns, Volume	Discrete (1, 0, -1)	Total Return	-	-	Pre-Train with GA
[47]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Sharpe Ratio	Ensemble Selection	Ensemble Selection	Cross-Validation
[39]	OHLCV, Metrics, Technical Analysis	Discrete (-n, ..., 0, ..., n)	Total Return	Denoising	Gaussian Noise	Adaptive Learning Rate
[44]	OHLCV	Continuous [-1, 1]	Log-Return	Normalisation, Regularisation	Policy ϵ -greedy Gaussian Noise	Train-Validate-Test Split
[45]	OHLCV	Discrete (1, 0, -1)	Sharpe Ratio	Wavelet Transform	-	Train-Validate-Test Split
[37]	OHLCV, Macroeconomic Metrics	Discrete (1, 0, -1)	Total Return, (1, 0, -1)	Normalisation, Regularisation	Policy ϵ -greedy (Adaptive)	Adaptive Synchronisation
[38]	OHLCV	Discrete (1, -1)	Total Return	Normalisation, Regularisation	Duplicate Environment (Inverse Positioning)	Adaptive Learning Rate, Huber Loss, Gradient Clip
[48]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Total Return	Normalisation, Regularisation	Policy ϵ -greedy Gaussian Noise	Train-Test Split
[40]	OHLCV	Discrete (1, 0, -1)	Total Return	Adjust Network	Gaussian Noise	Train-Validate-Test Split
[46]	OHLCV, Technical Analysis	Discrete (-n, ..., 0, ..., n)	Total Return (Turbulence Index)	Ensemble Selection	Ensemble Selection	Train-Validate-Test Split
[41]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Total Return	Normalisation	Gaussian Noise	Train-Test Split
[49]	OHLCV	Discrete (1, 0, -1)	Total Return	Adjust Learning Rate	-	-
[51]	OHLCV	Continuous (Position Sizing)	Sharpe Ratio	-	Gaussian Noise	Train-Validate-Test Split
[50]	OHLCV, Technical Analysis	Continuous [-1, 1]	Total Return (with Volatility)	Normalisation	-	Cross-Validation